

МОНГОЛ УЛСЫН ИХ СУРГУУЛЬ
МЭДЭЭЛЛИЙН ТЕХНОЛОГИ, ЭЛЕКТРОНИКИЙН СУРГУУЛЬ
МЭДЭЭЛЭЛ, КОМПЬЮТЕРЫН УХААНЫ ТЭНХИМ

Бал Хишигбаяр

МУИС-ИЙН ЭМНЭЛГИЙН МЭДЭЭЛЛИЙН СИСТЕМ

(NUM Hospital Management Information System)

Мэдээллийн технологи (D061303)
Бакалаврын судалгааны ажил

Улаанбаатар

2026 оны 05 дугаар сар

МОНГОЛ УЛСЫН ИХ СУРГУУЛЬ
МЭДЭЭЛЛИЙН ТЕХНОЛОГИ, ЭЛЕКТРОНИКИЙН СУРГУУЛЬ
МЭДЭЭЛЭЛ, КОМПЬЮТЕРЫН УХААНЫ ТЭНХИМ

МУИС-ИЙН ЭМНЭЛГИЙН МЭДЭЭЛЛИЙН СИСТЕМ
(NUM Hospital Management Information System)

Мэдээллийн технологи (D061303)
Бакалаврын судалгааны ажил

Удирдагч:	_____	Ахлах багш Б.Батням
Хамтран удирдагч:	_____	
Гүйцэтгэсэн:	_____	Б.Хишигбаяр (17b1num2078)

Улаанбаатар
2026 оны 05 дугаар сар

Зохиогчийн баталгаа

Миний бие Бал Хишигбаяр "МУИС-ИЙН ЭМНЭЛГИЙН МЭДЭЭЛЛИЙН СИСТЕМ" сэдэвтэй судалгааны ажлыг гүйцэтгэсэн болохыг зарлаж дараах зүйлсийг баталж байна:

- Ажил нь бүхэлдээ Монгол Улсын Их Сургуульд дээд боловсролын зэрэг горилохоор дэвшүүлсэн болно.
- Энэ ажлын аль нэг хэсгийг эсвэл бүхлээр нь ямар нэг их, дээд сургуулийн зэрэг горилохоор оруулж байгаагүй болно.
- Бусдын хийсэн ажлаас хуулбарлаагүй, эшлэл, зүүлтийг зохистой хийсэн болно.
- Ажлыг зохиогч би хийсэн ба миний хийсэн ажил, бусдын дэмжлэгийг дипломын ажилд тодорхой тусгасан болно.
- Ажилд тусалсан бүх эх сурвалжид талархаж байна.

Гарын үсэг: _____

Огноо: _____

ТОВЧИЛСОН ҮГИЙН ЖАГСААЛТ	ix
УДИРТГАЛ	1
1. СУДАЛГАА	3
1.1 МУИС-ийн эмнэлгийн тухай	3
1.2 Судалгааны зорилго, арга зүй ба хамрах хүрээ	5
1.3 Өмнөх ажлуудын харьцуулалт	8
1.4 Ижил төстэй системийн судалгаа	11
1.5 Технологийн сонголтын ерөнхий үндэслэл	14
1.6 Бүлгийн дүгнэлт	18
2. СИСТЕМИЙН ШИНЖИЛГЭЭ	19
2.1 Суурь нөхцөл	19
2.2 Бизнес процессын шинжилгээ	19
2.3 Системийн шаардлага	21
2.4 Хэрэглэгчийн дүр (User Roles) ба RBAC	31
2.5 Use Case загварчлал	32
2.6 Процесс загварчлал	35
2.7 Статик загварчлал	39
2.8 Бүлгийн дүгнэлт	40
3. СИСТЕМИЙН ЗОХИОМЖ	41
3.1 Системийн архитектур	41
3.2 Өгөгдлийн сангийн зохиомж	44
3.3 Модуль хоорондын API урсгалын зохиомж	55
3.4 Нэвтрэлт ба аюулгүй байдлын зохиомж	59
3.5 Дэлгэцийн зохиомж	63
3.6 Бүлгийн дүгнэлт	65

4. ХЭРЭГЖҮҮЛЭЛТ	66
4.1 Хөгжүүлэлтийн орчин ба төслийн бүтэц	66
4.2 Backend хэрэгжүүлэлт	68
4.3 Patient frontend хэрэгжүүлэлт	82
4.4 Admin frontend хэрэгжүүлэлт	86
4.5 Apollo Client ба frontend shared тохиргоо	93
4.6 Өгөгдлийн сангийн нөөцлөлт ба сэргээх зохион байгуулалт	94
4.7 Хэрэглэгчийн сэтгэл ханамжийн судалгаа	99
4.8 Хэрэгжүүлэлтийн хамрах хүрээ	103
4.9 Системийг ажиллуулах ба баталгаажуулах командууд	105
4.10 Функциональ тестийн хувилбарууд	107
4.11 Хэрэгжүүлэлтийн хязгаарлалт ба цаашдын ажил	112
4.12 Бүлгийн дүгнэлт	112
ДҮГНЭЛТ	114
НОМ ЗҮЙ	116
ХАВСРАЛТ	118
A. GraphQL Schema	119
A.1 Root Query ба Mutation	119
B. Кодын хэрэгжүүлэлт	122
B.1 Backend — Appointment Service	122
B.2 Frontend — useAuth Hook	123
C. Өгөгдлийн загвар — нэмэлт загварууд	126
C.1 Vital Signs загвар	126
C.2 Diagnosis загвар	127

C.3	Prescription загвар	128
C.4	Audit Log загвар	130

Зургийн жагсаалт

1.1	Технологийн стекийн бүтэц	17
1.2	Өмнөх судалгааны ажлуудын уялдааны зураглал	18
2.1	RBAC — 4 үндсэн дүрийн хандалтын эрхийн загвар	31
2.2	Системийн Use Case диаграм	33
2.3	Цаг товлох процессын Activity диаграм.....	36
2.4	Үзлэгийн процессын swimlane төрлийн үйл ажиллагааны диаграмм ...	38
3.1	Системийн ерөнхий архитектурын диаграм	41
3.2	Өгөгдлийн сангийн ER диаграм (гол collection-ууд)	53
3.3	API хүсэлтийн урсгал.....	56
3.4	Appointment Status — Төлвийн шилжилтийн диаграм	59
3.5	СИСИ OAuth 2.0 Authorization Code Grant — Sequence Diagram	61
4.1	Patient portal — хяналтын самбар.....	84
4.2	Patient portal — цаг захиалах workflow-ийн profile баталгаажуулах алхам	85
4.3	Patient portal — өвчтөний цаг товлолтын жагсаалт	86
4.4	Admin portal — хяналтын самбар.....	89
4.5	Admin portal — өвчтөний жагсаалт, хайлт	90
4.6	Admin portal — цаг товлолтын удирдлага	91
4.7	Admin portal — үйлчилгээ болон нөөцийн удирдлага	92
4.8	Admin portal — сарын тайлангийн дэлгэц	93

Хүснэгтийн жагсаалт

1.1	Эмнэлгийн оролцогч талууд ба тэдний үүрэг	4
1.2	Өмнөх ажлуудын судалгааны орон зайн харьцуулалт	10
1.3	Ижил төстэй системүүдийн харьцуулалт	13
1.4	Өгөгдлийн сангийн сонголтын үндэслэл	16
2.1	Нэвтрэлтийн модулийн функциональ шаардлага	22
2.2	Өвчтөний модулийн функциональ шаардлага	23
2.3	Цаг товлолтын модулийн функциональ шаардлага	23
2.4	Үзлэгийн модулийн функциональ шаардлага	24
2.5	Оношлогоо ба жорын модулийн функциональ шаардлага	25
2.6	Үйлчилгээ ба нөөцийн хуваарилалтын модулийн функциональ шаард- лага	26
2.7	Сэтгэл ханамжийн судалгааны модулийн функциональ шаардлага	27
2.8	Тайлан, аудит ба мэдэгдлийн модулийн функциональ шаардлага	28
2.9	Функциональ бус шаардлага	29
2.10	Шаардлага–use case–хэрэгжүүлэлт–тестийн уялдаа	30
2.11	RBAC — Хэрэглэгчийн дүр ба эрхүүд	32
2.12	Системийн гол Use Case-ууд	34
3.1	Collection-уудын хамрах хүрээний ангилал	45
3.2	Users collection-ий бүтэц	47
3.3	Patients collection-ий бүтэц	48
3.4	Appointments collection-ий бүтэц	49
3.5	Visits collection-ий бүтэц	50
3.6	Satisfaction survey templates collection-ий бүтэц	51
3.7	Satisfaction surveys collection-ий бүтэц	52
3.8	Үндсэн асуулгуудыг дэмжих индексүүд	54

3.9	Нэвтрэлт ба session-ийн аюулгүй байдлын зохиомж	60
3.10	RBAC permission matrix-ийн жишээ	63
4.1	Хөгжүүлэлтийн үндсэн орчин	67
4.2	GraphQL API-ийн domain-ууд	71
4.3	Backend model болон collection-ууд	73
4.4	Үйлчилгээ ба нөөцийн scheduling загвар	77
4.5	MongoDB backup workflow-ийн алхамууд	96
4.6	Backup түлхүүрийн хадгалалтын зарчим	97
4.7	Сэтгэл ханамжийн үнэлгээний онооны тайлбар	100
4.8	Сэтгэл ханамжийн судалгааны хадгалах өгөгдөл	103
4.9	Модулиудын хэрэгжүүлэлтийн хамрах хүрээ	104
4.10	Функциональ тестийн баталгаажуулсан хувилбарууд	107
4.11	Автоматжуулсан тестийн нэгтгэл	111
4.12	GraphQL performance smoke test-ийн хэмжилт	111

Кодын жагсаалт

3.1	Төслийн файлын бүтэц	42
4.1	Source code-ийн үндсэн бүтэц	67
4.2	Backend server-ийн үндсэн тохиргоо	68
4.3	Appointment model-ийн resource-based scheduling талбарууд	74
4.4	HttpOnly cookie-д суурилсан JWT шалгалт ба context үүсгэх хэлбэр	75
4.5	Цагийн боломж шалгах үндсэн санаа	78
4.6	Patient frontend-ийн үндсэн route-ууд	82
4.7	Admin frontend-ийн үндсэн route-ууд	86
4.8	Apollo Client cookie session-ийн тохиргоо	93
4.9	Өдөр бүр 03:00 цагт backup ажиллуулах cron тохиргооны жишээ	98
4.10	Survey template version history-ийн жишээ	99
4.11	Survey required шалгах бизнес дүрэм	101
4.12	Template delete/archive дүрэм	101
4.13	Backend ажиллуулах команд	105
4.14	Frontend ажиллуулах командууд	106
4.15	Build шалгах командууд	106
4.16	Health endpoint шалгах жишээ	106
4.17	Автомат тест ажиллуулсан командууд	109
A.1	Root Query type (хэсэгчлэн)	119
A.2	Root Mutation type (хэсэгчлэн)	120
B.1	getAvailableSlots service	122
B.2	useAuth custom hook	123
C.1	VitalSign model	126
C.2	Diagnosis model	127
C.3	Prescription model	128
C.4	AuditLog model	130

Товчилсон үгийн жагсаалт

Товчлол	Тайлбар
HMIS	Hospital Management Information System буюу эмнэлгийн мэдээллийн систем
API	Application Programming Interface буюу програм хоорондын өгөгдөл солилцох интерфейс
GraphQL	Клиент шаардлагатай өгөгдлөө query/mutation хэлбэрээр авах API технологи
RBAC	Role-Based Access Control буюу хэрэглэгчийн дүрд суурилсан эрхийн хяналт
JWT	JSON Web Token буюу нэвтрэлтийн token-д суурилсан баталгаажуулалт
OAuth 2.0	Гуравдагч талын нэвтрэлт, зөвшөөрлийн стандарт урсгал
OTP	One-Time Password буюу нэг удаагийн баталгаажуулах код
ICD	International Classification of Diseases буюу өвчний олон улсын ангилал
S3	Amazon Simple Storage Service буюу object storage үйлчилгээ
UAT	User Acceptance Test буюу хэрэглэгчийн хүлээн зөвшөөрөх тест

УДИРТГАЛ

Монгол Улсын Их Сургуулийн (МУИС) эмнэлэг нь оюутан, багш, ажилтнуудад анхан шатны эрүүл мэндийн тусламж, үйлчилгээ үзүүлдэг байгууллага юм. Эмнэлгийн өдөр тутмын үйл ажиллагаанд өвчтөний бүртгэл, цаг товлолт, үзлэг, оношлогоо, жор, үйлчилгээний хуваарь, тайлан зэрэг олон төрлийн мэдээлэл зэрэгцэн үүсдэг боловч эдгээрийг цаасан бүртгэл болон тусгаарлагдсан хэлбэрээр удирдах нь мэдээллийн давхардал, хайлтын хүндрэл, тайлан гаргалтын удаашрал, хандалтын хяналтын сул талыг бий болгож байна.

Энэхүү бакалаврын судалгааны ажлын зорилго нь МУИС-ийн эмнэлгийн үндсэн үйл ажиллагааг нэгтгэн авч үзэж, модульчилсан бүтэцтэй эмнэлгийн мэдээллийн системийн шинжилгээ, зохиомж, хэрэгжүүлэлтийн суурь хувилбарыг боловсруулахад оршино. Судалгаанд баримт бичгийн шинжилгээ, өмнөх ажлуудын харьцуулсан судалгаа, шаардлагын шинжилгээ, системийн загварчлал, прототип хэрэгжүүлэлт ба функциональ тестлэлтийн аргыг ашиглав. Мөн С.Оюунбилэгийн бизнес процесс ба шаардлагын шинжилгээ (2019), Н.Наранчимэгийн үзлэгийн програм (2023), Э.Мягмарсүрэний эмчилгээний програм (2023), Ө.Агиймаагийн архитектурын судалгаа (2025)-ыг хамрах хүрээ, арга зүй, модуль хоорондын уялдаа, хэрэгжүүлэлтийн түвшнээр харьцуулан авч үзэв.

Ажлын хүрээнд нэвтрэлт ба хэрэглэгчийн эрх, өвчтөний бүртгэл, цаг товлолт ба нөөцийн хуваарилалт, эмчийн үзлэг, оношлогоо, жор, үйлчилгээний удирдлага, тайлан, аудитын модулиудыг тодорхойлж, тэдгээрийн өгөгдлийн загвар, бизнес процесс, хэрэглэгчийн дүр, хандалтын эрхийг боловсруулсан. Ингэснээр өмнөх судалгаануудын хэсэгчилсэн үр дүнг нэг системийн зохиомжид нэгтгэх, эмнэлгийн үйлчилгээний тасралтгүй урсгалыг дэмжих боломж бүрдсэн.

Хэрэгжүүлэлтийн хэсэгт системийг веб орчинд ажиллах гурван хэсэгтэй бүтэцтэйгээр хөгжүүлсэн: өвчтөний интерфэйс, эмнэлгийн ажилтны интерфэйс, сервер талын үйлчилгээ. Нэвтрэлт, өвчтөн, цаг товлолт, үзлэг, оношлогоо, жорын үндсэн урсгалыг хэрэгжүүлж, тайлан, аудит, мэдэгдэл, үйлчилгээ ба нөөцийн хуваарилалт, encrypted backup/restore болон

template/version бүхий хэрэглэгчийн сэтгэл ханамжийн судалгааны урсгалыг тусгав. Мөн шаардлага, use case, хэрэгжүүлэлт, функциональ тест, backup баталгаажуулалт болон хэрэглэгчийн үнэлгээний уялдааг нэгтгэн харуулсан.

Түлхүүр үгс: Эмнэлгийн мэдээллийн систем, МУИС-ийн эмнэлэг, модульчилсан зохиомж, бизнес процесс, өвчтөний бүртгэл, цаг товлолт, үзлэг, RBAC, backup, сэтгэл ханамжийн судалгаа

1. СУДАЛГАА

Энэ бүлэгт МУИС-ийн эмнэлгийн одоогийн нөхцөл байдал, өмнөх судалгааны ажлуудын шинжилгээ, ижил төстэй системүүдийн судалгаа, технологийн сонголтын судалгааг тусгасан.

1.1 МУИС-ийн эмнэлгийн тухай

Монгол Улсын Их Сургуулийн эмнэлэг нь 1942 онд байгуулагдсан бөгөөд МУИС-ийн оюутан, багш нар, ажилтнуудад анхан шатны эрүүл мэндийн тусламж, үйлчилгээ үзүүлдэг байгууллага юм. Эмнэлэг нь дараах үндсэн үйл ажиллагааг явуулдаг:

- Амбулаторийн үзлэг, оношлогоо
- Эмчилгээ, процедур
- Жор бичих, эмийн зөвлөгөө
- Урьдчилан сэргийлэх үзлэг
- Лабораторийн шинжилгээ (хязгаарлагдмал)
- Сэтгэл зүйн зөвлөгөө

Эмнэлгийн үйл ажиллагаанд дараах оролцогч талууд (stakeholders) оролцдог:

Хүснэгт 1.1: Эмнэлгийн оролцогч талууд ба тэдний үүрэг

№	Оролцогч	Үүрэг
1	Өвчтөн (Оюу-тан/Багш/Ажилтан)	МУИС-ийн имэйл/СИСИ мэдээлэл эсвэл бүртгэлтэй хэрэглэгчийн мэдээллээр баталгаажин системд нэвтрэх, өөрөө цаг товлох, үзлэгт хамрагдах, жор болон үзлэгийн түүхээ харах
2	Эмч	Үзлэг хийх, оношлогоо тавих, жор бичих, эмчилгээ болон тоног төхөөрөмжийн үйлчилгээний зөвлөгөө өгөх
3	Сувилагч	Эмчийн зөвлөгөөний дагуу тариа, эмчилгээ болон төхөөрөмжийн цаг авсан үйлчлүүлэгчид үйлчилгээ үзүүлэх
4	Системийн админ	Ажилтан, үйлчилгээ, тоног төхөөрөмжийн нөөц, цагийн боломж болон системийн үндсэн тохиргоог удирдах

1.1.1 Одоогийн нөхцөл байдал

МУИС-ийн эмнэлгийн одоогийн үйл ажиллагааны мэдээллийг өмнөх судалгааны ажлууд болон МУИС-ийн эмнэлгийн эмч З.Гаравсүрэнгээс авсан мэдээлэлд тулгуурлан нэгтгэв [1, 2, 3, 4]. Эмнэлэгт тусдаа цаг бүртгэгч буюу бүртгэлийн ажилтан ажилладаггүй бөгөөд энэ системийн зорилтот хувилбарт өвчтөн өөрөө шууд цаг товлох боломжтой байна. Мөн эмнэлэгт өмнөх хэсгүүдэд бүрэн дурдагдаагүй тоног төхөөрөмжийн үйлчилгээний нөөц бий. Тухайлбал массажны сандал, шарлагын аппарат зэрэг нийт 8 нэр төрлийн аппарат, төхөөрөмжийн цагийг үйлчилгээний нөөцтэй уялдуулан бүртгэх шаардлагатай.

Эмнэлгийн бодит зохион байгуулалтад тусдаа цаг бүртгэгч эсвэл өгөгдөл оруулагч ажилтан байхгүй тул системийн шинжилгээнд ийм тусдаа хэрэглэгчийн дүр тодорхойлоогүй. Өвч-

төн өөрөө цаг товлох, эмч болон системийн админ шаардлагатай үед бүртгэл, төлөвийн мэдээллийг засварлах зарчмаар нарийсгав.

Одоогийн нөхцөл байдлаас дараах асуудлууд ажиглагдаж байна:

1. **Мэдээллийн давхардал ба алдагдал:** Цаасан болон салангид бүртгэл нь гэмтэх, алдагдах, давхардах эрсдэлтэй
2. **Хайлт хийх хүндрэл:** Өвчтөний өмнөх үзлэгийн түүх, эмчилгээ, тоног төхөөрөмжийн үйлчилгээний мэдээллийг хурдан хайж олоход хүндрэлтэй
3. **Статистик гаргах хүндрэл:** Тайлан, судалгааны мэдээлэл цуглуулахад гар ажиллагаа их шаарддаг
4. **Цаг товлолтын нэгдсэн бүртгэл дутмаг:** Өвчтөн өөрөө цаг товлох, эмчийн үзлэг болон төхөөрөмжийн үйлчилгээний цагийг нэг дор удирдах автоматжуулалт шаардлагатай
5. **Мэдээллийн аюулгүй байдал:** Эмнэлгийн мэдээлэлд хандсан үйлдлийг бүртгэх, эрхээр хянах боломж хязгаарлагдмал
6. **СИСИ системтэй бүрэн уялдаагүй:** Их сургуулийн мэдээллийн системээс хэрэглэгчийн суурь мэдээллийг автоматаар авах нэгтгэл шаардлагатай

1.2 Судалгааны зорилго, арга зүй ба хамрах хүрээ

Энэ ажил нь зөвхөн програмын дэлгэц, код бичих ажлаар хязгаарлагдахгүй, МУИС-ийн эмнэлгийн үндсэн үйлчилгээний мэдээллийн урсгалыг шинжилж, түүнийг нэг системийн зохиомжид буулгах зорилготой. Иймээс судалгааны асуудлыг дараах байдлаар тодорхойлов.

1.2.1 Судалгааны асуудал

Одоогийн цаасан болон салангид бүртгэл нь өвчтөний мэдээлэл давхардах, өмнөх үзлэгийн түүхийг хурдан хайх боломжгүй болох, цаг товлолт ба дарааллын мэдээлэл бодит цагт

харагдахгүй байх, эмчийн үзлэгийн дүн тайлан болон статистикт шууд ашиглагдахгүй байх, мөн эмзэг мэдээлэлд хэн хандсаныг бүртгэх боломж хязгаарлагдах зэрэг асуудлыг үүсгэж байна. Эдгээр асуудлыг зөвхөн нэг модулийн програм хийснээр бус, өвчтөн–цаг товлолт–үзлэг–оношлогоо–жор–тайлан гэсэн дараалсан өгөгдлийн урсгалыг нэгтгэсэн зохиомжоор шийдвэрлэх шаардлагатай.

1.2.2 Судалгааны зорилго ба зорилтууд

Судалгааны зорилго нь МУИС-ийн эмнэлгийн үндсэн үйл ажиллагаанд тохирсон, хэрэглэгчийн эрхийн ялгаатай, модульчилсан бүтэцтэй эмнэлгийн мэдээллийн системийн шинжилгээ, зохиомж, хэрэгжүүлэлтийн суурь хувилбарыг боловсруулахад оршино.

Энэхүү зорилгыг хэрэгжүүлэхийн тулд дараах зорилтуудыг дэвшүүлэв:

1. МУИС-ийн эмнэлгийн одоогийн бүртгэл, цаг товлолт, үзлэгийн үндсэн процессыг шинжлэх;
2. Өмнөх бакалаврын судалгааны ажлууд болон ижил төстэй HMIS системүүдийн хамрах хүрээ, давуу болон сул талыг харьцуулах;
3. Системийн функциональ болон функциональ бус шаардлагуудыг тодорхойлох;
4. Өвчтөн, эмч, сувилагч, системийн админы хэрэглэгчийн дүр ба эрхийн загварыг боловсруулах;
5. Өгөгдлийн сан, API, нэвтрэлт, аюулгүй байдал, дэлгэцийн зохиомжийг боловсруулах;
6. Гол хэрэглэгчийн урсгалуудыг веб орчинд хэрэгжүүлж, функциональ тестийн хувилбаргаар шалгах.

1.2.3 Судалгааны объект, судлах зүйл

Судалгааны объект нь МУИС-ийн эмнэлгийн өвчтөнд үзүүлэх анхан шатны тусламж, үйлчилгээний мэдээллийн урсгал юм. **Судлах зүйл** нь уг урсгалыг дэмжих мэдээллийн сис-

темийн шаардлага, өгөгдлийн загвар, хэрэглэгчийн эрхийн зохиомж, API болон веб хэрэгжүүлэлтийн шийдэл байна.

1.2.4 Судалгааны арга зүй

Ажлын хүрээнд дараах арга зүйг ашиглав:

- **Баримт бичгийн шинжилгээ:** Өмнөх бакалаврын судалгааны ажлууд, HMIS системийн баримт бичиг, ашигласан технологийн албан ёсны материалуудыг харьцуулан судлах [24, 17, 18, 19];
- **Харьцуулсан судалгаа:** OpenMRS, GNU Health, HospitalRun зэрэг системүүдийг хамрах хүрээ, өгөгдлийн сан, өргөтгөх боломж, нэвтрэлт, орон нутгийн хэрэглээнд нийцэх байдлаар харьцуулах [17, 18, 19];
- **Шаардлагын шинжилгээ:** Эмнэлгийн үндсэн workflow, эмч З.Гаравсүрэнгээс авсан мэдээлэл, цаг товлолт болон тоног төхөөрөмжийн үйлчилгээний бодит хэрэгцээнд тулгуурлан функциональ болон функциональ бус шаардлагуудыг модуль тус бүрээр тодорхойлох;
- **Загварчлал:** Use case, activity diagram, RBAC, ER diagram, API урсгалын диаграмаар системийн бүтэц, үйл ажиллагааг дүрслэх;
- **Прототип хэрэгжүүлэлт ба тестлэлт:** Гол модулиудыг хэрэгжүүлж, API болон функциональ тестийн хувилбаруудаар шалгах.

1.2.5 Хамрах хүрээ ба хязгаарлалт

Энэ ажилд өвчтөний бүртгэл, өөрөө цаг товлдох урсгал, үзлэг, оношлогоо, жор, хэрэглэгчийн эрх, СИСИ OAuth нэвтрэлтийн бэлтгэл, утасны дугаарт мессеж илгээх гуравдагч талын үйлчилгээ, ICD-11 Docker API-ийн нэгтгэл, мэдэгдэл, аудит, backup/restore болон хэрэглэгчийн сэтгэл ханамжийн судалгааны урсгалыг авч үзэв. Харин лабораторийн бүрэн workflow,

эмийн сангийн нөөцийн удирдлага, төлбөр тооцоо, HL7/FHIR зэрэг гадаад эрүүл мэндийн системтэй өгөгдөл солилцох бүрэн интеграцийг энэ хувилбарт хэрэгжүүлээгүй бөгөөд цаашдын хөгжүүлэлтийн чиглэлд тусгав.

Мөн хэрэглэгчийн сэтгэл ханамжийн судалгааг системээр авах боломжийг бүрдүүлсэн. Харин production орчны performance хэмжилт, автоматжуулсан unit/integration test болон олон хэрэглэгчтэй давтан үнэлгээг дараагийн шатанд илүү нарийвчлах шаардлагатай.

1.2.6 Практик ач холбогдол

Системийн зохиомж хэрэгжсэнээр өвчтөн, цаг товлолт, үзлэг, оношлогоо, жорын мэдээлэл нэг өгөгдлийн урсгалд холбогдож, өвчтөний түүх хайх, эмчийн дараалал харах, цаг товлолтын давхардал шалгах, хэрэглэгчийн эрхээр хандах, тайлангийн өгөгдөл цуглуулах зэрэг үйл ажиллагааг автоматжуулах боломж бүрдэнэ.

1.3 Өмнөх ажлуудын академик харьцуулалт

МУИС-ийн эмнэлгийн мэдээллийн системийн чиглэлээр өмнө нь хэд хэдэн бакалаврын судалгааны ажил хийгдсэн [1, 2, 3, 4]. Энэ хэсэгт тэдгээр ажлыг судалгааны хамрах хүрээ, бизнес процессын загварчлал, модульчилсан зохиомж, өгөгдлийн уялдаа, хэрэгжүүлэлтийн түвшин гэсэн шалгуураар авч үзнэ. Харьцуулалтын үр дүнд өмнөх ажлуудын хувь нэмэр, хязгаарлалт, энэ ажлын судалгааны орон зай тодорхойлогдоно.

1.3.1 С.Оюунбилэг — МУИС-ийн эмнэлгийн мэдээллийн системийн шинжилгээ (2019)

Энэ ажил нь эмнэлгийн тухайн үеийн бизнес процессыг шинжилж, системийн шаардлагыг тодорхойлсон судалгаа юм [1]. Гол агуулга:

- Эмнэлгийн бизнес процессыг BPMN диаграмаар загварчилсан
- MoSCoW аргаар шаардлагыг эрэмблэсэн

- Объект хандлагат шинжилгээний загвар (UML) боловсруулсан
- Амбулаторийн үзлэг, оношлогоо, эмчилгээ, халдвар хамгааллын процессыг тодорхойлсон

Хязгаарлалт: Судалгааны гол үр дүн нь шинжилгээ, шаардлага, загварчлалын түвшинд төвлөрсөн тул ажиллах програм хангамжийн хэрэгжүүлэлт, модуль хоорондын өгөгдлийн урсгал, хэрэглэгчийн эрхийн нарийвчилсан зохиомжийг бүрэн хамраагүй.

1.3.2 Н.Наранчимэг — МУИС-ийн эмнэлэг: Үзлэгийн програм (2023)

Энэ ажил нь үзлэгийн бүртгэлийн процессыг програм хангамжийн хэлбэрээр хэрэгжүүлэхэд чиглэсэн [2]:

- React, Node.js, MongoDB технологи дээр суурилсан
- Өвчтөний бүртгэл, үзлэг бүртгэх функцийг хэрэгжүүлсэн
- Функциональ болон процесс загварчлал хийсэн
- Үзлэгийн бүртгэлийн хэрэглэгчийн урсгалыг тодорхойлсон

Хязгаарлалт: Судалгааны хамрах хүрээ нь голчлон үзлэгийн бүртгэлд төвлөрсөн тул цаг товлолт, үйлчилгээний нөөцийн хуваарилалт, жор, тайлан, хэрэглэгчийн эрхийн ялгаа зэрэг системийн бусад чухал хэсэгтэй бүрэн уялдаагүй.

1.3.3 Э.Мягмарсүрэн — МУИС-ийн эмнэлэг: Эмчилгээний програм (2023)

Энэ ажил нь эмчилгээний захиалга, гүйцэтгэлийн процессыг авч үзсэн [3]:

- Эмчилгээний захиалга, гүйцэтгэл, хяналтын модуль
- Ижил технологийн стек (React, Node.js, MongoDB)

- Сервис нэгтгэлийн зохиомж

Хязгаарлалт: Эмчилгээний модулийг тусад нь авч үзсэн боловч үзлэг, цаг товлот, өвчтөний түүх, тайлангийн модулиудтай нэг өгөгдлийн загвар дээр бүрэн нэгтгээгүй. Хэрэглэгчийн дүр, эрхийн хяналтын загвар мөн хангалттай нарийвчлагдаагүй.

1.3.4 Ө.Агиймаа — Цэвэр архитектур ба түүний хэрэгжүүлэлт (2025)

Энэ ажил нь Clean Architecture зарчмыг МУИС-ийн эмнэлгийн системийн жишээн дээр авч үзсэн [4, 20]:

- Архитектурын давхаргууд (Entity, Use Case, Interface Adapter, Framework) тодорхойлсон
- Dependency Inversion зарчмыг хэрэгжүүлсэн
- Кодын давхарга хоорондын хамаарлыг багасгах чиглэлийг авч үзсэн
- Тестлэх боломжтой бүтэц боловсруулах асуудлыг тусгасан

Хязгаарлалт: Архитектурын зарчим, кодын зохион байгуулалтад төвлөрсөн тул эмнэлгийн бизнес процессын бүх модулийг хамарсан бүтээгдэхүүний түвшний хэрэгжүүлэлт, үйлчилгээний урсгалын бүрэн зохиомжийг гол зорилго болгоогүй.

1.3.5 Өмнөх ажлуудын харьцуулсан шинжилгээ

Хүснэгт 1.2: Өмнөх ажлуудын судалгааны орон зайн харьцуулалт

Ажил	Хүчтэй тал	Хязгаарлалт	Энэ ажлын авч үзэх чиглэл
С.Оюунбилэг, 2019	Бизнес процесс, шаардлага, UML загварчлалын суурь боловсруулсан	Ажиллах програм хангамж, модуль хоорондын өгөгдлийн урсгал бүрэн тусгагдаагүй	Процессын шинжилгээг бодит веб системийн өгөгдлийн урсгалтай холбож үзэх

Ажил	Хүчтэй тал	Хязгаарлалт	Энэ ажлын авч үзэх чиглэл
Н.Наранчимэг, 2023	Өвчтөн бүртгэл, үзлэгийн бүртгэлийн урсгалыг хэрэгжүүлсэн	Цаг товлолт, жор, тайлан, эрхийн ялгаа зэрэг бусад модультай уялдаа сул	Үзлэгийг өвчтөний түүх, оношлогоо, жор, цаг товлолттой уялдуулах
Э.Мягмарсүрэн, 2023	Эмчилгээний захиалга, гүйцэтгэлийн процессыг авч үзсэн	Эмчилгээний модуль нь үзлэг, өвчтөний түүх, тайлантай нэг өгөгдлийн загвараар бүрэн холбогдоогүй	Эмчилгээ, үйлчилгээ болон төхөөрөмжийн цагийг appointment ба visit урсгалтай холбох
Ө.Агиймаа, 2025	Clean Architecture, dependency inversion, тестлэх боломжийг судалсан	Архитектурын зарчимд төвлөрсөн, эмнэлгийн бүх workflow-ийн бүтээгдэхүүний түвшний хэрэгжүүлэлт биш	Модульчилсан архитектурыг эмнэлгийн үндсэн workflow-ийн суурь хэрэгжүүлэлттэй уялдуулах
Энэ ажил, 2026	Өмнөх ажлуудын үр дүнг нэгтгэн өвчтөн-цаг-үзлэг-онош-жорын урсгалыг нэг зохиомжид авч үзсэн	Production орчны бүрэн туршилт, хэрэглэгчийн өргөн үнэлгээ нэмэлтээр шаардлагатай	МУИС-ийн эмнэлэгт тохирсон нэгдсэн HMIS-ийн шинжилгээ, зохиомж, суурь хэрэгжүүлэлтийн хувилбар боловсруулах

Хүснэгтээс харахад өмнөх ажлууд нь МУИС-ийн эмнэлгийн мэдээллийн системийн тодорхой хэсгийг тус тусад нь судалсан байна. Энэ ажил нь тэдгээр судалгааны үр дүнг үгүйсгэх бус, харин өвчтөний бүртгэлээс эхлэн цаг товлолт, үзлэг, оношлогоо, жор, үйлчилгээ, тайлан хүртэлх өгөгдлийн урсгалыг нэг зохиомжид уялдуулах боломжийг судална.

1.4 Ижил төстэй системийн судалгаа

Дэлхий дахинд эмнэлгийн мэдээллийн систем (Hospital Management Information System — HMIS) өргөн хэрэглэгддэг [24]. Энэ хэсэгт нээлттэй эхийн болон арилжааны системүүдийг авч үзнэ.

1.4.1 OpenMRS

OpenMRS (Open Medical Record System) нь 2004 оноос хөгжүүлэгдэж буй нээлттэй эхийн эмнэлгийн бүртгэлийн систем юм [17]. Голчлон хөгжиж буй орнуудын эмнэлгүүдэд зориу-

лагдсан.

- **Хамрах хүрээ:** Өвчтөний бүртгэл, үзлэг, эмнэлгийн бүртгэлийн үндсэн процесс
- **Зохиомжийн онцлог:** Модуль нэмэх боломжтой бүтэц, олон хэлний дэмжлэг
- **Энэ ажилд хамаарах санаа:** Өвчтөний түүх болон үзлэгийн мэдээллийг нэг бүртгэлийн загварт холбох зарчим

1.4.2 GNU Health

GNU Health нь Нэгдсэн Үндэстний Байгууллагын дэмжлэгтэй хөгжүүлэгдсэн нээлттэй эхийн эрүүл мэндийн мэдээллийн систем юм [18].

- **Хамрах хүрээ:** Эрүүл мэндийн бүртгэл, нийгмийн эрүүл мэнд, лабораторийн мэдээлэл
- **Зохиомжийн онцлог:** Оношийн кодчилал, тайлангийн өгөгдлийг системтэй хадгалах бүтэц
- **Энэ ажилд хамаарах санаа:** Оношлогоо болон тайлангийн модулийг тусдаа өгөгдлийн загвартай авч үзэх шаардлага

1.4.3 HospitalRun

HospitalRun нь эмнэлгийн өдөр тутмын workflow-ийг веб орчинд хэрэгжүүлэхэд чиглэсэн нээлттэй эхийн систем юм [19].

- **Хамрах хүрээ:** Өвчтөн, цаг товлолт, эмнэлгийн үйлчилгээний workflow
- **Зохиомжийн онцлог:** Хэрэглэгчийн интерфэйсийг эмнэлгийн ажилбарын дараалалтай уялдуулах хандлага
- **Энэ ажилд хамаарах санаа:** Өвчтөн болон эмнэлгийн ажилтны урсгалыг тусдаа интерфэйсээр зохион байгуулах шаардлага

1.4.4 Харьцуулалт

Хүснэгт 1.3: Ижил төстэй системүүдийн харьцуулалт

Шалгуур	OpenMRS	GNU Health	HospitalRun	NUM HMS
Хамрах хүрээ	Өвчтөний бүртгэл, үзлэг	Нийгмийн эрүүл мэнд, лаборатори	Эмнэлгийн өдөр тутмын workflow	МУИС-ийн эмнэлгийн үйлчилгээ
Өргөтгөх боломж	Өндөр, модуль нэмдэг	Өндөр	Дунд	Модуль нэмэх боломжтой
Өгөгдлийн сан	MySQL	PostgreSQL	CouchDB	MongoDB
Нэвтрэлт	Дотоод хэрэглэгч	Дотоод хэрэглэгч	Дотоод хэрэглэгч	СИСИ OAuth болон дотоод хэрэглэгч
Орон нутгийн нийцэл	Олон улсын ерөнхий хэрэглээ	Нийгмийн эрүүл мэндийн чиглэл хүчтэй	Ерөнхий эмнэлгийн workflow	МУИС-ийн оюутан, багш, ажилтны workflow-д чиглэсэн
Монгол хэл	Үгүй	Үгүй	Үгүй	Тийм
Судалгаанд авах санаа	Patient history	ICD/тайлангийн бүтэц	Workflow ба UI урсгал	Нэгдсэн зохиомж

1.5 Технологийн сонголтын ерөнхий үндэслэл

Энэ хэсэгт системийн зохиомжийг хэрэгжүүлэхэд шаардагдах ерөнхий техникийн орчныг товч авч үзнэ. Технологийн нарийвчилсан тохиргоо, API, frontend болон backend хэрэгжүүлэлтийг 4-р бүлэгт тусад нь тайлбарласан.

1.5.1 *TypeScript*

TypeScript нь Microsoft-ийн бүтээсэн JavaScript-ийн superset хэл юм [9]. Статик төрөл шалгалт нь кодын чанарыг сайжруулж, алдааг эрт илрүүлэх боломж олгодог. Энэ төсөлд TypeScript-ийг backend болон frontend хоёуланд нь ашигласан нь кодын нэгдмэл байдлыг хангаж, хөгжүүлэлтийн бүтээмжийг нэмэгдүүлсэн.

1.5.2 *API ба серверийн холболтын хандлага*

Системийн хоёр хэрэглэгчийн интерфэйс нь сервер талын нэг үйлчилгээтэй холбогдож, өвчтөн, цаг товлот, үзлэг, оношлогоо, жор, тайлангийн өгөгдлийг солилцоно. Иймээс API зохиомж нь дараах шаардлагыг хангах ёстой:

- **Модуль хоорондын нэгэн жигд холболт:** Бүх модуль ижил хандалтын зарчмаар өгөгдөл солилцох
- **Эрхийн хяналт:** Хэрэглэгчийн дүр бүрт зөвшөөрөгдсөн үйлдлийг сервер талд шалгах
- **Өргөтгөх боломж:** Шинэ модуль нэмэх үед одоо байгаа урсгалыг эвдэхгүй байх
- **Баримтжуулалт:** Клиент болон серверийн өгөгдлийн гэрээг тодорхой байлгах

Энэ ажлын хэрэгжүүлэлтэд сонгосон API технологийн нарийвчилсан тайлбарыг 4-р бүлэгт авч үзнэ.

1.5.3 *React ба Vite*

React нь Meta-ийн бүтээсэн хэрэглэгчийн интерфэйсийн сан бөгөөд компонент хандлагатай, виртуал DOM ашигладаг [7]. Vite нь frontend төслийн build болон development server-ийн хэрэгсэл бөгөөд HMR (Hot Module Replacement), module bundling, development build-ийн ажиллагааг хангана [10].

1.5.4 *MongoDB*

MongoDB нь документ хандлагат NoSQL өгөгдлийн сан юм [8]. Эмнэлгийн системд MongoDB сонгосон шалтгаан:

- Уян хатан schema — эмнэлгийн өгөгдөл нь олон төрлийн бүтэцтэй;
- Mongoose ODM — TypeScript-тэй сайн нийцдэг [16];
- Aggregation pipeline — тайлан, статистик гаргах өгөгдлийн боловсруулалтад ашиглагдана;
- Индексжүүлэлт болон collection-ийн уян хатан бүтэц нь эхний шатны прототип хөгжүүлэлтэд тохиромжтой.

Гэхдээ эмнэлгийн өгөгдөл нь харилцан хамаарал ихтэй тул PostgreSQL зэрэг relational өгөгдлийн сантай харьцуулж үзэх шаардлагатай. Энэ ажлын хувьд богино хугацаанд прототип хөгжүүлэх, TypeScript/Mongoose-тэй нэг мөр ажиллах, JSON хэлбэрийн эмнэлгийн тэмдэглэл хадгалах хэрэгцээг үндэслэн MongoDB-г сонгов.

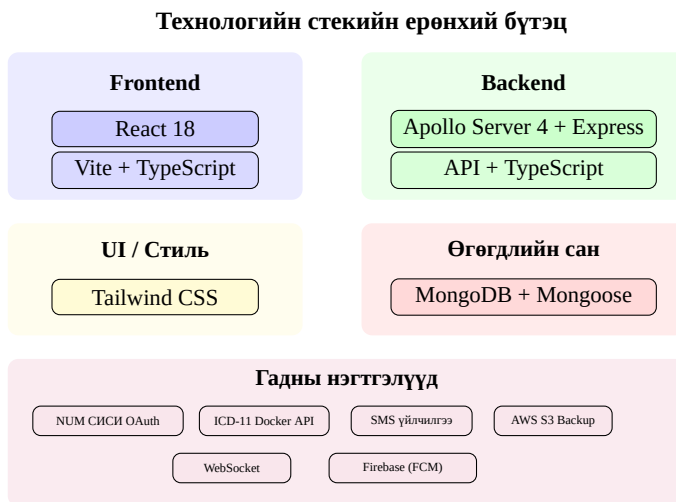
Хүснэгт 1.4: Өгөгдлийн сангийн сонголтын үндэслэл

Шалгуур	MongoDB	PostgreSQL	Тайлбар
Schema-ийн уян хатан байдал	Өндөр	Дунд	Үзлэгийн тэмдэглэл, харшил, жорын items зэрэг уян бүтэцтэй өгөгдөлд MongoDB тохиромжтой
Холбоост өгөгдлийн integrity	Дунд	Өндөр	Production түвшинд transaction, foreign key шаардлага их бол PostgreSQL давуу
TypeScript backend-тэй нийцэл	Өндөр	Өндөр	Mongoose ODM ашигласнаар schema болон validation-ийг кодын түвшинд тодорхойлсон
Прототип хөгжүүлэлтийн хурд	Өндөр	Дунд	Богино хугацаанд collection нэмэх, өөрчлөхөд MongoDB илүү уян
Тайлан, статистик	Дунд	Өндөр	Энэ ажилд aggregation pipeline ашиглах боломжийг авч үзсэн боловч нарийн тайлангийн хэсгийг цаашид өргөжүүлнэ

1.5.5 Бодит цагийн мэдэгдэл

Бодит цагийн мэдэгдэл нь эмчийн дарааллын өөрчлөлт, цаг товлолтын баталгаажуулалт, хэрэглэгчид илгээх системийн мэдэгдэл зэрэгт шаардлагатай. Ийм боломжийг хэрэгжүүлэхдээ серверээс клиент рүү автоматаар мэдээлэл дамжуулах сувгийг ашиглана [21].

1.5.6 OAuth 2.0 ба NUM SISI нэгтгэл



Зураг 1.1: Технологийн стекийн бүтэц

МУИС-ийн СИСИ OAuth 2.0 нэгтгэлийг системийн нэвтрэлтийн урсгалд холбохоор бэлтгэсэн [13, 26]. Одоогийн хэрэгжүүлэлтэд админ талын хэрэглэгчид утасны дугаар, нууц үгээр нэвтрэх боломжтой. Patient талд МУИС-ийн имэйлээр нэвтрэх урсгал болон эмчийн бүртгэсэн хэрэглэгч утасны дугаараараа нэвтрэх хувилбар тусгагдсан. Цаашид СИСИ OAuth 2.0 нэгтгэлийн албан зөвшөөрөл, магадлан итгэмжлэл бүрэн баталгаажсаны дараа Authorization Code Grant урсгалаар холбох боломжтой байдлаар зохиомжилсон.

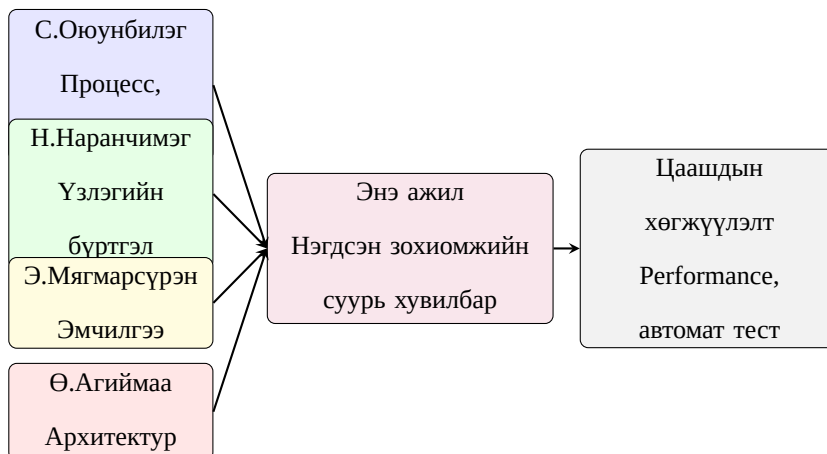
Энэхүү нэгтгэлийн хүрээнд дараах боломжуудыг бэлтгэсэн:

- Оюутан, багш, ажилтан СИСИ хаягаараа нэвтрэх боломжийг дэмжих
- СИСИ-ийн мэдээллийг (нэр, имэйл, утас) автоматаар татахад бэлэн өгөгдлийн зураглал гаргах
- Authorization Code Grant flow-ийн redirect, callback, state шалгалтын урсгалыг backend талд тусгах
- ICD-11 Docker API-ийг оношийн кодчиллын эх сурвалж болгон ашиглах

- Утасны дугаар руу мессеж илгээх гуравдагч талын үйлчилгээг мэдэгдэл болон баталгаажуулалтын урсгалд ашиглах

1.6 Бүлгийн дүгнэлт

Өмнөх ажлууд ба энэ ажлын уялдаа



Зураг 1.2: Өмнөх судалгааны ажлуудын уялдааны зураглал

Энэ бүлэгт МУИС-ийн эмнэлгийн одоогийн нөхцөл байдал, өмнөх 4 судалгааны ажил, ижил төстэй системүүд, технологийн сонголтын үндэслэлийг авч үзсэн. Өмнөх ажлууд нь тус тусын чиглэлээр чухал суурь мэдээлэл өгч байгаа тул энэ ажил тэдгээрийг үгүйсгэх бус, харин нэгтгэн уялдуулах замаар МУИС-ийн эмнэлгийн хэрэгцээнд тохирсон HMIS-ийн суурь хувилбарыг боловсруулахад чиглэв. Цаашид performance хэмжилт, автоматжуулсан тест болон аюулгүй байдлын нарийвчилсан шалгалтыг үргэлжлүүлэн хийх шаардлагатай.

2. СИСТЕМИЙН ШИНЖИЛГЭЭ

Энэ бүлэгт системийн суурь нөхцөл, бизнес процессын шинжилгээ, функциональ болон функциональ бус шаардлагууд, хэрэглэгчийн дүр (RBAC), use case загварчлал, процесс загварчлал, статик загварчлалыг авч үзсэн.

2.1 Суурь нөхцөл

МУИС-ийн эмнэлгийн мэдээллийн системийг шинжлэхдээ дараах суурь нөхцлүүдийг тодорхойлсон:

- Систем нь МУИС-ийн дотоод сүлжээ болон интернетээс хандах боломжтой байна
- Хэрэглэгчид вэб хөтчөөр (Chrome, Firefox, Safari) ажиллана
- СИСИ OAuth 2.0 нэгтгэлийн зөвшөөрөл баталгаажсаны дараа шууд холбох боломжтой байна
- Эмнэлгийн ажлын цагт (Даваа–Баасан, 09:00–17:00) идэвхтэй ажиллана
- MongoDB cloud instance (Atlas) эсвэл дотоод серверт байршина
- HTTPS протоколоор аюулгүй холбогдоно

2.2 Бизнес процессын шинжилгээ

Эмнэлгийн үндсэн бизнес процессуудыг дараах дарааллаар тодорхойлсон.

2.2.1 Өвчтөний бүртгэлийн процесс

Өвчтөний бүртгэл нь системийн анхны алхам юм. Эмнэлэгт тусдаа бүртгэлийн ажилтан байхгүй тул өвчтөн өөрөө нэвтрэх, эсвэл эмч/системийн админ шаардлагатай тохиолдолд хэрэглэгчийн мэдээллийг бүртгэх урсгалаар зохион байгуулна. Бүртгэлийн дарааллыг дараах байдлаар тодорхойлсон:

1. Өвчтөн системд өөрөө нэвтрэх хүсэлт гаргана эсвэл эмч/системийн админ хэрэглэгчийн мэдээллийг бүртгэнэ
2. Систем өвчтөний мэдээллийг утас, имэйл, регистрийн дугаар, СИСИ ID зэрэг талбараар шалгана
3. Хэрэв шинэ өвчтөн бол бүртгэлийн маягт бөглөгдөнө (овог, нэр, утас, регистрийн дугаар, СИСИ ID, цусны бүлэг, харшил)
4. Өвчтөний ангилал тодорхойлогдоно (оюутан/багш/ажилтан/гадны)
5. Хэрэв МУИС-ийн гишүүн бол СИСИ ID эсвэл МУИС-ийн имэйлээр баталгаажуулах боломжийг бэлтгэнэ
6. Бүртгэлийн дугаар автоматаар үүснэ
7. Өвчтөн системд бүртгэгдэнэ

2.2.2 Цаг товлох процесс

Системд цаг товлох дараалал:

1. Өвчтөн системд нэвтэрч өөрөө цаг товлон
2. Үйлчилгээ сонгоно
3. Тухайн үйлчилгээнд тохирох эмч эсвэл нөөц сонгоно
4. Чөлөөт цагийн жагсаалтаас огноо, цаг сонгоно
5. Үзлэгийн төрөл сонгоно (товлосон/шуурхай/дахин үзлэг/яаралтай)
6. Гол зовиураа бичнэ
7. Цаг баталгаажуулна
8. Мэдэгдэл илгээгдэнэ

2.2.3 Үзлэгийн процесс

Эмчийн үзлэгийн дараалал:

1. Өвчтөн ирсэн болохоо бүртгүүлнэ (check-in)
2. Дарааллын дугаар олгогдоно
3. Шаардлагатай тохиолдолд эмч амин үзүүлэлт болон үзлэгийн тэмдэглэлийг бүртгэнэ
4. Эмч өвчтөнийг хүлээж авна (appointment status: in_progress)
5. Эмч асуумж авна (гол зовиур, өвчний одоогийн түүх)
6. Биеийн үзлэг хийнэ (physical examination)
7. Оношлогоо тавина (ICD кодтой)
8. Эмчилгээний төлөвлөгөө боловсруулна
9. Жор бичнэ (шаардлагатай бол)
10. Үзлэг дуусна (visit status: completed)

2.3 Системийн шаардлага

2.3.1 Функциональ шаардлага

Системийн функциональ шаардлагыг модуль тус бүрээр тодорхойлсон.

Нэвтрэлтийн модуль (Authentication)

Хүснэгт 2.1: Нэвтрэлтийн модулийн функциональ шаардлага

ID	Шаардлага	Чухал
FR-A01	Утасны дугаар, нууц үгээр нэвтрэх	Must
FR-A02	Имэйл OTP-ээр нэвтрэх	Should
FR-A03	СИСИ OAuth 2.0-ээр нэвтрэх урсгалыг бэлтгэх	Should
FR-A04	Шинэ хэрэглэгч бүртгүүлэх	Must
FR-A05	Нууц үг солих	Must
FR-A06	Нууц үг сэргээх (OTP)	Should
FR-A07	Профайл засварлах	Should
FR-A08	HttpOnly cookie-д суурилсан JWT session	Must

Өвчтөний модуль (Patient)

Хүснэгт 2.2: Өвчтөний модулийн функциональ шаардлага

ID	Шаардлага	Чухал
FR-P01	Өвчтөн бүртгэх (овог, нэр, утас, РД, СИСИ ID)	Must
FR-P02	Өвчтөний мэдээлэл засварлах	Must
FR-P03	Өвчтөн хайх (нэр, утас, РД-аар)	Must
FR-P04	Ангилал тодорхойлох (оюутан/багш/ажилтан/гадны)	Must
FR-P05	Цусны бүлэг, харшлын мэдээлэл хадгалах	Should
FR-P06	Яаралтай үеийн холбоо барих мэдээлэл	Should
FR-P07	Өвчтөний түүх харах	Must

Цаг товлолтын модуль (Appointment)

Хүснэгт 2.3: Цаг товлолтын модулийн функциональ шаардлага

ID	Шаардлага	Чухал
FR-AP01	Цаг товлдох (эмч, огноо, цаг сонгох)	Must
FR-AP02	Чөлөөт цагийн жагсаалт харах	Must
FR-AP03	Цаг цуцлах (шалтгаантай)	Must
FR-AP04	Ирсэн бүртгэл (check-in)	Must
FR-AP05	Эмчийн дараалал харах	Must
FR-AP06	Үзлэг эхлүүлэх/дуусгах	Must
FR-AP07	Ирээгүй тэмдэглэл (no-show)	Should
FR-AP08	Цаг товлолтын жагсаалт шүүлтүүртэй	Should

Үзлэгийн модуль (Visit)

Хүснэгт 2.4: Үзлэгийн модулийн функциональ шаардлага

ID	Шаардлага	Чухал
FR-V01	Үзлэг үүсгэх (appointment-тай холбоотой)	Must
FR-V02	Гол зовиур, өвчний түүх бичих	Must
FR-V03	Биеийн үзлэгийн тэмдэглэл	Must
FR-V04	Үнэлгээ, төлөвлөгөө бичих	Must
FR-V05	Амин үзүүлэлт бүртгэх боломжтой байх	Should
FR-V06	Үзлэг дуусгах	Must
FR-V07	Өвчтөний үзлэгийн түүх	Must
FR-V08	Өнөөдрийн үзлэгүүдийн жагсаалт	Should

Оношлогоо ба жорын модуль

Хүснэгт 2.5: Оношлогоо ба жорын модулийн функциональ шаардлага

ID	Шаардлага	Чухал
FR-D01	Оношлогоо үүсгэх (ICD код, нэр, тайлбар)	Must
FR-D02	Оношлогооны төрөл (primary/secondary/differential)	Must
FR-D03	Хүндрэлийн зэрэг (mild/moderate/severe/critical)	Should
FR-D04	Оношлогоо засварлах, устгах	Must
FR-RX01	Жор үүсгэх (эмийн нэр, тун, давтамж, хугацаа)	Must
FR-RX02	Жорын дугаар автоматаар үүсгэх	Must
FR-RX03	Жорын төлөв (идэвхтэй/олгосон/цуцалсан)	Must
FR-RX04	Өвчтөний жорын түүх	Should

Үйлчилгээ ба нөөцийн хуваарилалтын модуль

Хүснэгт 2.6: Үйлчилгээ ба нөөцийн хуваарилалтын модулийн функциональ шаардлага

ID	Шаардлага	Чухал
FR-S01	Эмнэлгийн үйлчилгээний төрөл бүртгэх (үзлэг, тариа, дусал, төхөөрөмжийн үйлчилгээ гэх мэт)	Must
FR-S02	Үйлчилгээ бүрийн үргэлжлэх хугацаа, завсарлага, хариуцах ажилтны төрлийг тодорхойлох	Must
FR-S03	Өрөө, төхөөрөмж, эмч, сувилагч зэрэг нөөцийг тусад нь бүртгэх	Must
FR-S04	Нөөцийн багтаамжийг тооцох (жишээлбэл дуслын өрөө олон суудалтай байх)	Must
FR-S05	Давхардсан цаг товлолт, завсарлага, ашиглах боломжгүй хугацааг шалгах	Must
FR-S06	Үйлчилгээ, нөөц, цагийн уялдаагаар чөлөөт цагийн жагсаалт гаргах	Should

Сэтгэл ханамжийн судалгааны модуль

Хүснэгт 2.7: Сэтгэл ханамжийн судалгааны модулийн функциональ шаардлага

ID	Шаардлага	Чухал
FR-SV01	Patient portal дээр идэвхтэй survey template-ийг татаж харуулах	Must
FR-SV02	Өвчтөн 1–5 онооны үнэлгээ болон нээлттэй саналаар судалгаа бөглөх	Must
FR-SV03	3 completed үйлчилгээ авсан боловч одоогийн survey version-ийг бөглөөгүй өвчтөнд судалгааг заавал бөглүүлэх	Must
FR-SV04	Admin portal дээр судалгааны гарчиг, тайлбар, асуулт, category, идэвхтэй төлвийг удирдах	Must
FR-SV05	Судалгааны асуулт өөрчлөгдөх бүрт template version history-г validFrom, validTo огноотой хадгалах	Must
FR-SV06	Хариулт хадгалах үед тухайн version-ийн асуултын label, category, score-г response дотор snapshot байдлаар хадгалах	Must
FR-SV07	Хариулттай template устгах үед шууд delete хийхгүй, archive төлөвт шилжүүлэх	Must
FR-SV08	Admin талд судалгааны хариултыг жагсааж, тайлан болон сайжруулалтын дүгнэлтэд ашиглах	Should

Тайлан, аудит ба мэдэгдлийн модуль

Хүснэгт 2.8: Тайлан, аудит ба мэдэгдлийн модулийн функциональ шаардлага

ID	Шаардлага	Чухал
FR-RA01	Сарын тайланг насны бүлэг, хүйс, үйлчилгээ, нөөц, эмчээр нэгтгэн гаргах	Must
FR-RA02	Өвчтөний үзлэг, онош, жорын түүхийг нэг дор харах	Must
FR-RA03	Чухал үйлдлүүдийг аудитын бүртгэлд хадгалах	Must
FR-RA04	Цаг товлолт, үзлэгийн төлөв, системийн мэдээллийг хэрэглэгчид мэдэгдэх	Should
FR-RA05	Тайлангийн өгөгдлийг хугацаагаар шүүх, харьцуулах боломжтой байх	Should

2.3.2 Функциональ бус шаардлага

Хүснэгт 2.9: Функциональ бус шаардлага

ID	Ангилал	Шаардлага
NFR-01	Гүйцэтгэл	API хариу өгөх хугацаа 500ms-аас бага
NFR-02	Аюулгүй байдал	HttpOnly cookie-д суурилсан JWT session, bcrypt нууц үг хэшлэлт, CSRF эрсдэлийн хяналт
NFR-03	Аюулгүй байдал	RBAC — 4 үндсэн дүрийн хандалтын хяналт
NFR-04	Аюулгүй байдал	Audit log бүх чухал үйлдлийг бүртгэх
NFR-05	Нийцэмж	Орчин үеийн вэб хөтчүүдэд ажиллана
NFR-06	Хэрэглэх чадвар	Монгол хэлний интерфейс
NFR-07	Найдвартай	Audit log 1 жилийн хугацаатай (TTL)
NFR-08	Масштабжуулалт	MongoDB-ийн индексжүүлэлт

2.3.3 Шаардлага, хэрэгжүүлэлт ба тестийн уялдаа

Шүүмжлэгчийн өнцгөөс системийн шаардлагууд зөвхөн жагсаалт байдлаар үлдэхгүй, дараагийн бүлгүүдийн зохиомж, хэрэгжүүлэлт, тесттэй холбогдох ёстой. Иймээс гол шаардлагуудын traceability-г хүснэгт 2.10-д нэгтгэн харуулав.

Хүснэгт 2.10: Шаардлага–use case–хэрэгжүүлэлт–тестийн уялдаа

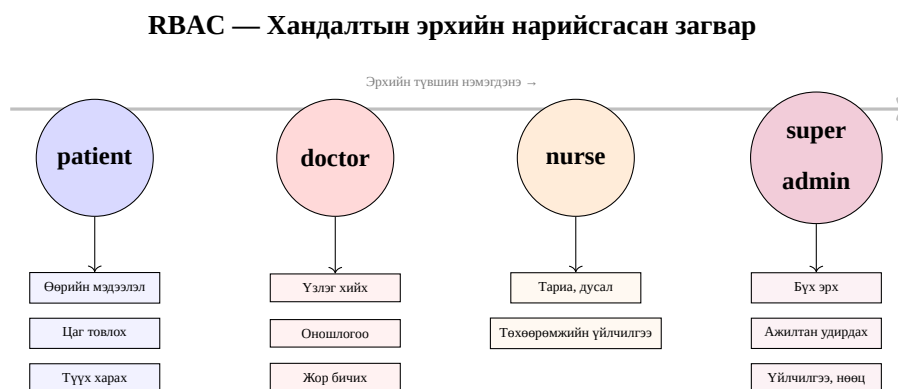
Шаардлага	Хамрах хүрээ	Use case	Хэрэгжүүлсэн хэсэг	Шалгах тест
FR-A01, FR-A03, FR-A08	Дотоод болон СИСИ нэвтрэлт, JWT session	UC-01, UC-02	Auth resolver, OAuth callback, context middleware	TC-01, TC-02, TC-12
FR-P01–FR- P07	Өвчтөн бүртгэл, хайлт, түүх харах	UC-03, UC-10	Patient model, search query, patient profile page	TC-03
FR-AP01– FR-AP08	Цаг товлох, check-in, дараалал	UC-04, UC-05	Appointment model, available slots service, doctor queue	TC-04–TC-07
FR-V01–FR- V08	Үзлэг үүсгэх, амин үзүүлэлт, дуусгах	UC-06, UC-07	Visit model, VitalSign model, examine page	TC-08, TC-11
FR-D01–FR- D04	Оношлогоо үүсгэх, засварлах, ICD код	UC-08	Diagnosis model, diagnosis resolver	TC-09
FR-RX01– FR-RX04	Жор үүсгэх, төлөв, түүх	UC-09, UC-10	Prescription model, prescription resolver	TC-10
FR-S01–FR- S06	Үйлчилгээ ба нөөцийн хуваарилалт	UC-04	Services/resources/ unavailable blocks зохиомж	Зохиомжийн түвшин
FR-SV01– FR-SV08	Сэтгэл ханамжийн судалгаа, template version, заавал бөглүүлэх логик	UC-14, UC-15	Satisfaction survey template, response model, patient/admin survey pages	TC-18–TC-22
FR-RA01– FR-RA05	Тайлан, аудит, мэдэгдэл	UC-11	Notification, AuditLog, aggregation-ийн суурь зохиомж	Хэсэгчлэн шалгасан
NFR-02– NFR-04	Аюулгүй байдал, RBAC, audit	Бүх use case	JWT, bcrypt, role guard, audit model	TC-12, security review

Хүснэгтээс харахад нэвтрэлт, өвчтөн, цаг товлалт, үзлэг, оношлогоо, жор болон сэтгэл

ханамжийн судалгааны урсгалууд хэрэгжүүлэлт ба тесттэй шууд холбогдож байна. Харин тайлан, аудит, мэдэгдэл, үйлчилгээ ба нөөцийн хуваарилалтын зарим хэсэг нь энэ ажлын хүрээнд зохиомж болон суурь бүтэц хэлбэрээр тодорхойлогдсон тул хамгаалалтын үед хэрэгжүүлсэн болон цаашид хийх хэсгийн заагийг тодорхой тайлбарлах шаардлагатай.

2.4 Хэрэглэгчийн дүр (User Roles) ба RBAC

Системийн бодит үйл ажиллагаанд тусдаа өгөгдөл оруулагч эсвэл бүртгэлийн ажилтан байхгүй гэж нарийсгасан. Иймээс хандалтын эрхийг өвчтөн, эмч, сувилагч, системийн админ гэсэн 4 үндсэн дүрээр тодорхойлов. Цаг товлох болон ирц баталгаажуулах урсгал нь өвчтөн өөрөө хийх боломжтой байхаар, харин эмнэлгийн ажилтны талын үзлэг, үйлчилгээ, нөөцийн тохиргоог эрх бүхий эмч, сувилагч, системийн админы эрхээр хязгаарлана.



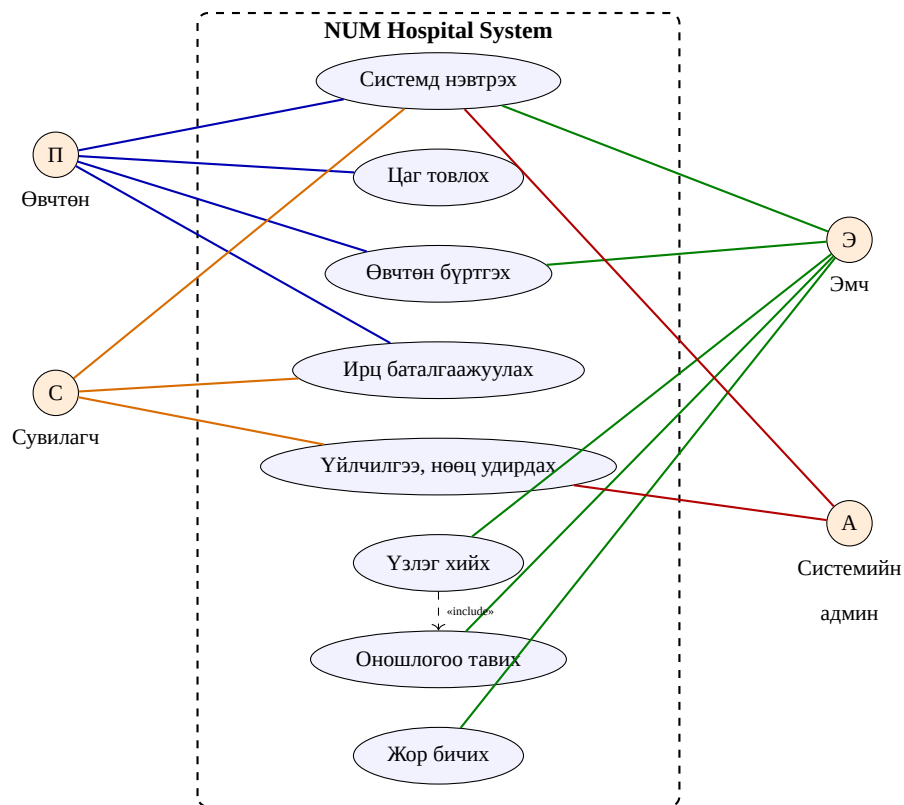
Зураг 2.1: RBAC — 4 үндсэн дүрийн хандалтын эрхийн загвар

Хүснэгт 2.11: RBAC — Хэрэглэгчийн дүр ба эрхүүд

Дүр (Role)	Эрх
patient	Өөрийн мэдээлэл харах/засах, цаг товлдох/цуцлах, өөрийн ирц баталгаажуулах, үзлэгийн түүх, жор, оношлогоо харах, мэдэгдэл хүлээн авах
doctor	Дарааллын жагсаалт харах, үзлэг эхлүүлэх/засах/дуусгах, оношлогоо тавих, жор бичих, өвчтөний эмнэлгийн мэдээлэл харах
nurse	Эмчийн зөвлөгөөний дагуу тариа, дусал болон тоног төхөөрөмжийн үйлчилгээ үзүүлэх, өөрт хамаарах үйлчилгээний цаг болон мэдээлэл харах
superadmin	Бүх эрх, хэрэглэгч/ажилтан, үйлчилгээ, тоног төхөөрөмжийн нөөц, системийн үндсэн тохиргоо удирдах

2.5 Use Case загварчлал

Системийн гол use case-үүдийг дараах диаграмд харуулав.



Зураг 2.2: Системийн Use Case диаграм

Системийн гол use case-үүдийг дараах хүснэгтэд нэгтгэн харуулав.

Хүснэгт 2.12: Системийн гол Use Case-ууд

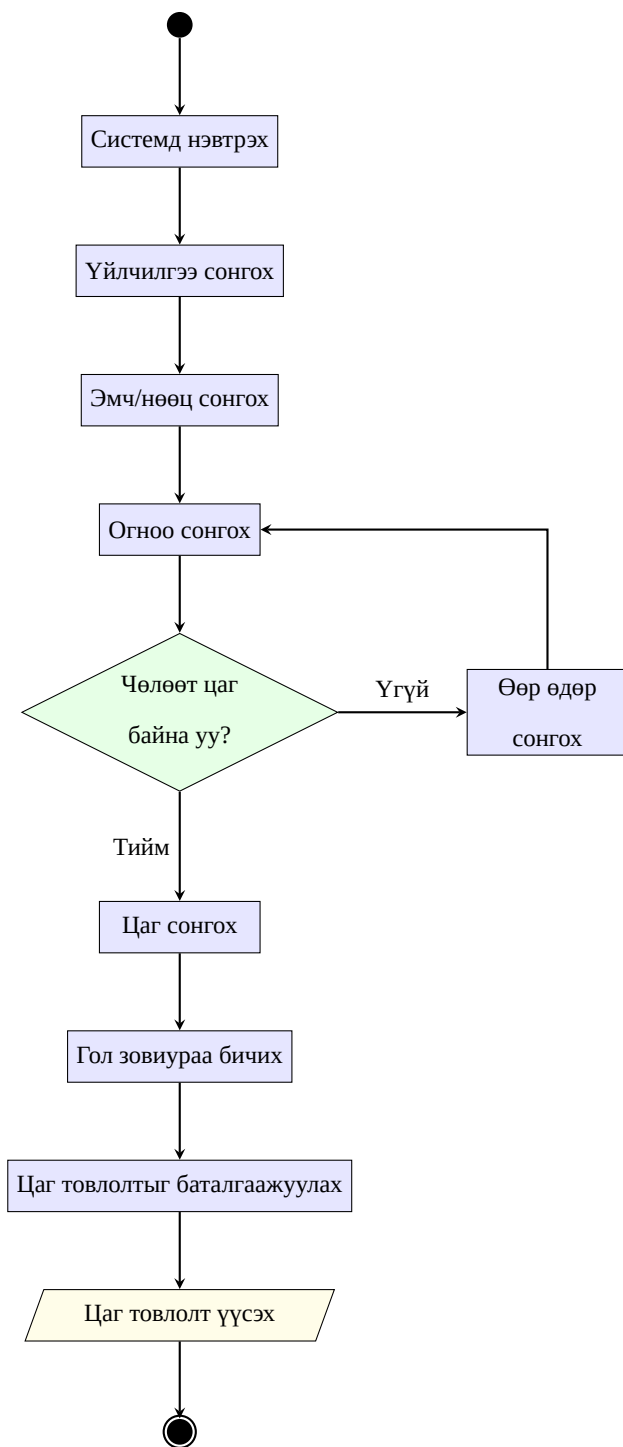
UC ID	Use Case	Оролцогч
UC-01	Системд нэвтрэх	Бүх хэрэглэгч
UC-02	Байгууллагын мэдээллээр нэвтрэх	Оюутан, багш
UC-03	Өвчтөн бүртгэх	Өвчтөн, эмч
UC-04	Цаг товлон	Өвчтөн
UC-05	Ирц баталгаажуулах	Өвчтөн, эмч, сувилагч
UC-06	Үйлчилгээ, тоног төхөөрөмжийн цаг удирдах	Сувилагч, системийн админ
UC-07	Үзлэг хийх	Эмч
UC-08	Оношлогоо тавих	Эмч
UC-09	Жор бичих	Эмч
UC-10	Үзлэгийн түүх харах	Өвчтөн, эмч
UC-11	Мэдэгдэл хүлээн авах	Бүх хэрэглэгч
UC-12	Үйлчилгээ, нөөц удирдах	Системийн админ
UC-13	Ажилтан удирдах	Системийн админ
UC-14	Сэтгэл ханамжийн судалгаа бөглөх	Өвчтөн
UC-15	Судалгааны template удирдах	Системийн админ

2.6 Процесс загварчлал

Системийн гол процессуудыг Activity Diagram-аар загварчилсан.

2.6.1 Цаг товлох процессын дэлгэрэнгүй

Цаг товлох процессын Activity диаграм:

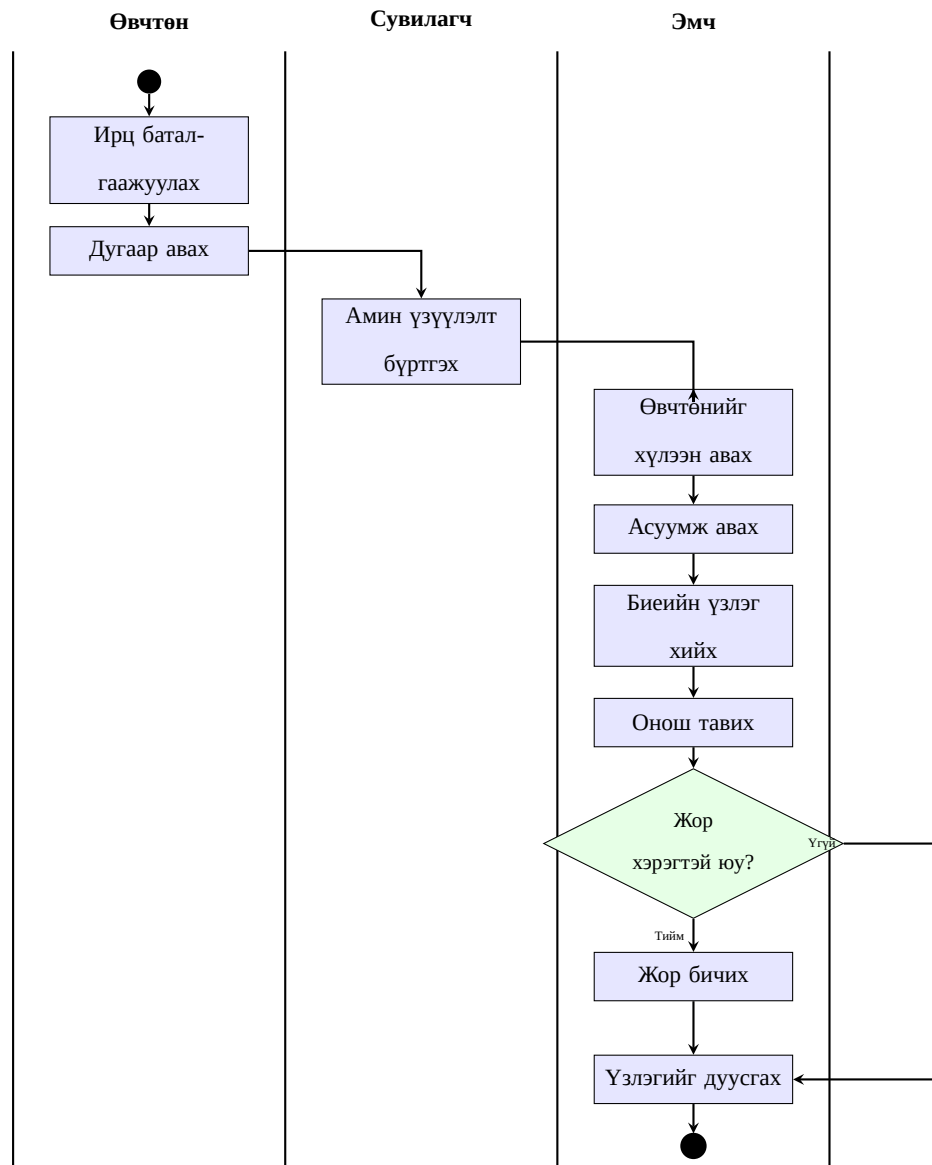


Зураг 2.3: Цаг товллох процессын Activity диаграм

Цаг товллох процесс нь дараах алхмуудтай:

1. Өвчтөн “Цаг товлох” хэсэгт орно
2. Үйлчилгээ сонгоно (жишээ нь: эмчийн үзлэг, тариа, дусал, шарлага, массаж)
3. Тухайн үйлчилгээнд тохирох эмч, сувилагч эсвэл төхөөрөмжийн нөөцөөс сонгоно
4. Өвчтөн өөрт тохирох өдрийг сонгоно
5. Систем сонгосон өдөрт чөлөөт цаг байгаа эсэхийг шалгана
6. Хэрэв боломжит цаг байхгүй бол өөр өдөр сонгон дахин шалгана
7. Боломжит цаг байвал өвчтөн тохирох цагаа сонгоно
8. Гол зовиур болон шаардлагатай нэмэлт мэдээллээ бөглөнө
9. Систем цаг товлолтыг баталгаажуулж бүртгэнэ

2.6.2 Үзлэгийн процессын дараалал



Зураг 2.4: Үзлэгийн процессын swimlane төрлийн үйл ажиллагааны диаграмм

Үзлэгийн процессын үндсэн дарааллыг дараах байдлаар тодорхойлов:

1. Өвчтөн ирцээ баталгаажуулж, дарааллын дугаар авна
2. Шаардлагатай тохиолдолд сувилагч амин үзүүлэлтийг бүртгэнэ

3. Эмч өвчтөнийг үзлэгт хүлээн авч, асуумж авна
4. Гол зовиур, өвчний одоогийн түүх болон биеийн үзлэгийн мэдээллийг тэмдэглэнэ
5. Онош тогтоож, цаашдын эмчилгээний төлөвлөгөөг тодорхойлно
6. Шаардлагатай бол жор бичиж, үзлэгийг дуусгана

2.7 Статик загварчлал

Системийн статик загварыг MongoDB-ийн collection-уудаар илэрхийлсэн. Нийт 17 collection ашиглаж байгаа бөгөөд тэдгээрийг хэрэглэгч, өвчтөн, ажилтан, цаг товлот, үзлэг, үйлчилгээ, нөөц, мэдэгдэл, аудит болон сэтгэл ханамжийн судалгааны мэдээллийн чиглэлээр ангилж болно. Эдгээрээс `services`, `resources`, `departments`, `unavailable_blocks` нь цаг товлолтын нөөц болон боломжгүй хугацааны тооцооллыг дэмжих туслах бүтэц юм.

1. `users` — хэрэглэгчийн дансны мэдээлэл
2. `patients` — өвчтөний эмнэлгийн мэдээлэл
3. `staff` — ажилтны мэдээлэл (эмч, сувилагч, системийн админ)
4. `departments` — ажилтны харьяалах нэгж, тасгийн тохиргоо
5. `services` — цаг товлон боломжтой үйлчилгээний мэдээлэл
6. `resources` — эмч, сувилагч, өрөө, төхөөрөмж зэрэг ашиглагдах нөөц
7. `unavailable_blocks` — нөөц ашиглах боломжгүй хугацааны бүртгэл
8. `appointments` — цаг товлолтын мэдээлэл
9. `visits` — үзлэгийн мэдээлэл

- 10. `vital_signs` — амин үзүүлэлтийн мэдээлэл
- 11. `diagnoses` — оношийн мэдээлэл
- 12. `prescriptions` — жорын мэдээлэл
- 13. `notifications` — мэдэгдлийн мэдээлэл
- 14. `audit_logs` — аудитын бүртгэл
- 15. `external_identities` — гаднын нэвтрэлтийн холбоос (OAuth)
- 16. `satisfaction_survey_templates` — судалгааны template, version history, archive төлөв
- 17. `satisfaction_surveys` — patient-ийн бөглөсөн судалгааны response, rating snapshot

2.8 Бүлгийн дүгнэлт

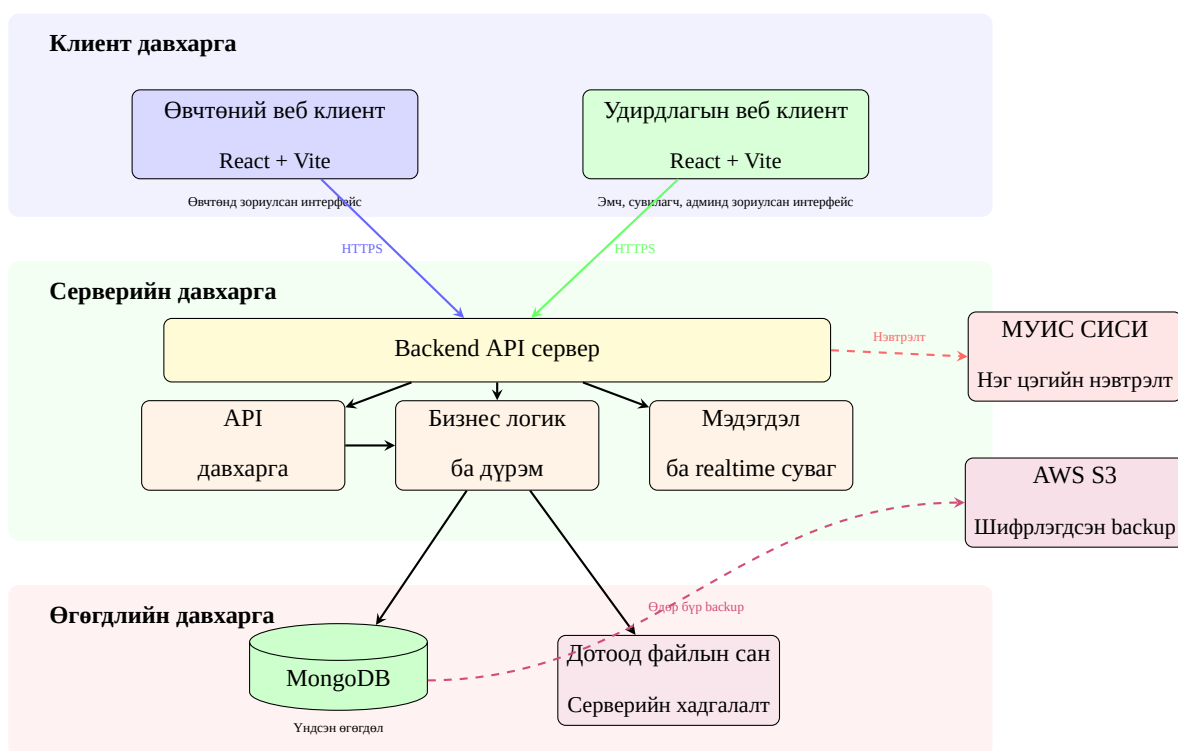
Энэ бүлэгт системийн бизнес процесс, функциональ болон функциональ бус шаардлагууд, RBAC эрхийн загвар, use case-ууд, процесс загварчлалыг тодорхойлсон. Өгөгдөл оруулагч болон дотоод нэгжийн тусдаа удирдлагын ойлголтыг хасаж, цаг товлолтыг үйлчилгээ-нөөц-эмч/сувилагчийн сонголтод төвлөрүүлсэн. Мөн сэтгэл ханамжийн судалгааг patient portal болон admin portal-той уялдсан template/version бүхий тусдаа бизнес урсгал болгон тодорхойлов. Нийт 40 орчим функциональ шаардлага, 8 функциональ бус шаардлага, 15 use case тодорхойлогдож, гол шаардлагуудыг хэрэгжүүлэлт болон тестийн хувилбаруудтай traceability хүснэгтээр холбосон. Эдгээр нь дараагийн бүлэг болох системийн зохиомжийн суурь болно.

3. СИСТЕМИЙН ЗОХИОМЖ

Энэ бүлэгт системийн ерөнхий архитектур, монорепо бүтэц, өгөгдлийн сангийн зохиомж, GraphQL API зохиомж, нэвтрэлт ба аюулгүй байдлын зохиомж, дэлгэцийн зохиомжийг тодорхойлсон.

3.1 Системийн архитектур

NUM Hospital Management System нь монорепо бүтэцтэй, клиент, сервер, өгөгдлийн сангийн гурван түвшинт веб систем юм.



Зураг 3.1: Системийн ерөнхий архитектурын диаграм

Архитектурын ерөнхий бүтэц:

1. **Backend API сервер** — клиентүүдээс ирэх хүсэлтийг хүлээн авч, бизнес логик, нэвтрэлт, өгөгдлийн хандалтыг зохицуулна

2. **Дотоод файлын сан** — хэрэглэгчийн оруулсан файлуудыг application server-ийн дотоод хадгалалтад байршуулна
3. **Өвчтөний веб клиент** — өвчтөн нэвтрэх, цаг товлох, үзлэгийн түүх болон жороо харах интерфейс
4. **Удирдлагын веб клиент** — эмч, сувилагч, системийн админд зориулсан өдөр тутмын ажиллагааны интерфейс
5. **AWS S3 backup** — MongoDB-ийн өдөр тутмын нөөц хуулбарыг шифрлэсэн байдлаар хадгална

3.1.1 Монорепо бүтэц

Төслийн файлын бүтцийг дараах байдлаар зохион байгуулсан:

```
1 NUM-Hospital-service/  
2   backend/  
3     src/  
4       graphql/      # API schema, resolvers  
5       models/       # Mongoose models (14)  
6       services/     # Business logic  
7       routes/       # REST routes (OAuth, health)  
8       utils/        # Auth, constants, helpers  
9       cron/         # Scheduled tasks  
10      index.ts       # Entry point  
11 patient-frontend/  
12   src/  
13     pages/          # Patient portal pages  
14     components/     # Reusable components  
15     graphql/        # Client API operations
```

```
16      hooks/                # Custom React hooks
17  admin-frontend/
18    src/
19      pages/                # Admin/doctor pages
20      components/          # Reusable components
21      graphql/              # Client API operations
22      hooks/                # Custom React hooks
```

Код 3.1: Төслийн файлын бүтэц

3.1.2 Backend архитектур

Backend нь давхаргат архитектуртай:

1. **API Layer** — typeDefs болон resolvers хэлбэрээр тодорхойлогдсон серверийн гэрээ
2. **Service Layer** — бизнес логик, validation, authorization шалгалт
3. **Model Layer** — Mongoose schema, өгөгдлийн загвар, индексжүүлэлт
4. **Utility Layer** — JWT auth, constants, helper функцүүд

Express.js сервер дээр API middleware-ийг суулгасан. Клиентүүд серверийн нэг API гарцаар дамжин өгөгдөл солилцож, мэдэгдлийн суваг нь тусдаа холболтоор ажиллана.

3.1.3 Frontend архитектур

Хоёр frontend аппликейшн нь ижил технологийн стек (React + Vite + Tailwind CSS) ашигладаг боловч тус тусдаа порт дээр ажилладаг:

- **Patient Frontend** (port 3000) — Өвчтөний интерфейс: нэвтрэлт, хяналтын самбар, цаг товлох, үзлэгийн түүх, жорын жагсаалт, профайл

- **Admin Frontend** (port 3001) — Удирдлагын интерфейс: нэвтрэлт, хяналтын самбар, өвчтөний жагсаалт, цаг товлот удирдах, эмчийн дараалал, үзлэг хийх, ажилтан удирдах

Хоёр frontend хоёулаа ижил API client тохиргоогоор backend үйлчилгээтэй холбогддог.

3.2 Өгөгдлийн сангийн зохиомж

Энэ хэсэгт MongoDB-д суурилсан өгөгдлийн сангийн ерөнхий бүтэц, гол collection-уудын үүрэг, тэдгээрийн хоорондын хамаарал болон индексжүүлэлтийн шийдлийг тайлбарлав. Нийт 17 collection ашиглаж байгаа бөгөөд системийн үндсэн эмнэлзүйн урсгалд шууд оролцох collection-уудыг гол бүтэц, харин үйлчилгээ ба нөөцийн хуваарилалт болон сэтгэл ханамжийн судалгааг дэмжих collection-уудыг туслах бүтэц гэж ангилсан.

3.2.1 Collection-уудын ангилал

Collection-уудыг хамрах хүрээгээр нь ангилснаар системийн үндсэн өгөгдөл хаана төвлөрч, туслах бүтэц ямар үүрэгтэйг нэг хараад ойлгох боломжтой. Энэ ангиллыг хүснэгт 3.1-д харуулав.

Хүснэгт 3.1: Collection-уудын хамрах хүрээний ангилал

Ангилал	Collection	Үүрэг
Хэрэглэгч ба эрх	users, staff, departments, external_ identities	Дотоод хэрэглэгч, эмнэлгийн ажилтан, ажилтны харьяалах нэгж болон нэвтрэлтийн холбоосын мэдээлэл
Өвчтөний үйл явц	patients, appointments, visits, vital_signs, diagnoses, prescriptions	Бүртгэлээс эхлээд үзлэг, онош, жор хүртэлх үндсэн эмнэлзүйн өгөгдлийн урсгал
Мэдэгдэл ба аудит	notifications, audit_logs	Хэрэглэгчид мэдэгдэл хүргэх, чухал үйлдлийг мөрдөх бүртгэл
Үйлчилгээ ба нөөцийн туслах бүтэц	services, resources, unavailable_blocks	Үйлчилгээний төрөл, эмч/өрөө/төхөөрөмжийн нөөц болон боломжгүй хугацааны блок
Сэтгэл ханамжийн судалгаа	satisfaction_ survey_templates, satisfaction_ surveys	Судалгааны template, version history, patient response болон rating snapshot

3.2.2 Гол collection-уудын бүтэц

Энд системийн үндсэн ажиллагаанд шууд оролцдог гол collection-уудын бүтцийг тоймлон харуулав. Collection бүрийн тайлбар нь тухайн бүтэц ямар өгөгдөл хадгалж, бусад collection-уудтай яаж холбогддогийг товч илэрхийлнэ.

users — Хэрэглэгчийн бүртгэл

users collection нь системд нэвтрэх бүх хэрэглэгчийн суурь бүртгэл, эрхийн мэдээллийг хадгална. Өвчтөн, эмч, сувилагч, системийн админ зэрэг дүрүүдийн нийтлэг дансны мэдээлэл энэ түвшинд төвлөрнө.

Хүснэгт 3.2: Users collection-ий бүтэц

Талбар	Төрөл	Заавал	Тайлбар
_id	ObjectId	Тийм	Автомат үүснэ
email	String	Үгүй	Имэйл хаяг
password	String	Үгүй	bcrypt хэшлэгдсэн
phone	String	Unique	Утасны дугаар
role	Enum	Тийм	patient, doctor, nurse, superadmin
firstname	String	Тийм	Нэр
lastname	String	Тийм	Овог
gender	Enum	Үгүй	male, female, other
birthdate	Date	Үгүй	Төрсөн огноо
status	Enum	Тийм	active, inactive, suspended
fcmToken	String	Үгүй	Firebase push notification
lastLoginAt	Date	Үгүй	Сүүлийн нэвтрэлт

patients — Өвчтөний эмнэлгийн бүртгэл

`patients` collection нь өвчтөний эмнэлзүйн үндсэн бүртгэл, холбоо барих болон ангиллын мэдээллийг хадгална. Энэ нь `users` collection-тэй 1:1 холбоотой бөгөөд өвчтөний бусад эмнэлгийн урсгалын суурь объект болдог.

Хүснэгт 3.3: Patients collection-ий бүтэц

Талбар	Төрөл	Заавал	Тайлбар
_id	ObjectId	Тийм	Автомат
userId	ObjectId	Ref	User-тэй холбоос
registrationNumber	String	Unique	Бүртгэлийн дугаар
firstname, lastname	String	Тийм	Нэр, овог
phone, email	String	Үгүй	Холбоо барих
gender	Enum	Үгүй	Хүйс
birthdate	Date	Үгүй	Төрсөн огноо
nationalId	String	Үгүй	Регистрийн дугаар
sisId	String	Үгүй	СИСИ дугаар
category	Enum	Тийм	student, teacher, employee, external
bloodType	Enum	Үгүй	Цусны бүлэг (8 төрөл)
allergies	[String]	Үгүй	Харшлын жагсаалт
emergencyContact	Object	Үгүй	Яаралтай холбоо (name, phone, relationship)

appointments — Цаг товлолт

`appointments` collection нь өвчтөн, эмч болон үйлчилгээтэй холбогдсон цаг товлолтын мэдээллийг хадгална. Төлөв, дараалал, огноо, цагийн мэдээлэл энэ collection-д төвлөрч, үзлэг эхлэхээс өмнөх урсгалыг удирдана.

Хүснэгт 3.4: Appointments collection-ий бүтэц

Талбар	Төрөл	Тайлбар
patientId	ObjectId	Өвчтөний холбоос
doctorId	ObjectId	Эмчийн холбоос
scheduledDate	Date	Товлосон огноо
scheduledTime	String	Товлосон цаг (“09:00”)
duration	Number	Үргэлжлэх хугацаа (минут)
type	Enum	walk_in, scheduled, follow_up, emergency
status	Enum	scheduled, checked_in, in_progress, completed, cancelled, no_show
queueNumber	Number	Дарааллын дугаар
chiefComplaint	String	Гол зовиур

visits — Үзлэгийн бүртгэл

`visits` collection нь эмчийн үзлэгийн явцад бүртгэгдэх эмнэлзүйн дэлгэрэнгүй мэдээллийг хадгална. Энэ нь `appointments` collection-тэй 1:1 холбоотой бөгөөд онош, амин үзүүлэлт, жор зэрэг дараагийн collection-уудын зангилаа болдог.

Хүснэгт 3.5: Visits collection-ий бүтэц

Талбар	Төрөл	Тайлбар
appointmentId	ObjectId	Цаг товлолттой холбоос
patientId	ObjectId	Өвчтөн
doctorId	ObjectId	Эмч
visitDate	Date	Үзлэгийн огноо
status	Enum	draft, active, completed, cancelled
chiefComplaint	String	Гол зовиур
historyOfPresentIllness	String	Өвчний түүх
physicalExamination	String	Биеийн үзлэг
assessment	String	Үнэлгээ
plan	String	Эмчилгээний төлөвлөгөө
completedAt	Date	Дууссан цаг

satisfaction_survey_templates — Судалгааны template

`satisfaction_survey_templates` collection нь patient portal дээр харагдах сэтгэл ханамжийн судалгааны бүтцийг хадгална. Асуултууд frontend код дотор тогтмол бичигдэхгүй, харин admin portal-оос удирдагдах template байдлаар хадгалагдана. Template засагдах бүрт `currentVersion` нэмэгдэж, өмнөх хувилбар `versions` жагсаалтад `validFrom`, `validTo` огноотой үлдэнэ.

Хүснэгт 3.6: Satisfaction survey templates collection-ий бүтэц

Талбар	Төрөл	Тайлбар
title	String	Судалгааны гарчиг
description	String	Судалгааны тайлбар
active	Boolean	Patient талд ашиглах эсэх
currentVersion	Number	Одоогийн идэвхтэй version
questions	Array	Одоогийн version-ийн асуултууд
versions	Array	Өмнөх version, хугацаа, асуултын snapshot
updatedBy	ObjectId	Сүүлд зассан admin
archivedAt	Date	Архивласан огноо
archivedBy	ObjectId	Архивласан admin

satisfaction_surveys — Судалгааны хариулт

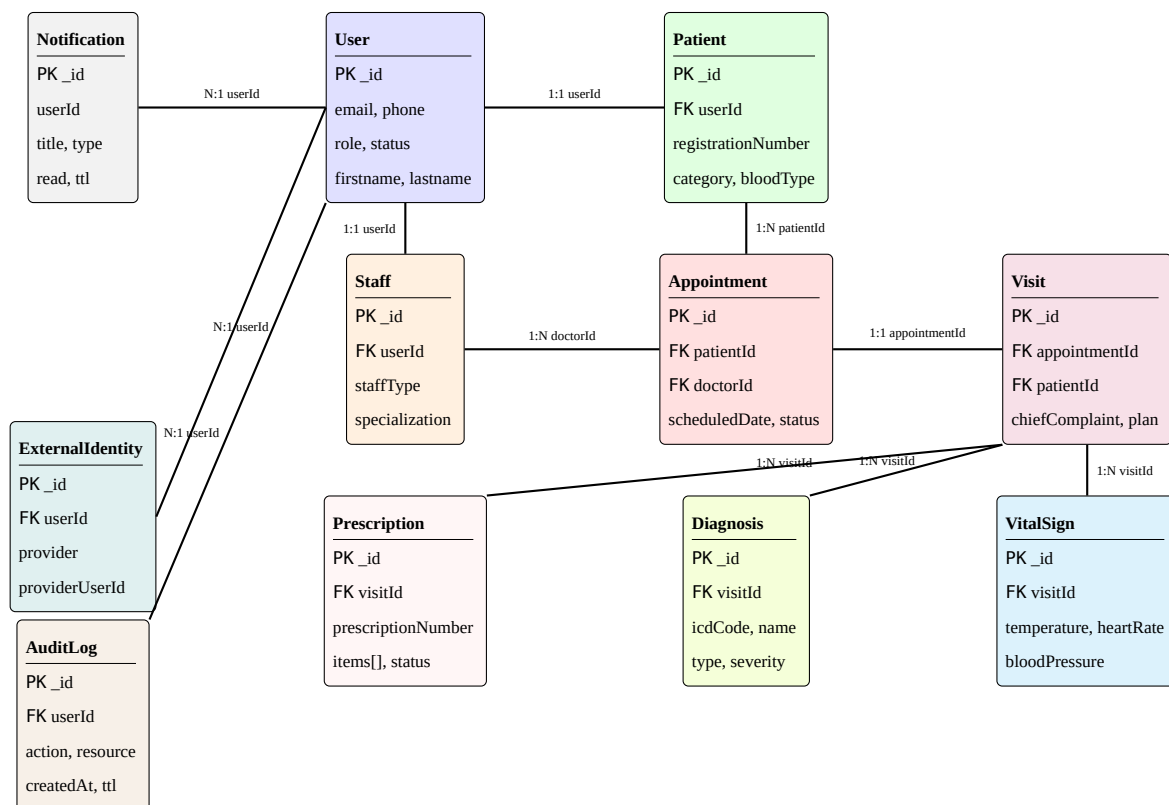
`satisfaction_surveys` collection нь patient-ийн бөглөсөн судалгааны хариултыг хадгална. Хариулт бүр тухайн үед идэвхтэй байсан template version-той холбогдох бөгөөд rating бүр дээр асуултын label, category, score-г snapshot байдлаар хадгална. Иймээс admin асуултын текстийг дараа нь өөрчилсөн ч өмнө бөглөсөн судалгааны агуулга алдагдахгүй.

Хүснэгт 3.7: Satisfaction surveys collection-ий бүтэц

Талбар	Төрөл	Тайлбар
templateId	ObjectId	Ашигласан survey template
templateVersion	Number	Бөглөсөн үеийн template version
userId	ObjectId	Системийн хэрэглэгч
patientId	ObjectId	Өвчтөний бүртгэл
occupation	String	Хэрэглэгчийн ажил/суралцагчийн төрөл
servicesReceived	Array	Авсан үйлчилгээний мэдээлэл
ratings	Array	Асуулт бүрийн label, category, score snapshot
overallRating	Number	Ерөнхий үнэлгээ
wouldReturn	Boolean	Дахин үйлчлүүлэх эсэх
wouldReturnReason	String	Шалтгаан
improvementSuggestion	String	Сайжруулах санал
createdAt	Date	Бөглөсөн огноо

3.2.3 Collection хоорондын хамаарал

Системийн гол collection-уудын хоорондын логик холбоог дараах ER диаграмд харуулав. Диаграм нь өвчтөний бүртгэлээс эхлэн цаг товлолт, үзлэг, онош, жор хүртэлх үндсэн өгөгдлийн урсгал хэрхэн холбогдож байгааг нэгтгэн үзүүлнэ.



Зураг 3.2: Өгөгдлийн сангийн ER диаграм (гол collection-ууд)

Collection-уудын гол хамаарлыг дараах байдлаар бүлэглэж ойлгож болно:

- **Хэрэглэгчийн түвшин:** **User** нь **Patient** болон **Staff**-тай 1:1 холбоотой, харин **ExternalIdentity**, **Notification**, **AuditLog** нь **User**-тай N:1 холбоотой.
- **Цаг товлолтын түвшин:** **Appointment** нь **Patient** болон **Staff** (эмч)-тай N:1 холбоотой бөгөөд үзлэг эхлэхээс өмнөх үндсэн урсгалыг илэрхийлнэ.
- **Үзлэгийн түвшин:** **Visit** нь **Appointment**-тай 1:1 холбоотой бөгөөд мөн **Patient**-тай N:1 холбоотой байна.
- **Эмнэлзүйн дэлгэрэнгүй түвшин:** **VitalSign**, **Diagnosis**, **Prescription** нь бүгд **Visit**-тай N:1 холбоотойгоор үзлэгийн үр дүнг дэлгэрүүлнэ.

- **Судалгааны түвшин:** **SatisfactionSurveyTemplate** нь олон **SatisfactionSurvey** response-той N:1 холбоотой бөгөөд response бүр тухайн template version-ийн snapshot мэдээллийг хадгална.

3.2.4 Индексжүүлэлтийн стратеги

Өгөгдлийн санд хайлт, шүүлт, хамааралт асуулгуудыг үр ашигтай гүйцэтгэхийн тулд үндсэн workflow-д тулгуурласан индексүүдийг тодорхойлсон. Индексүүд нь хэрэглэгчийн нэвтрэлт, өвчтөний хайлт, цаг товлолтын хуваарь, үзлэгийн түүх, мэдэгдэл болон аудитын бүртгэлийг хурдан уншихад чиглэнэ.

Хүснэгт 3.8: Үндсэн асуулгуудыг дэмжих индексүүд

Collection	Индекс	Зорилго
users	{email: 1} (unique), {phone: 1} (unique)	Нэвтрэлт, давхардал шалгах
external_identities	{provider: 1, providerUserId: 1} (unique)	OAuth хэрэглэгчийг холбох
patients	{userId: 1} (unique), {sisiId: 1} (sparse)	Хэрэглэгч ба өвчтөний профайл холбох
patients	{firstname: "text", lastname: "text"}, {phone: 1}	Нэрээр хайх, утсаар шүүх
staff	{userId: 1} (unique), {staffType: 1, status: 1}	Ажилтны профайл, дүрээр шүүх
appointments	{doctorId: 1, scheduledStart: 1}	Эмчийн цагийн хуваарь
appointments	{resourceId: 1, scheduledStart: 1}, {status: 1}	Нөөцийн боломжит цаг, төлвөөр шүүх

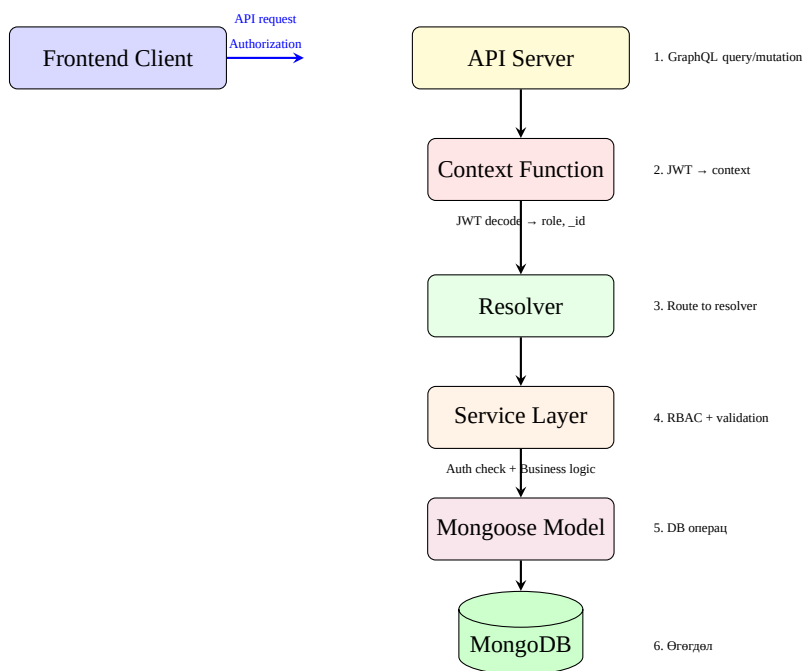
Collection	Индекс	Зорилго
appointments	{patientId: 1, scheduledStart: -1}	Өвчтөний цаг товлолтын түүх
visits	{patientId: 1, visitDate: -1}, {appointmentId: 1} (unique, sparse)	Үзлэгийн түүх, товлолттой холбох
diagnoses	{visitId: 1}, {icdCode: 1}	Үзлэгийн онош, ICD кодоор шүүх
prescriptions	{visitId: 1}, {patientId: 1, createdAt: -1}	Жорын дэлгэрэнгүй ба түүх
notifications	{userId: 1, createdAt: -1}, {read: 1}	Мэдэгдэл унших, уншсан төлөв
satisfaction_survey_templates	{active: 1}, {currentVersion: 1}	Идэвхтэй template болон version хайх
satisfaction_surveys	{userId: 1, templateId: 1, templateVersion: 1}, {createdAt: -1}	Өвчтөн тухайн version-ийг бөглөсөн эсэх, тайлангийн жагсаалт
audit_logs	{createdAt: 1} (TTL: 1 жил), {userId: 1, createdAt: -1}	Хугацаатай устгал, хэрэглэгчийн үйлдлийн мөр

Индексүүдийг бүх талбарт автоматаар нэмэхээс зайлсхийж, зөвхөн давтамж өндөртэй query болон uniqueness шаардлагатай талбарууд дээр төвлөрүүлсэн. Ингэснээр унших хурдыг сайжруулахын зэрэгцээ бичих үйлдлийн нэмэлт ачааллыг хэт өсгөхгүй байх боломжтой.

3.3 Модуль хоорондын API урсгалын зохиомж

Системийн модулиуд нь хэрэглэгчийн интерфейс, серверийн үйлчилгээний давхарга, өгөгдлийн загвар гэсэн дарааллаар холбогдоно. API-ийн гол зорилго нь модуль бүрийн өгөгдлийн гэрээг нэгэн жигд байлгах, хэрэглэгчийн эрхийг сервер талд шалгах, бизнес логикийг

өгөгдлийн загвараас тусгаарлах явдал юм. Хүсэлтийн ерөнхий урсгалыг дараах диаграмд харуулав.



Зураг 3.3: API хүсэлтийн урсгал

3.3.1 Модулийн гэрээний бүтэц

Сервер талын API гэрээ нь дараах үндсэн домэйн модулиудаар зохион байгуулагдсан:

1. **auth.typeDefs** — AuthPayload, OAuthInitPayload, RegisterUserInput, OTPInput
2. **user.typeDefs** — User type, UpdateProfileInput
3. **patient.typeDefs** — Patient type, PatientList, CreatePatientInput, UpdatePatientInput
4. **staff.typeDefs** — Staff type, WorkSchedule, CreateStaffInput
5. **appointment.typeDefs** — Appointment type, AppointmentList, AppointmentFilterInput
6. **visit.typeDefs** — Visit type, VitalSign type, CreateVisitInput, VitalSignInput

7. **diagnosis.typeDefs** — Diagnosis type, CreateDiagnosisInput
8. **prescription.typeDefs** — Prescription type, PrescriptionItem, CreatePrescriptionInput
9. **notification.typeDefs** — Notification type
10. **report.typeDefs** — MonthlyReport, тайлангийн нэгтгэл
11. **scheduling.typeDefs** — Service, Resource, unavailable block, чөлөөт цагийн тооцоолол
12. **satisfactionSurvey.typeDefs** — Survey template, version, response, requirement type
13. **Root typeDefs** — Модулиудын нийт query, command, subscription гэрээ

3.3.2 Унших үйлдлийн бүлэг

Системийн унших үйлдлүүдийг хэрэглэгчийн дүр болон домэйн модулиар ангилна:

- **Auth:** me
- **User:** getUser, listUsers
- **Patient:** searchPatients, getPatient, getPatientByRegistration, getMyPatientProfile
- **Staff:** listStaff, getStaff, getDoctors
- **Appointment:** getAppointment, listAppointments, getMyAppointments, getDoctorQueue, getAvailableSlots
- **Visit:** getVisit, listVisitsByPatient, getMyVisitHistory, getTodayVisits
- **Diagnosis:** getDiagnosesByVisit, getDiagnosesByPatient, getMyDiagnoses
- **Prescription:** getPrescription, getPrescriptionsByVisit, getPrescriptionsByPatient, getMyPrescriptions
- **Notification:** getMyNotifications

- **Satisfaction survey:** getActiveSatisfactionSurveyTemplate, getMySatisfactionSurvey, getMySatisfactionSurveyRequirement, listSatisfactionSurveys

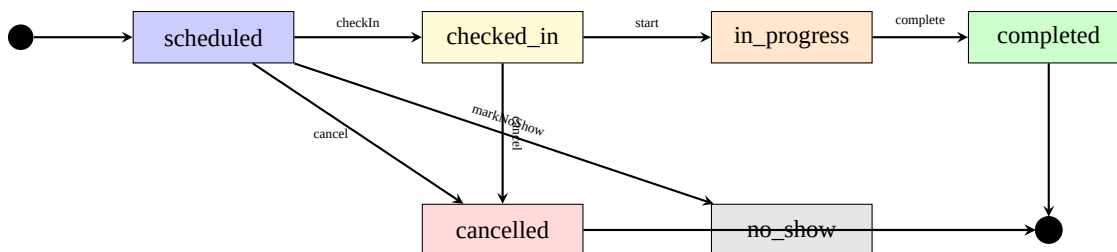
3.3.3 Өөрчлөх үйлдлийн бүлэг

Системийн өөрчлөх үйлдлүүд нь өгөгдөл үүсгэх, засах, төлөв шилжүүлэх, баталгаажуулах командуудаас бүрдэнэ:

- **Auth** (9): loginUser, sendEmailLoginOTP, loginWithEmailOTP, registerUser, changePassword, forgotPassword, verifyOTP, initiateOAuth, updateMe
- **Patient** (2): createPatient, updatePatient
- **Staff** (2): createStaff, updateStaff
- **Appointment** (6): createAppointment, checkInAppointment, cancelAppointment, startAppointment, completeAppointment, markNoShow
- **Visit** (4): createVisit, updateVisit, completeVisit, recordVitalSigns
- **Diagnosis** (3): createDiagnosis, updateDiagnosis, deleteDiagnosis
- **Prescription** (2): createPrescription, updatePrescriptionStatus
- **Notification** (1): readNotifications
- **Satisfaction survey** (3): submitSatisfactionSurvey, updateSatisfactionSurveyTemplate, removeSatisfactionSurveyTemplate

3.3.4 Appointment Status — Төлвийн диаграм

Цаг товлолтын төлвийн шилжилтийг дараах State диаграмд харуулав.



Зураг 3.4: Appointment Status — Төлвийн шилжилтийн диаграм

3.4 Нэвтрэлт ба аюулгүй байдлын зохиомж

3.4.1 JWT Authentication

Системд JSON Web Token (JWT) ашигладаг. JWT нь хэрэглэгчийн `_id`, утас, `role`, хугацаа зэрэг мэдээллийг агуулж, API хүсэлт бүр дээр хэрэглэгчийг танихад ашиглагдана. Эмнэлгийн мэдээлэл нь эмзэг өгөгдөл тул JWT нь зөвхөн session баталгаажуулалтын нэг хэсэг бөгөөд HTTPS, role check, token expiry, audit log-той хамт хэрэглэгдэх ёстой. Нэвтрэлтийн урсгал:

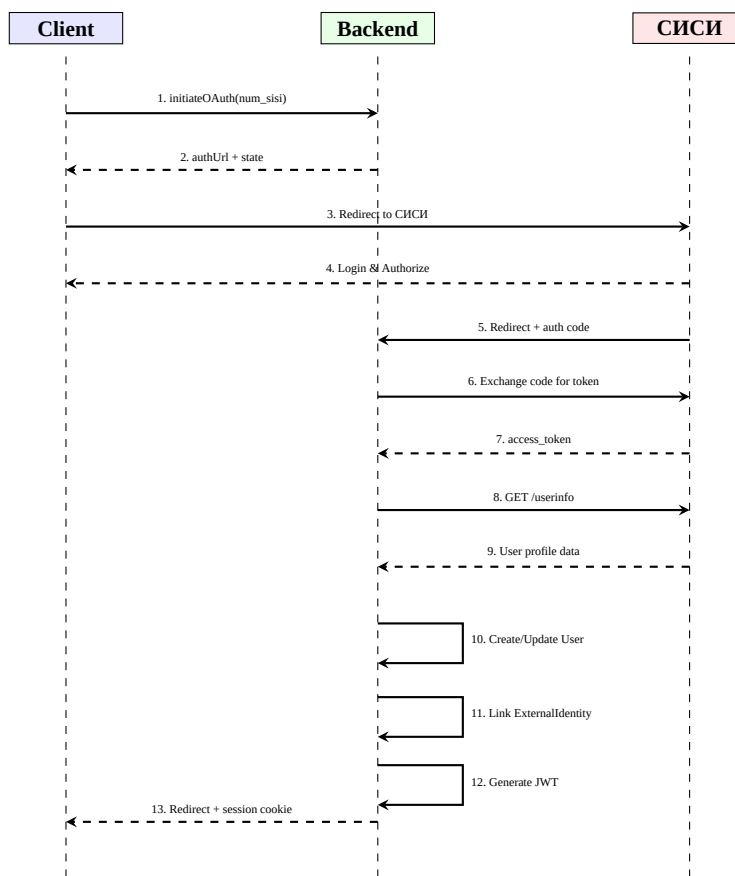
1. Хэрэглэгч утас/нууц үг эсвэл OAuth-ээр нэвтэрнэ
2. Сервер хэрэглэгчийн мэдээллийг шалгана
3. Амжилттай бол JWT token үүсгэнэ (`_id`, `phone`, `role`, `exp`)
4. Сервер token-ийг `HttpOnly`, `Secure`, `SameSite` тохиргоотой cookie хэлбэрээр клиентэд онооно
5. Клиент дараагийн хүсэлт бүрд cookie-г автоматаар дамжуулна
6. Сервер context функцэд cookie-оос token-ийг уншиж decode хийн, хэрэглэгчийн мэдээллийг resolver-т дамжуулна

Хүснэгт 3.9: Нэвтрэлт ба session-ийн аюулгүй байдлын зохиомж

Эрсдэл	Зохиомжийн шийдэл	Цаашид сайжруулах
Нууц үг алдагдах	bcrypt hashing ашиглан password-ийг plain text байдлаар хадгалахгүй	Password policy, rate limiting, account lockout
Token хулгайлагдах	JWT expiry, HttpOnly, Secure, SameSite cookie, HTTPS орчинд ажиллуулах шаардлага	Refresh token rotation, session invalidation
CSRF халдлага	Cookie-д SameSite тохиргоо ашиглаж, state-changing mutation-уудыг сервер талд шалгах	CSRF token болон origin/referrer шалгалтыг бүрэн автоматжуулах
OAuth redirect халдлага	state утга үүсгэж callback дээр шалгах	Public client ашиглах тохиолдолд PKCE нэмэх
Эрхийн хэтрэлт	Resolver/service түвшинд RBAC шалгах	Permission matrix автомат тест
Эмзэг өгөгдөлд мөрдөх чадваргүй хандах	AuditLog collection-д чухал үйлдлийг бүртгэх зохиомж	Audit event-ийн бүрэн жагсаалт, retention policy

3.4.2 OAuth 2.0 — СИСИ нэгтгэл

СИСИ OAuth 2.0 нэгтгэлийн Authorization Code Grant flow-ийг дараах Sequence диаграмд харуулав.



Зураг 3.5: СИСИ OAuth 2.0 Authorization Code Grant — Sequence Diagram

СИСИ OAuth 2.0 нэгтгэлийн Authorization Code Grant flow:

1. Клиент initiateOAuth mutation дуудна (provider: “num_sisi”)
2. Сервер state token үүсгэж, СИСИ-ийн authorize URL руу redirect URL буцаана
3. Хэрэглэгч СИСИ-д нэвтрэж, эрх зөвшөөрнө
4. СИСИ callback URL руу authorization code-тай redirect хийнэ
5. Backend callback endpoint code-ийг хүлээн аваад СИСИ-ийн token endpoint-ээс access token авна
6. Access token-оор userinfo endpoint-ээс хэрэглэгчийн мэдээлэл авна

7. ExternalIdentity record үүсгэх/шинэчлэх
8. User record үүсгэх/шинэчлэх
9. JWT token үүсгэж, HttpOnly session cookie оноосны дараа frontend руу redirect хийнэ

3.4.3 RBAC хэрэгжүүлэлт

Хандалтын хяналтыг service давхаргад хэрэгжүүлсэн. Resolver бүр context-ээс хэрэглэгчийн role-ийг авч, эрхийг шалгадаг. Жишээ нь:

- createPatient — зөвхөн doctor, superadmin
- createDiagnosis — зөвхөн doctor
- getMyAppointments — зөвхөн patient (өөрийн мэдээлэл)
- submitSatisfactionSurvey — зөвхөн patient
- updateSatisfactionSurveyTemplate — зөвхөн superadmin
- listUsers — зөвхөн superadmin

RBAC-ийн гол зарчим нь хэрэглэгчийн interface дээр товч нуух бус, сервер талын resolver болон service түвшинд эрхийг дахин шалгах явдал юм. Ингэснээр frontend-ээс зориудаар зөвшөөрөлгүй GraphQL mutation илгээсэн ч сервер тухайн үйлдлийг гүйцэтгэхгүй. Хамгаалалтын тестийн үед role бүрээр зөвшөөрөгдөх болон зөвшөөрөгдөхгүй үйлдлийг тусад нь шалгах шаардлагатай.

Хүснэгт 3.10: RBAC permission matrix-ийн жишээ

Үйлдэл	patient	nurse	doctor	superadmin
Өөрийн цаг харах	Тийм	Үгүй	Үгүй	Тийм
Өөрийн ирц баталгаажуулах	Тийм	Үгүй	Үгүй	Тийм
Өвчтөн бүртгэх/засах	Үгүй	Үгүй	Тийм	Тийм
Үйлчилгээний цаг харах	Үгүй	Тийм	Тийм	Тийм
Амин үзүүлэлт бүртгэх	Үгүй	Тийм	Тийм	Тийм
Оношлогоо үүсгэх	Үгүй	Үгүй	Тийм	Тийм
Жор бичих	Үгүй	Үгүй	Тийм	Тийм
Судалгаа бөглөх	Тийм	Үгүй	Үгүй	Үгүй
Судалгааны template удирдах	Үгүй	Үгүй	Үгүй	Тийм
Хэрэглэгч удирдах	Үгүй	Үгүй	Үгүй	Тийм

3.5 Дэлгэцийн зохиомж

3.5.1 Patient Frontend — Хуудаснууд

Өвчтөний интерфэйс нь дараах хуудаснуудтай:

1. **LoginPage** — Нэвтрэх (утас/нууц үг, СИСИ OAuth)
2. **AuthCallbackPage** — OAuth callback хүлээн авах
3. **DashboardPage** — Хяналтын самбар (ойрын цаг товлолт, сүүлийн үзлэг)
4. **AppointmentBookPage** — Цаг товлох (үйлчилгээ → эмч/нөөц → огноо → цаг)

5. **AppointmentListPage** — Цаг товлолтын жагсаалт
6. **VisitListPage** — Үзлэгийн түүх
7. **VisitDetailPage** — Үзлэгийн дэлгэрэнгүй (оношлогоо, жор, амин үзүүлэлт)
8. **PrescriptionsPage** — Жорын жагсаалт
9. **SurveyPage** — Сэтгэл ханамжийн судалгаа бөглөх, шаардлагатай үед protected page-ээс redirect хүлээн авах
10. **ProfilePage** — Хувийн мэдээлэл засварлах

3.5.2 Admin Frontend — Хуудаснууд

Удирдлагын интерфейс нь дараах хуудаснуудтай:

1. **LoginPage** — Нэвтрэх
2. **DashboardPage** — Хяналтын самбар (статистик, шуурхай мэдээлэл)
3. **PatientListPage** — Өвчтөний жагсаалт, хайлт, шинэ өвчтөн бүртгэх
4. **AppointmentListPage** — Цаг товлолтын удирдлага (шүүлтүүр, check-in, цуцлах)
5. **DoctorQueuePage** — Эмчийн дарааллын самбар
6. **ExaminePage** — Үзлэг хийх (гол зовиур, биеийн үзлэг, оношлогоо, жор)
7. **StaffListPage** — Ажилтны жагсаалт, шинэ ажилтан нэмэх
8. **SatisfactionSurveyPage** — Судалгааны template, version, archive болон patient response удирдах

3.5.3 Layout бүтэц

- **PatientLayout** — Зүүн талын навигаци (Dashboard, Appointments, Visits, Prescriptions, Survey, Profile), дээд хэсэгт хэрэглэгчийн нэр, мэдэгдлийн тоо
- **AdminLayout** — Зүүн талын навигаци (Dashboard, Patients, Appointments, Queue, Staff, Survey), дээд хэсэгт хэрэглэгчийн нэр, role badge

3.6 Бүлгийн дүгнэлт

Энэ бүлэгт системийн ерөнхий архитектур, монорепо бүтэц, backend давхаргат архитектур, өгөгдлийн загвар, модуль хоорондын API урсгал, нэвтрэлт/аюулгүй байдлын зохиомж, хоёр frontend-ийн дэлгэцийн зохиомжийг тодорхойлсон. Дараагийн бүлэгт эдгээр зохиомжийг ямар технологиор хэрэгжүүлснийг авч үзнэ.

4. ХЭРЭГЖҮҮЛЭЛТ

Энэ бүлэгт хөгжүүлэлтийн орчин, backend хэрэгжүүлэлт, frontend хэрэгжүүлэлт, өгөгдлийн сангийн backup зохион байгуулалт, хэрэглэгчийн сэтгэл ханамжийн судалгаа, тестлэлтийн дэлгэрэнгүйг тусгасан.

4.1 Хөгжүүлэлтийн орчин ба төслийн бүтэц

Энэ бүлэгт өмнөх бүлгүүдэд тодорхойлсон шаардлага, зохиомжийг бодит програм хангамж болгон хэрэгжүүлсэн байдлыг тайлбарлав. Системийн эх код нь `NUM-Hospital-service-main` нэртэй монорепо хэлбэртэй бөгөөд нэг backend service болон хоёр frontend application-оос бүрдэнэ. Ингэснээр өвчтөнд зориулсан портал, эмч/админд зориулсан удирдлагын интерфейс, өгөгдлийн сан болон API давхаргыг тусдаа хөгжүүлэх боломж бүрдсэн.

Хүснэгт 4.1: Хөгжүүлэлтийн үндсэн орчин

Бүрэлдэхүүн	Ашигласан технологи, хэрэгсэл
Backend runtime	Node.js 18.x, TypeScript, Express.js
API framework	Apollo Server 4, GraphQL, GraphQL WebSocket subscription
Өгөгдлийн сан	MongoDB, Mongoose ODM
Authentication	JWT, bcrypt password hashing, email OTP, NUM/SISI OAuth callback-ийн суурь
Patient frontend	React 18, Vite, TypeScript, Apollo Client, React Router, Tailwind CSS
Admin frontend	React 18, Vite, TypeScript, Apollo Client, React Router, Tailwind CSS
Гадаад интеграц	ICD-11 local Docker API, email/SMS/Firebase болон backup-ийн S3 тохиргооны суурь
Package manager	Backend-д Yarn 4, frontend-д npm

Төслийн файлын зохион байгуулалтыг дараах байдлаар хийсэн.

```

1 NUM-Hospital-service-main/
2   backend/
3     src/
4       graphql/      # typeDefs, resolvers, context
5       models/       # Mongoose collection загварууд
6       services/     # бизнеслогикбаэрхийншалгалт
7       routes/       # OAuth callback, health route
8       utils/        # JWT, constants, helper, backup
9       cron/         # scheduler суурь

```

```

10     index.ts      # серверийнэхлэхцэг
11 patient-frontend/
12   src/
13     pages/        # patient portal-ийн дэлгэцүүд
14     components/   # layout, common components
15     graphql/      # query, mutation
16     hooks/        # auth context
17 admin-frontend/
18   src/
19     pages/        # admin/doctor дэлгэцүүд
20     components/   # layout, ICD picker, common components
21     graphql/      # query, mutation
22     hooks/        # auth context

```

Код 4.1: Source code-ийн үндсэн бүтэц

Энэхүү бүтэц нь backend, patient frontend, admin frontend-ийг тус тусдаа ажиллуулах боломжтой. Local development үед backend нь 4000, patient frontend нь 3000, admin frontend нь 3001 порт дээр ажиллахаар тохируулагдсан. Frontend талын Vite proxy нь /graphql болон /auth хүсэлтийг backend рүү дамжуулна.

4.2 Backend хэрэгжүүлэлт

4.2.1 Серверийн эхлэх цэг ба middleware тохиргоо

Backend service нь backend/src/index.ts файлаас эхэлнэ. Энэ файлд Express application үүсгэж, HTTP server, WebSocket server, Apollo GraphQL server, MongoDB холболт, REST route болон cron scheduler-ийг нэгтгэн тохируулсан.

```

1 const app = express();
2 const httpServer = createServer(app);

```

```
3
4 const wsServer = new WebSocketServer({
5   server: httpServer,
6   path: "/subscriptions",
7 });
8
9 const serverCleanup = useServer({ schema, context }, wsServer);
10
11 const server = new ApolloServer({
12   typeDefs,
13   resolvers,
14   csrfPrevention: NODE_ENV !== "development",
15   plugins: [
16     ApolloServerPluginDrainHttpServer({ httpServer }),
17     {
18       async serverWillStart() {
19         return { async drainServer() { await serverCleanup.dispose(); }
20           };
21       },
22     },
23   ],
24 });
```

Код 4.2: Backend server-ийн үндсэн тохиргоо

Серверийн middleware тохиргоонд CORS, JSON body parser, URL encoded parser, file upload middleware, health route болон OAuth route бартсан. Production орчинд зөвшөөрөгдсөн frontend origin-уудыг PATIENT_FRONTEND_URL, ADMIN_FRONTEND_URL тохиргоогоор хязгаарлана. Мөн graphql-upload ашиглан 10MB хүртэлх файл upload хүлээн авч,

application server-ийн дотоод хадгалалтад байрлуулах суурь тохиргоо хийгдсэн.

4.2.2 GraphQL API-ийн зохион байгуулалт

GraphQL schema нь `backend/src/graphql/typeDefs` дотор domain бүрээр тусдаа файлд хуваагдсан. Үндсэн schema файлд root Query, Mutation, Subscription-уудыг нэгтгэж, дараах domain-уудыг API contract болгон тодорхойлсон.

Хүснэгт 4.2: GraphQL API-ийн domain-ууд

Domain	Үндсэн API боломж
Auth/User	login, register, email OTP, OAuth initiate, profile update, user list
Patient	өвчтөн хайх, өвчтөн бүртгэх, мэдээлэл шинэчлэх, өөрийн profile бөглөх
Staff	ажилтан, эмч, сувилагч, системийн админы бүртгэл
Service/Resource	үйлчилгээ, эмч/сувилагч/өрөө/төхөөрөмжийн нөөц, default scheduling seed
Appointment	цаг авах, цагийн жагсаалт, check-in, эхлүүлэх, дуусгах, цуцлах, no-show
Visit	үзлэг үүсгэх, үзлэгийн тэмдэглэл шинэчлэх, үзлэг дуусгах, амин үзүүлэлт бүртгэх
Diagnosis	ICD-11 metadata-тай онош нэмэх, засах, устгах
Prescription	жор үүсгэх, жорын төлөв шинэчлэх, өвчтөний жорын түүх харах
Notification	мэдэгдэл харах, уншсан төлөв болгох, subscription суурь
ICD-11	root chapter, children, search query-г backend proxy-оор дамжуулах
Report	сарын тайлан: нас, хүйс, үйлчилгээ, нөөц, эмчээр нэгтгэх

Resolver давхарга нь request-ийг шууд өгөгдлийн сантай холбохгүй, харин service давхаргын функцүүдийг дууддаг. Жишээлбэл `createAppointment`, `checkInAppointment`, `startAppointment`, `completeAppointment` зэрэг mutation

нь `appointmentService.ts` дотор байрлах бизнес логигоор хэрэгжинэ. Энэ нь API contract, бизнес логик, өгөгдлийн сангийн загварыг тусгаарлаж, цаашид test хийх болон өргөтгөх боломжийг нэмэгдүүлнэ.

4.2.3 Давхаргат backend архитектур

Backend-ийн кодын зохион байгуулалтыг дөрвөн үндсэн давхаргад хуваасан.

1. **API давхарга:** GraphQL typeDefs, resolvers, REST route.
2. **Service давхарга:** validation, authorization, scheduling, report, ICD proxy зэрэг бизнес логик.
3. **Model давхарга:** Mongoose schema, collection-ийн индекс, relation.
4. **Utility давхарга:** JWT, bcrypt, constants, helper function, email/SMS болон backup-ийн тохиргоо.

Энэ зохион байгуулалт нь кодын хамаарлыг багасгаж, нэг module өөрчлөгдөхөд бусад module-д үзүүлэх нөлөөг багасгах зорилготой. Жишээ нь цаг товлолтын давхардал шалгах логик `schedulingService.ts` дотор төвлөрсөн тул patient frontend болон admin frontend хоёулаа ижил дүрмээр чөлөөт цаг авна.

4.2.4 Өгөгдлийн загварын хэрэгжүүлэлт

MongoDB өгөгдлийн санг Mongoose ODM ашиглан загварчилсан. Системийн үндсэн collection-уудыг хүснэгт 4.3-д үзүүлэв.

Хүснэгт 4.3: Backend model болон collection-ууд

Model	Collection	Үүрэг
User	users	Нэвтрэх данс, role, contact, status
ExternalIdentity	external_identities	NUM/SISI зэрэг external provider-той холбосон хэрэглэгч
Patient	patients	Өвчтөний demographic болон эмнэлгийн profile
Staff	staff	Эмч, сувилагч, админ ажилтны мэдээлэл
Service	services	Эмчийн үзлэг, тариа, дусал, шарлага, массаж зэрэг үйлчилгээ
Resource	resources	Эмч, сувилагч, өрөө, capacity room, төхөөрөмжийн цагийн нөөц
UnavailableBlock	unavailable_blocks	Эмч/нөөцийн боломжгүй хугацаа
Appointment	appointments	Үйлчилгээ болон нөөцөд суурилсан цаг товлолт
Visit	visits	Эмчийн үзлэгийн үндсэн тэмдэглэл
VitalSign	vital_signs	Амин үзүүлэлтийн хэмжилт
Diagnosis	diagnoses	ICD-11 мэдээлэлтэй онош
Prescription	prescriptions	Жор болон эмийн заавар
Notification	notifications	Хэрэглэгчийн мэдэгдэл, уншсан төлөв
AuditLog	audit_logs	Чухал үйлдлийн бүртгэл

Жишээ болгон Appointment model нь уламжлалт эмчид суурилсан цаг авах хэлбэрээс гадна үйлчилгээ ба нөөцөд суурилсан цаг авах хэлбэрийг дэмждэг. Үүнд serviceId, resourceId, scheduledStart, scheduledEnd, blockedUntil, durationMinutes, bufferMinutes, appointmentKind зэрэг талбарууд ашиглагдана.

```
1 serviceId: { type: ObjectId, ref: "Service" },
2 resourceId: { type: ObjectId, ref: "Resource" },
3 scheduledDate: { type: Date, required: true },
4 scheduledTime: { type: String, required: true },
5 scheduledStart: { type: Date },
6 scheduledEnd: { type: Date },
7 blockedUntil: { type: Date },
8 durationMinutes: { type: Number },
9 bufferMinutes: { type: Number, default: 0 },
10 seatNumber: { type: Number },
11 appointmentKind: {
12   type: String,
13   enum: ["consultation", "injection", "infusion", "procedure", "device",
14         , "lab", "follow_up", "other"],
15   default: "consultation",
16 }
```

Код 4.3: Appointment model-ийн resource-based scheduling талбарууд

4.2.5 Нэвтрэлт ба эрхийн шалгалтын хэрэгжүүлэлт

Нэвтрэлтийн хэсэг нь JWT session, bcrypt password hashing, email OTP, OAuth callback гэсэн хэд хэдэн механизмаас бүрдэнэ. Дотоод ажилтан болон админ хэрэглэгчид утасны дугаар, нууц үгээр нэвтрэх боломжтой. Нууц үгийг `bcrypt.hashSync(password, 10)`

функцээр hash хийж хадгална.

JWT token-д хэрэглэгчийн `_id`, `phone`, `role` мэдээллийг агуулна. Token-г frontend-ийн `localStorage`-д хадгалахгүй, харин серверээс `HttpOnly`, `Secure`, `SameSite` тохиргоотой cookie хэлбэрээр оноож, GraphQL context дээр cookie-оос уншин request бүрд эрхийн мэдээлэл болгон ашиглана.

```
1 const token = req.cookies?.accessToken || "";
2 if (token) {
3   const decoded = decodeToken(token);
4   return {
5     ...decoded,
6     authenticated: true,
7     pubsub,
8     req,
9     res,
10  };
11 }
```

Код 4.4: HttpOnly cookie-д суурилсан JWT шалгалт ба context үүсгэх хэлбэр

Role-based access control буюу дүрд суурилсан эрхийн шалгалтыг `requireAuth`, `requireRole` helper функцүүдээр хийсэн. Судалгааны баримт бичигт өгөгдөл оруулагч эсвэл дотоод нэгжийн админ гэсэн тусдаа role тодорхойлохгүй; staff management, service/resource үүсгэх зэрэг үйлдэлд зөвхөн `superadmin` эрх шаарддаг. Харин үзлэг хийх, онош бичих, тайлан харах зэрэг үйлдлүүдэд `doctor` болон `superadmin` эрх ашиглагдана.

Patient portal дээр МУИС-ийн имэйлтэй хэрэглэгчид email OTP ашиглан нэвтрэх боломжтой. `sendEmailLoginOTP` mutation нь зөвхөн `@num.edu.mn`, `@stud.num.edu.mn` домэйнтэй имэйлийг зөвшөөрөхөөр шалгадаг. OTP нь MVP түвшинд memory store-д 5 минутын хугацаатай хадгалагдана. Production орчинд Redis зэрэг төвлөрсөн cache ашиглах шаардла-

гатай.

NUM/SISI OAuth login-ийн суурь нь REST route хэлбэрээр хэрэгжсэн. /auth/sisi endpoint нь authorization URL руу redirect хийж, /auth/sisi/callback endpoint нь authorization code-г token болгон солих, userinfo авах, local user болон ExternalIdentity үүсгэх, JWT session cookie оноон frontend рүү redirect хийх дарааллаар ажиллана.

4.2.6 Үйлчилгээ ба нөөцөд суурилсан цаг товлолт

Энэ системийн гол онцлог нь цаг товлолтыг зөвхөн эмчийн цаг гэж үзэхгүй, харин **үйлчилгээ + нөөц + хугацаа** гэсэн загвараар хэрэгжүүлсэн явдал юм. Үйлчилгээ нь эмч, сувилагч эсвэл төхөөрөмжийн аль нэгэнд хамаарах бөгөөд resource нь тухайн үйлчилгээг гүйцэтгэх бодит нөөцийг илэрхийлнэ.

Хүснэгт 4.4: Үйлчилгээ ба нөөцийн scheduling загвар

Ойлголт	Жишээ	Хэрэгжүүлэлтийн тайлбар
Service	Эмчийн үзлэг, тариа, дусал, шарлага, массаж	Үйлчилгээний нэр, code, category, default duration, buffer, шаардлагатай staff/device
Resource	Эмч, сувилагч, өрөө, төхөөрөмж	Цаг авах боломжтой бодит нөөц; capacity, work schedule, slot interval-тай
Capacity resource	Дуслын өрөө	Нэг хугацаанд хэд хэдэн өвчтөн авах боломжтой; default capacity нь 8
Device resource	Шарлагын төхөөрөмж, массажны сандал	Нэг хугацаанд нэг хэрэглэгч; ажил дууссаны дараах buffer minute-ийг тооцно
Unavailable block	Эмчийн чөлөө, төхөөрөмж засвар, өрөө хаалт	Тухайн хугацаанд цаг авах боломжгүй болгож хаана

`seedDefaultScheduling` mutation нь system admin-д зориулсан default seed бөгөөд дараах үйлчилгээ, нөөцүүдийг үүсгэх суурьтай.

- `DOCTOR_CONSULT` — Эмчийн үзлэг, 20 минут, эмч шаардлагатай;
- `NURSE_INJECTION` — Сувилагчийн тариа, 15 минут, сувилагч шаардлагатай;
- `IV_INFUSION` — Дусал, 90 минут, сувилагч шаардлагатай;
- `IV_ACCESS` — Гар судас тариа, 20 минут;
- `IRRADIATION` — Шарлага, 20 минут + 10 минут buffer, төхөөрөмж шаардлагатай;

- `MESSAGE_CHAIR` — Массажны сандал, 30 минут + 10 минут `buffer`, төхөөрөмж шаардлагатай;
- Дуслын өрөө — `capacity_room`, `capacity = 8`;
- Шарлагын төхөөрөмж 1, Массажны сандал 1 — `device`, `capacity = 1`.

Чөлөөт цаг тооцоолох үндсэн функц нь `schedulingService.ts` дотор байрлах `getAvailableSlots` юм. Энэ функц үйлчилгээ, `resource`, `date` оролтоор тухайн өдрийн боломжит `slot`-уудыг үүсгэж, аль нь чөлөөтэй, аль нь захиалгатай, аль нь хаалттай, аль нь цайны цаг эсвэл өнгөрсөн цаг болохыг ялгана.

```
1 const duration = resource.defaultDurationMinutes || service?.
  defaultDurationMinutes || 30;
2 const buffer = resource.defaultBufferMinutes ?? service?.
  defaultBufferMinutes ?? 0;
3 const interval = resource.slotIntervalMinutes || duration + buffer ||
  30;
4 const capacity = resource.capacity || 1;
5
6 for (let cursor = startOfWork; addMinutes(cursor, duration + buffer) <=
  endOfWork; cursor = addMinutes(cursor, interval)) {
7   const slotEnd = addMinutes(cursor, duration);
8   const blockedUntil = addMinutes(slotEnd, buffer);
9   const overlapCount = existing.filter((appointment) =>
10     overlaps(existingStart, existingBlockedUntil, cursor, blockedUntil)
11   ).length;
12
13   if (overlapCount >= capacity) status = "booked";
14 }
```

Код 4.5: Цагийн боломж шалгах үндсэн санаа

Цаг үүсгэх үед `createAppointment` mutation нь эхлээд өвчтөний profile бүрэн эсэхийг шалгана. Дараа нь `assertResourceAvailability` функцээр тухайн service/resource дээр давхардал, цайны цаг, өнгөрсөн цаг, unavailable block, capacity хэтэрсэн эсэхийг шалгаад appointment үүсгэнэ.

1. Өвчтөний бүртгэл байгаа эсэхийг шалгах;
2. Цаг авахад шаардлагатай profile талбарууд бөглөгдсөн эсэхийг шалгах;
3. Сонгосон service/resource-ийн duration, buffer, capacity-г тодорхойлох;
4. Unavailable block болон цайны цагтай давхцаж байгаа эсэхийг шалгах;
5. Өмнөх appointment-уудтай overlap тооцож capacity хүрсэн эсэхийг шалгах;
6. Давхардалгүй бол `scheduled` төлөвтэй appointment үүсгэх;
7. Тухайн өдрийн дарааллын дугаарыг `queueNumber` талбарт оноох;
8. Audit log-д `appointment` үүсгэсэн үйлдлийг бүртгэх.

4.2.7 Unavailable block буюу цаг хаах хэрэгжүүлэлт

Эмч, сувилагч эсвэл system admin нь тодорхой хугацаанд цаг авах боломжгүй болгохын тулд unavailable block үүсгэнэ. Эмч/сувилагч нь зөвхөн өөрийн schedule-д block үүсгэх боломжтой, харин superadmin нь бүх resource болон staff дээр block үүсгэж болно.

`createUnavailableBlock` функц нь start/end хугацааг шалгаж, тухайн хугацаанд `scheduled` эсвэл `checked_in` төлөвтэй appointment байвал тэдгээр appointment-ийг цуцалж, block-ийн `cancelledAppointmentIds` талбарт холбох боломжтой байдлаар зохиомжлогдсон. Энэ нь эмчийн чөлөө, төхөөрөмжийн засвар, өрөөний хаалт зэрэг бодит workflow-д хэрэгтэй.

4.2.8 Эмчийн үзлэг, онош, жорын хэрэгжүүлэлт

Эмчийн үзлэгийн module нь Visit, VitalSign, Diagnosis, Prescription collection-ууд дээр тулгуурлана. DoctorQueuePage дээр тухайн өдрийн appointment-уудыг харуулж, эмч startAppointment болон createVisit mutation-уудыг ашиглан үзлэг эхлүүлнэ. Үзлэгийн дэлгэц ExaminePage нь дараах мэдээллийг нэг workflow-д нэгтгэсэн.

- өвчтөний үндсэн мэдээлэл, өмнөх үзлэгийн түүх;
- амин үзүүлэлт: халуун, даралт, зүрхний цохилт, амьсгал, хүчилтөрөгч, жин, өндөр;
- гол зовуурь, өвчний одоогийн түүх, өмнөх өвчний түүх;
- биеийн үзлэг, үнэлгээ, төлөвлөгөө, эмчийн зөвлөгөө;
- ICD-11 metadata бүхий онош;
- жорын дугаар, эмийн нэр, тун, давтамж, хугацаа, заавар;
- үзлэг дуусгах үед completed төлөвт шилжүүлэх.

Diagnosis module нь ICD-11-ийн код, нэр, хувилбар, linearization, URI зэрэг metadata-г хадгалдаг. Ингэснээр зөвхөн текст онош хадгалахаас гадна олон улсын өвчний ангиллын мэдээлэлтэй холбох боломжтой болсон.

4.2.9 ICD-11 интеграцийн хэрэгжүүлэлт

ICD-11 integration нь browser-оос WHO API руу шууд хандахгүй, backend proxy-р дамждаг. Backend талын icdService.ts нь local Docker API-ийн baseUrl, release, linearization, language тохиргоог environment-ээс авч, дараах GraphQL query-уудыг exposed хийсэн.

- icd11RootChapters(language) — ICD-11 root chapter-уудыг авах;

- `icd11Children(uri, language)` — сонгосон бүлгийн child entity-үүдийг авах;
- `icd11Search(q, language)` — оношийн нэр/код хайх.

Admin frontend талд `Icd11Picker.tsx` component нь ICD хайлт, бүлгээр navigation хийх, сонгосон оношийн code/title/URI-г `createDiagnosis` mutation-д дамжуулах үүрэгтэй. Энэ нь эмчийн үзлэгийн хуудасны онош нэмэх хэсэгт ашиглагдана.

4.2.10 Сарын тайлан, мэдэгдэл ба *audit log*

Сарын тайлангийн module нь `monthlyReport(month: String!)` query-ээр ажиллана. `reportService.ts` нь тухайн сарын `completed` төлөвтэй appointment болон visit өгөгдлийг авч, дараах байдлаар нэгтгэнэ.

- нийт дууссан appointment;
- нийт дууссан doctor visit;
- насны бүлэг;
- хүйс;
- үйлчилгээ;
- resource буюу өрөө/төхөөрөмж;
- эмчээр нэгтгэл;
- нас-хүйсийн matrix хэлбэрийн тайлан.

Мэдэгдлийн module нь `Notification` model болон GraphQL subscription-ийн суурьтай. `notification` subscription нь PubSub channel ашигладаг бөгөөд цаашид appointment reminder, schedule change, treatment request зэрэг бодит цагийн мэдэгдлийг өргөтгөх боломжтой.

Audit log нь **AuditLog** collection дээр хэрэгжсэн. User login, user create, appointment create, service/resource create, unavailable block create/update/cancel зэрэг чухал үйлдлүүдийг **logAudit** функцээр бүртгэнэ. Audit log-д **userId**, **action**, **resource**, **resourceId**, **details**, **IP address**, **userAgent** зэрэг мэдээллийг хадгалах боломжтой. Мөн **createdAt** талбарт 1 жилийн TTL index тохируулсан.

4.3 Patient frontend хэрэгжүүлэлт

Patient frontend нь өвчтөн өөрөө profile бөглөж, үйлчилгээ сонгон цаг авах, өөрийн appointment, үзлэгийн түүх, жорын мэдээллээ харах зориулалттай React application юм. Route хамгаалалтыг **ProtectedRoute** component-аар хийж, нэвтрээгүй хэрэглэгчийг **/login** хуудас руу буцаана.

1	/login	# Нэвтрэх
2	/auth/callback	# OAuth callback token авах
3	/	# Dashboard
4	/profile	# Өвчтөний profile
5	/appointments	# Минийцагууд
6	/appointments/book	# Цагавах
7	/appointments/:id	# Цагийн дэлгэрэнгүй
8	/visits	# Үзлэгийн түүх
9	/visits/:id	# Үзлэгийн дэлгэрэнгүй
10	/prescriptions	# Жорын жагсаалт

Код 4.6: Patient frontend-ийн үндсэн route-ууд

4.3.1 Өвчтөний нэвтрэлт ба profile

Patient frontend-ийн login workflow нь email OTP болон OAuth callback-ийн суурьтай. **SEND_EMAIL_LOGIN_OTP** mutation-оор МУИС-ийн имэйл рүү OTP илгээж,

LOGIN_WITH_EMAIL_OTP mutation амжилттай болох үед backend HttpOnly session cookie онооно. Иймээс frontend token-г localStorage-д хадгалахгүй бөгөөд Apollo Client хүсэлт бүрд cookie-г browser автоматаар дамжуулна.

Өвчтөн цаг авахын өмнө өөрийн profile-ийг бөглөх шаардлагатай. AppointmentBookPage.tsx дээр profile form эхний алхам болгон орж, upsertMyPatientProfile mutation-аар хадгална. Profile-д үндсэн мэдээллээс гадна цусны бүлэг, харшил, архаг өвчин, тогтмол хэрэглэдэг эм, эмнэлгийн анхааруулга, яаралтай холбоо барих хүний мэдээлэл багтсан.

4.3.2 Цаг авах workflow

Patient portal-ийн цаг авах хуудас нь таван алхамтай wizard хэлбэрээр хэрэгжсэн.

1. **Миний мэдээлэл** — profile бөглөх/баталгаажуулах;
2. **Үйлчилгээ** — эмчийн үзлэг, дусал, тариа, шарлага, массаж зэрэг үйлчилгээ сонгох;
3. **Сонголт** — тухайн үйлчилгээнд тохирох эмч, сувилагч эсвэл төхөөрөмж/resource сонгох;
4. **Өдөр & Цаг** — getAvailableSlots query-ээр боломжит slot харах;
5. **Баталгаажуулах** — createAppointment mutation-аар цаг үүсгэх.

Slot бүр нь time, available, status, reason, remaining, capacity, startsAt, endsAt талбаруудтай. Иймээс patient portal дээр дуслын өрөөний үлдсэн суудал, төхөөрөмжийн buffer-тэй хаалт, цайны цаг, захиалгатай цагийг ялгаж харуулах боломжтой.

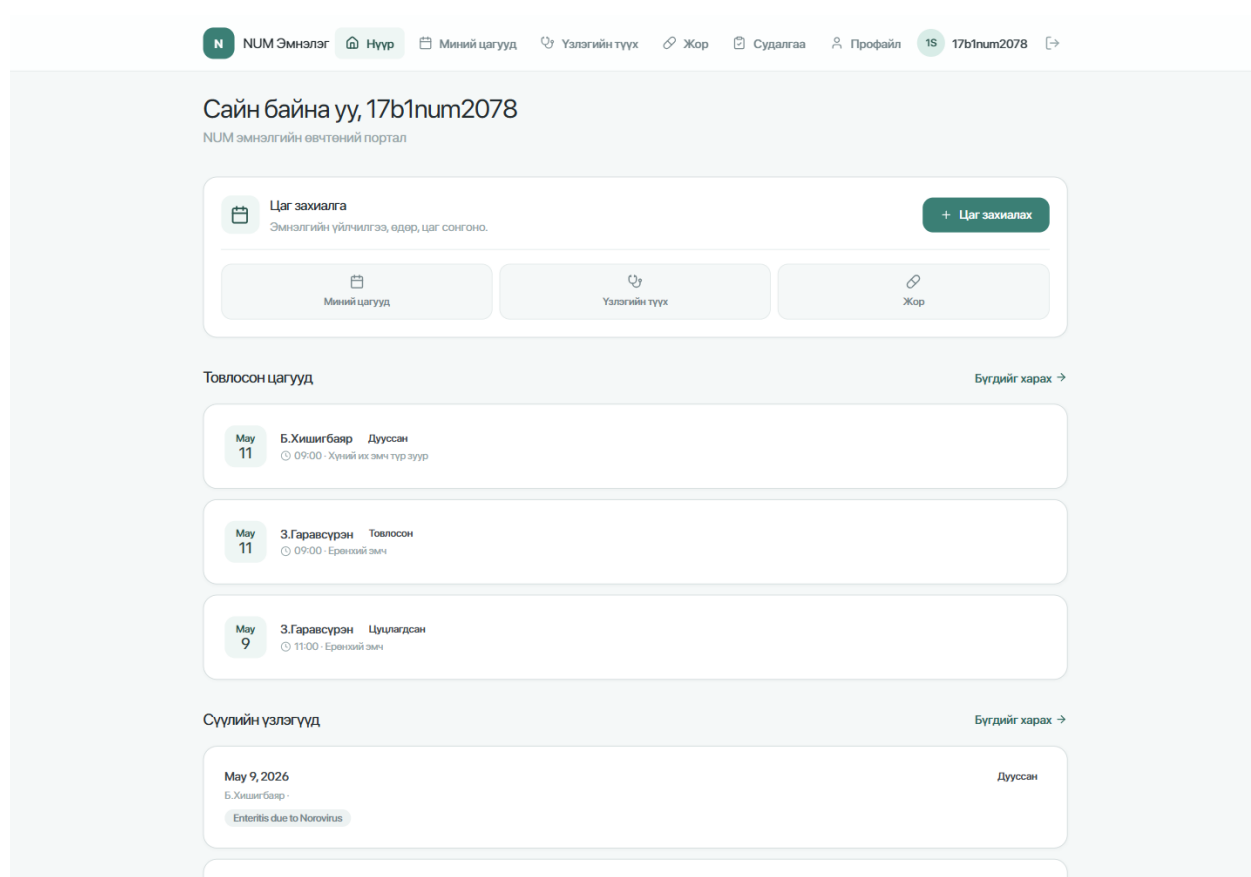
4.3.3 Өөрийн мэдээлэл харах дэлгэцүүд

Patient frontend нь цаг захиалгын жагсаалт, цагийн дэлгэрэнгүй, үзлэгийн түүх, үзлэгийн дэлгэрэнгүй, жорын жагсаалт гэсэн дэлгэцүүдтэй. Өвчтөн зөвхөн өөрийн хэ-

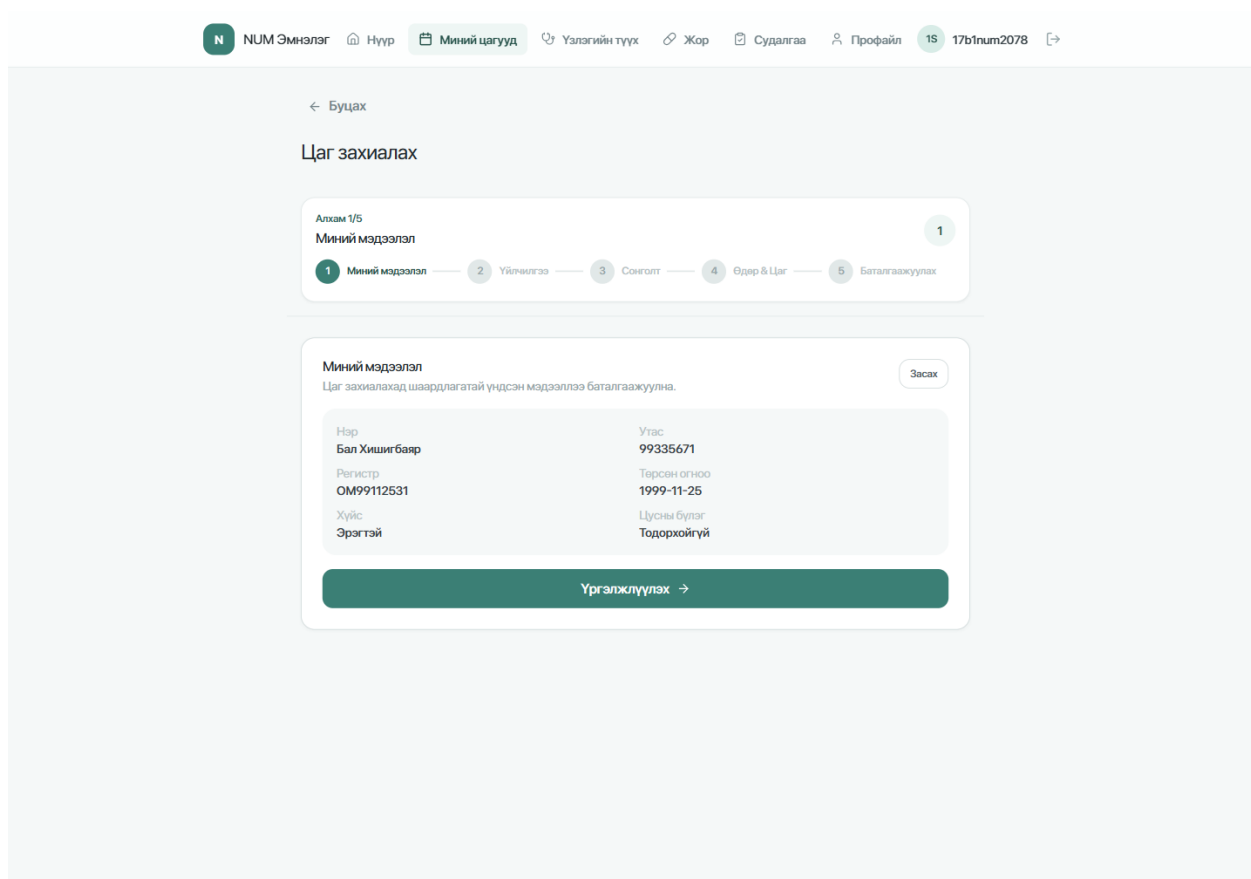
рэглэгчийн token-оор холбогдсон patient record-той холбоотой мэдээллийг харах зориулалттай query-уудыг ашиглана. Үүнд `getMyAppointments`, `getMyVisitHistory`, `getMyVisitByAppointment`, `getMyPrescriptions` зэрэг query багтана.

4.3.4 Patient frontend-ийн бодит дэлгэцүүд

Patient portal-ийн үндсэн дэлгэцүүдийг live server дээр ажиллаж буй хувилбараас авч, зураг 4.1–4.3-д харуулав. Эдгээр дэлгэц нь өвчтөн өөрийн мэдээлэл, цаг товлолт, үзлэгийн түүх болон жорын мэдээлэл рүү нэг навигацаар хандах боломжтойг харуулж байна.



Зураг 4.1: Patient portal — хяналтын самбар



NUM Эмнэлэг Нүүр Миний цагууд Үзлэгийн түүх Жор Судалгаа Профайл 1S 17b1num2078

← Буцах

Цаг захиалах

Алхам 1/5
Миний мэдээлэл

1 Миний мэдээлэл 2 Үйлчилгээ 3 Сонголт 4 Өдөр & Цаг 5 Баталгаажуулах

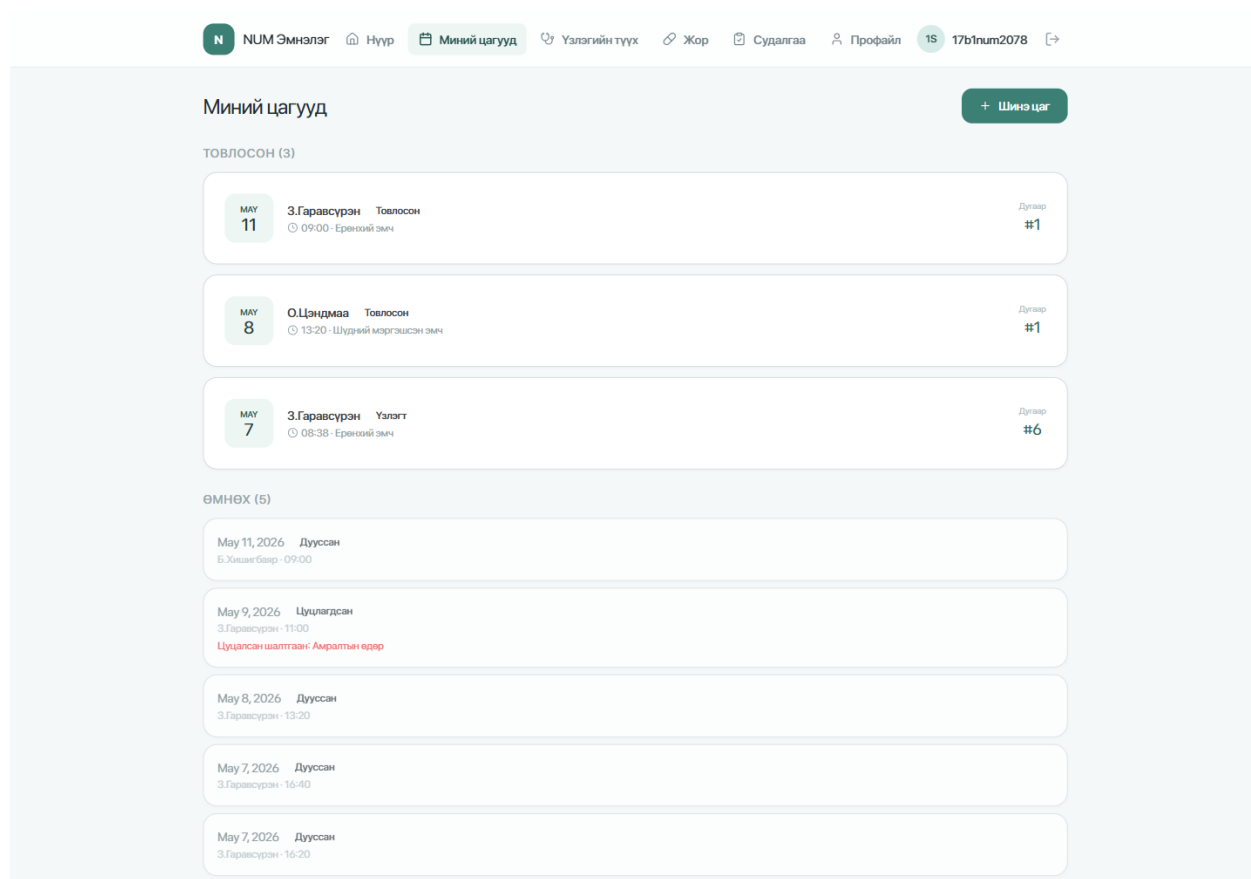
Миний мэдээлэл
Цаг захиалахад шаардлагатай үндсэн мэдээллээ баталгаажуулна.

Засах

Нэр	Утас
Бал Хишигбаяр	99335671
Регистр	Төрсөн огноо
ОМ99112531	1999-11-25
Хүйс	Цусны бүлэг
Эрэгтэй	Тодорхойгүй

Үргэлжлүүлэх →

Зураг 4.2: Patient portal — цаг захиалах workflow-ийн profile баталгаажуулах алхам



Зураг 4.3: Patient portal — өвчтөний цаг товлолтын жагсаалт

4.4 Admin frontend хэрэгжүүлэлт

Admin frontend нь эмч, сувилагч, system admin-д зориулсан удирдлагын web application юм. Энэ application нь role-д суурилсан route хамгаалалттай бөгөөд doctor, nurse, superadmin эрхүүдээр дэлгэцийн хандалтыг ялгана. Тусдаа өгөгдөл оруулагчийн role болон дотоод нэгжийн удирдлагын дэлгэц тодорхойлоогүй бөгөөд өвчтөн/цаг товлолтын төлөвийн ажиллагаа үндсэн role-уудын эрхээр шийдэгдэнэ.

1	/login	# Staff login
2	/	# Dashboard
3	/patients	# Өвчтөнхайх , бүртгэх
4	/patients/:id	# Өвчтөнийдэлгэрэнгүй , түүх

5	/appointments	# Цаг товлолтын жагсаалт
6	/appointments/:id	# Цагийн дэлгэрэнгүй
7	/schedule/unavailable	# Цаг хаах , хаалтцуулах
8	/doctor/queue	# Эмчийн тухайн өдрийн дараалал
9	/doctor/examine/:visitId	# Эмчийн үзлэг хийх дэлгэц
10	/staff	# Ажилтан удирдах
11	/services	# Үйлчилгээ ба нөөц удирдах
12	/reports	# Сарын тайлан

Код 4.7: Admin frontend-ийн үндсэн route-ууд

4.4.1 Өвчтөн ба цаг товлолтын удирдлага

`PatientListPage` нь `searchPatients query` ашиглан өвчтөнийг нэр, утас, бүртгэлийн дугаараар хайна. Мөн `createPatient mutation`-аар шинэ өвчтөн бүртгэх боломжтой. `PatientDetailPage` нь өвчтөний дэлгэрэнгүй мэдээлэл, үзлэгийн түүх, жорын түүхийг нэгтгэн харуулна.

`AppointmentListPage` нь `appointment`-уудыг огноо, төлөв, `service`, `resource` зэрэг `filter`-ээр харах боломжтой. `Appointment` дээр `check-in` хийх, цуцлах, дэлгэрэнгүй харах үйлдэл багтсан. `AppointmentDetailPage` нь өвчтөн, `service`, `resource`, эмч, төлөв, цуцлах шалтгаан зэрэг мэдээллийг харуулна.

4.4.2 Эмчийн дараалал ба үзлэгийн дэлгэц

`DoctorQueuePage` нь тухайн өдрийн эмчийн `appointment`-уудыг дарааллын дугаар, цаг, өвчтөн, төлөвөөр харуулна. Эмч `appointment`-ийг эхлүүлэх үед `backend` дээр `startAppointment mutation` дуудагдаж, дараа нь `createVisit mutation`-аар үзлэгийн `draft record` үүсгэн `/doctor/examine/:visitId` хуудас руу шилжинэ.

`ExaminePage` нь эмчийн өдөр тутмын гол ажлын дэлгэц бөгөөд нэг хуудсанд өвчтөн

ний мэдээлэл, өмнөх түүх, амин үзүүлэлт, онош, жор, үнэлгээ, төлөвлөгөөг оруулах боломжтой. ICD-11 picker component нь энэ хуудсанд холбогдон ажиллаж, сонгосон оношийн code/title/URI-г createDiagnosis mutation-д дамжуулдаг.

4.4.3 Үйлчилгээ, нөөц ба schedule хаалт

ServicesPage нь listServices, listResources query болон createService, updateService, createResource, seedDefaultScheduling mutation-уудыг ашиглана. System admin үйлчилгээний default duration, buffer, service owner type, assigned staff, resource capacity, work schedule зэрэг мэдээллийг тохируулах боломжтой.

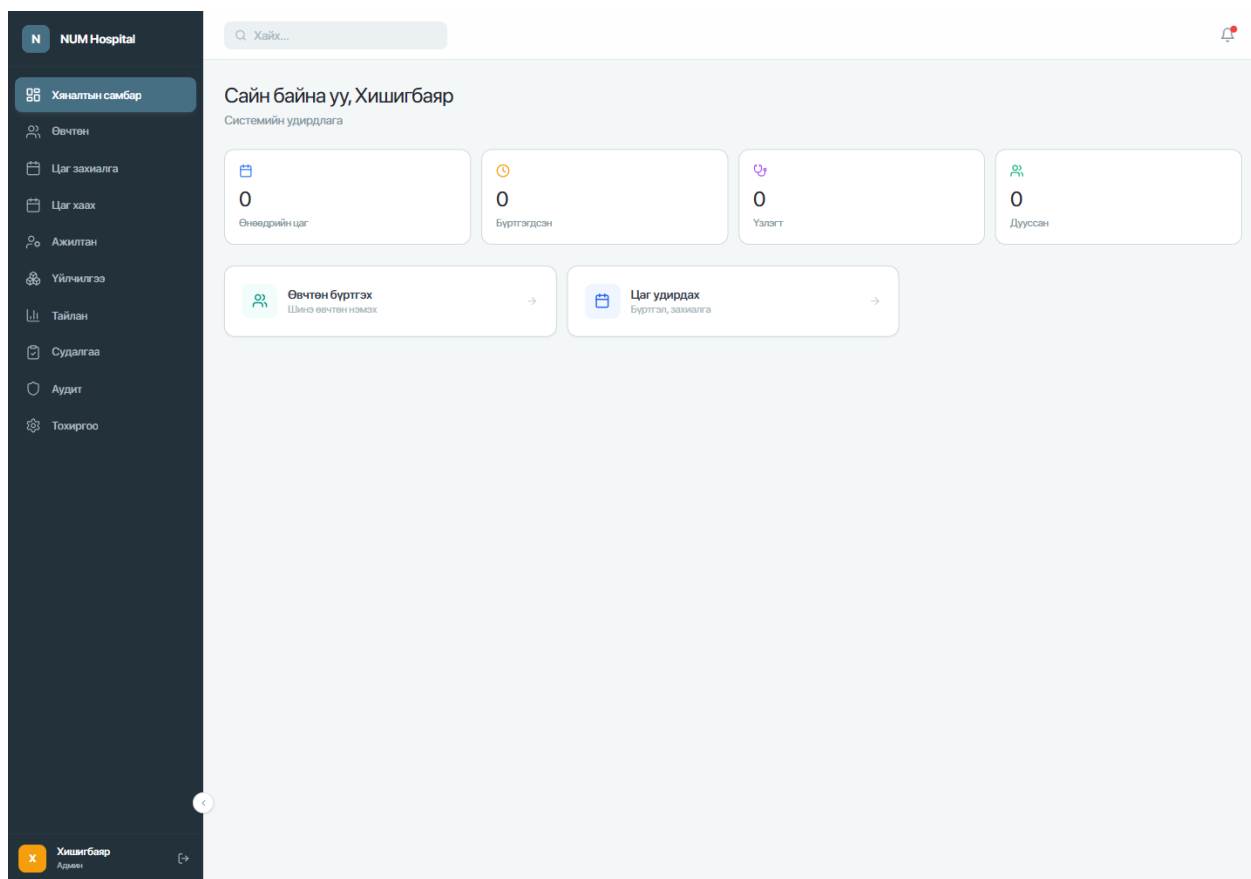
UnavailableBlocksPage нь эмч, сувилагч, system admin-д зориулсан цаг хаах дэлгэц юм. Энэ дэлгэцээр resource эсвэл staff сонгож тодорхой хугацааг хаах, хаалтыг засах, цуцлах боломжтой. Backend дээр role болон owner шалгалт хийгддэг тул эмч/сувилагч өөрийн үүсгэсэн хаалтыг л өөрчлөх боломжтой.

4.4.4 Сарын тайлангийн дэлгэц

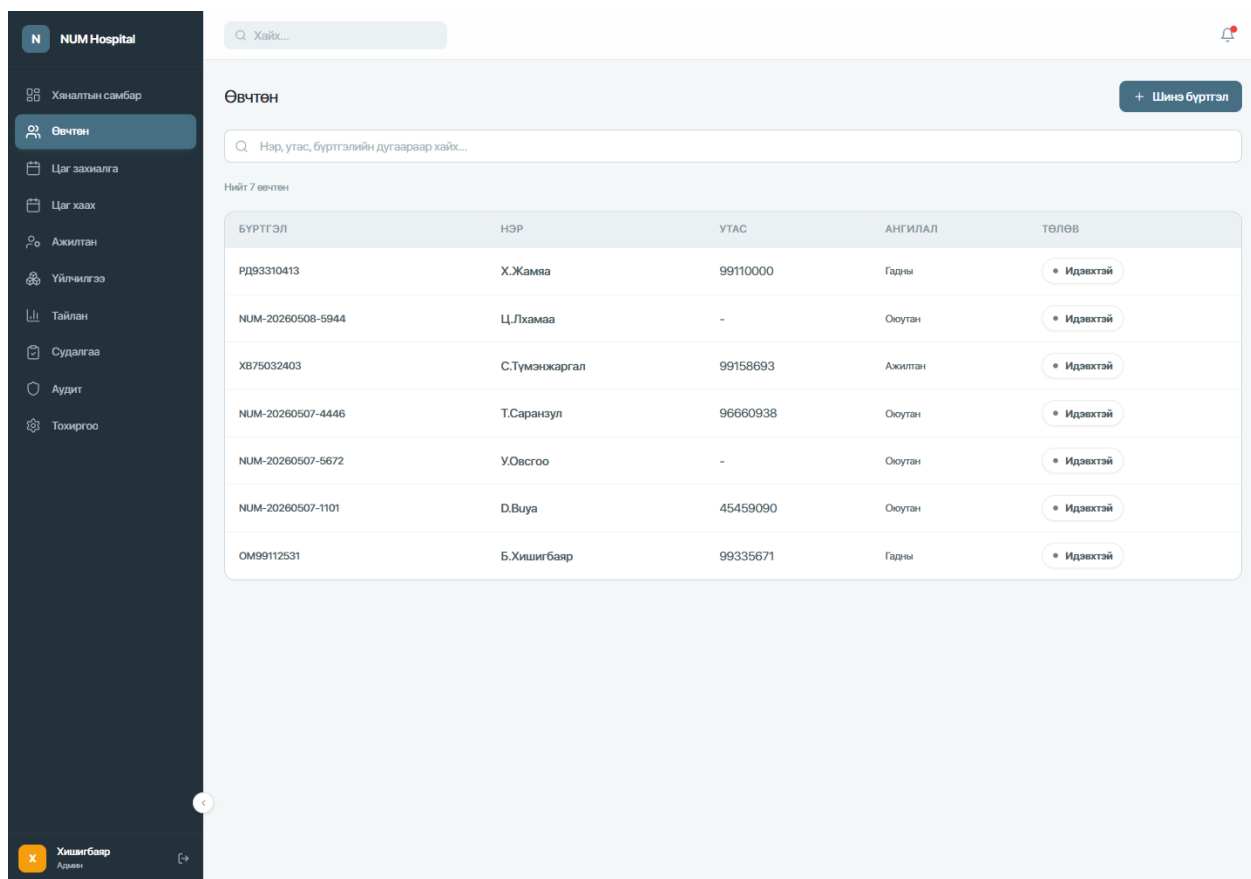
MonthlyReportPage нь monthlyReport(month) query ашиглан тухайн сарын тайланг харуулна. UI дээр нийт дууссан appointment, нийт дууссан visit, насны бүлэг, хүйс, service, resource, эмчээр ангилсан нэгтгэлүүдийг харуулах боломжтой. Энэ нь эмнэлгийн сарын үйл ажиллагааг тоон байдлаар дүгнэхэд ашиглагдана.

4.4.5 Admin frontend-ийн бодит дэлгэцүүд

Admin portal-ийн хэрэгжсэн дэлгэцүүдийг зураг 4.4–4.8-д харуулав. Эдгээр нь системийн админ өвчтөн, цаг товлолт, үйлчилгээ/нөөц болон тайлангийн мэдээллийг нэг удирдлагын орчноос харах боломжтойг харуулна.



Зураг 4.4: Admin portal — хяналтын самбар



NUM Hospital

Хайх...

Өвчтөн

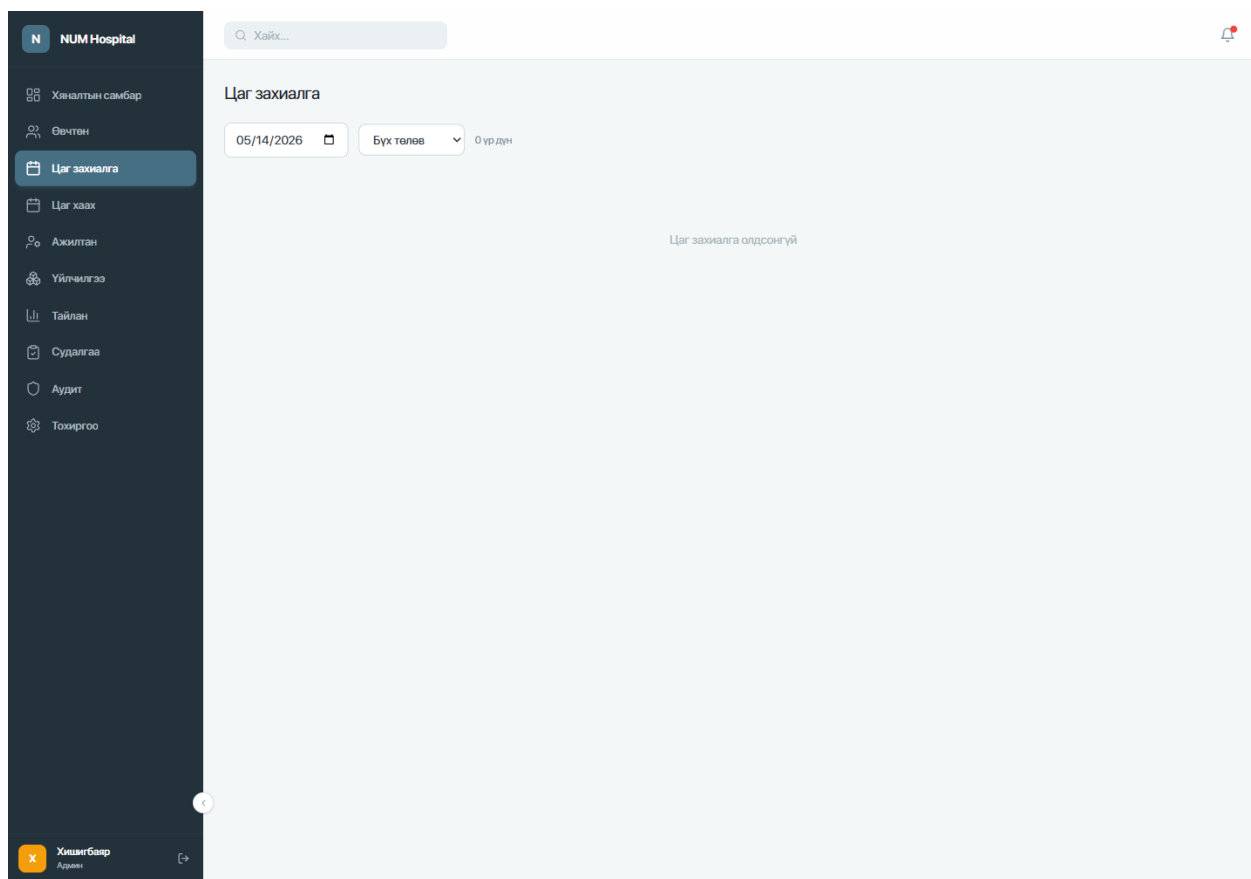
Нэр, утас, бүртгэлийн дугаараар хайх...

Нийт 7 өвчтөн

БҮРТГЭЛ	НЭР	УТАС	АНГИЛАЛ	ТӨЛӨВ
РД93310413	Х.Жамьяа	99110000	Гарны	• Идэвхтэй
NUM-20260508-5944	Ц.Лхамаа	-	Оюутан	• Идэвхтэй
ХВ75032403	С.Түмэнжаргал	99158693	Ажилтан	• Идэвхтэй
NUM-20260507-4446	Т.Саранзул	96660938	Оюутан	• Идэвхтэй
NUM-20260507-5672	У.Овсгоо	-	Оюутан	• Идэвхтэй
NUM-20260507-1101	Д.Буяа	45459090	Оюутан	• Идэвхтэй
ОМ99112531	Б.Хишигбаяр	99335671	Гарны	• Идэвхтэй

Хишигбаяр
Админ

Зураг 4.5: Admin portal — өвчтөний жагсаалт, хайлт



Зураг 4.6: Admin portal — цаг товлолтын удирдлага

Үйлчилгээ
Scheduling-д ашиглагдах үйлчилгээ, өрөө, төхөөрөмжийг энд удирдана.

Default үйлчилгээ оруулах + Шинэ үйлчилгээ

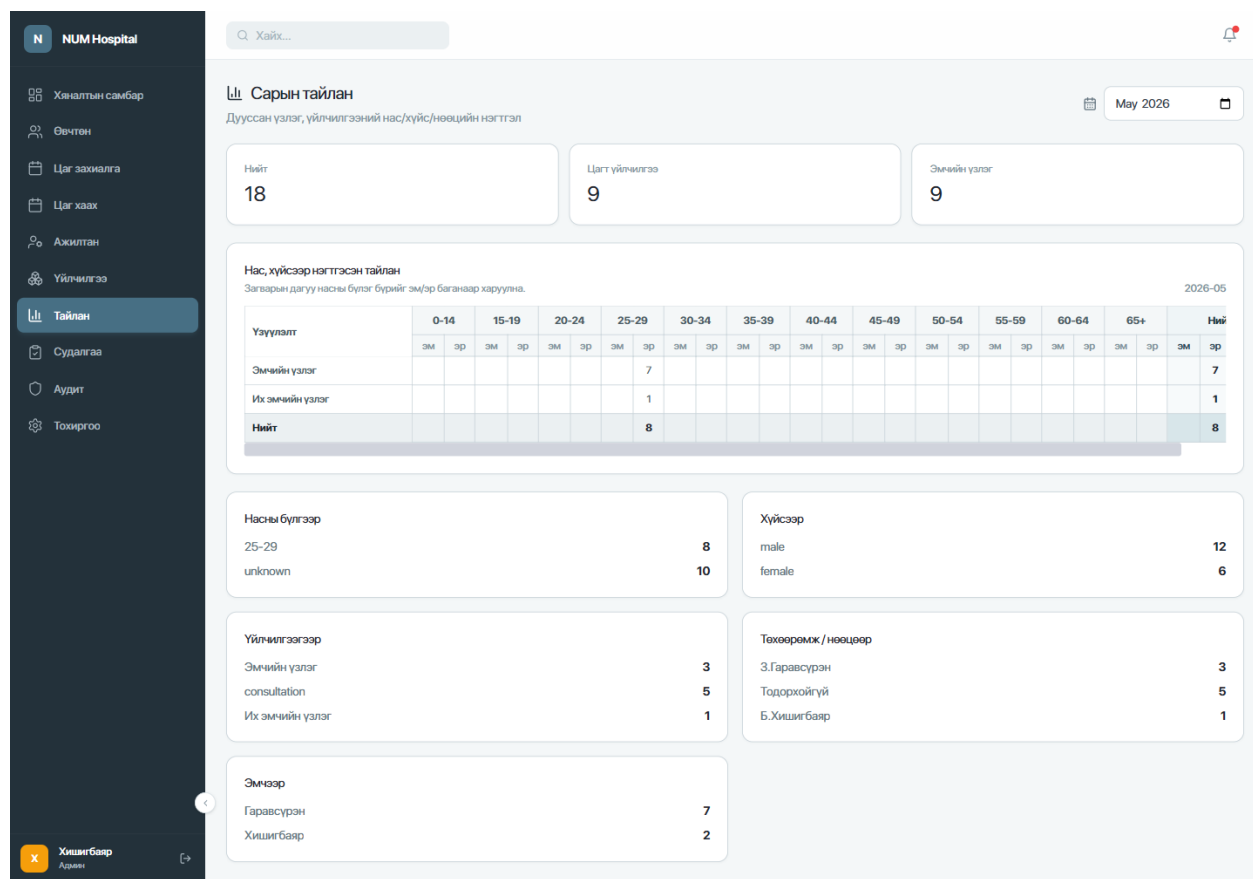
Үйлчилгээ Өрөө & төхөөрөмж

Бүх ангилал 4 үйлчилгээ

НЭР	CODE	АНГИЛАЛ	ХАМААРАХ	ОНООСОН	ХУГАЦАА	ТӨЛӨВ
Их эмчийн үзлэг Тайлбаргүй	72	Үзлэг	Эмч	З.Гаравсүрэн Б.Хишигбаяр	20 мин / завсар 0 мин	Идэвхтэй Засах
Шарлага хийлгэх Тайлбаргүй	2	Үзлэг	Төхөөрөмж	О.Цэндмаа	20 мин / завсар 10 мин	Идэвхтэй Засах
Эмч дээр үзүүлэх Ерөнхий эвээн шатны үзлэг хийх ба дараагийн шатны зөвлөмж өгнө	12	Үзлэг	Эмч	О.Цэндмаа	20 мин / завсар 0 мин	Идэвхтэй Засах
Эмчийн үзлэг Анхан шатны үзлэг оношлогоо хийх, зөвлөгөө, эмчилгээ бичнэ.	3	Үзлэг	Эмч	З.Гаравсүрэн	20 мин / завсар 0 мин	Идэвхтэй Засах

Хишигбаяр Ажлын

Зураг 4.7: Admin portal — үйлчилгээ болон нөөцийн удирдлага



Зураг 4.8: Admin portal — сарын тайлангийн дэлгэц

4.5 Apollo Client ба frontend shared тохиргоо

Хоёр frontend application хоёулаа Apollo Client ашиглаж backend GraphQL API руу ханддаг. `src/graphql/client.ts` файлд HTTP link дээр `credentials: "include"` тохируулж, `HttpOnly` session cookie-г browser-оор автоматаар дамжуулна. Ингэснээр token JavaScript runtime-д шууд уншигдахгүй, XSS халдлагаар token алдагдах эрсдэл буурна.

```

1 const httpLink = createHttpLink({
2   uri: import.meta.env.VITE_GRAPHQL_URL || "http://localhost:4000/
   graphql",
3   credentials: "include",
4 });

```



```
5  
6 export const client = new ApolloClient({  
7   link: httpLink,  
8   cache: new InMemoryCache(),  
9 });
```

Код 4.8: Apollo Client cookie session-ийн тохиргоо

Shared component хэсэгт loading spinner, empty state, status badge, toast provider, layout component-ууд багтсан. Ингэснээр patient болон admin талд UI-ийн давтагдсан хэсгийг нэг pattern-ээр хэрэгжүүлэх боломж бүрдсэн.

4.6 Өгөгдлийн сангийн нөөцлөлт ба сэргээх зохион байгуулалт

Эмнэлгийн мэдээллийн системд өвчтөн, цаг товлот, үзлэг, онош, жор, үйлчилгээ, нөөц, ажилтан болон тайлангийн өгөгдөл хадгалагддаг. Ийм төрлийн мэдээлэл нь өдөр тутмын үйл ажиллагааны тасралтгүй байдалд шууд нөлөөлөх тул өгөгдлийн сангийн нөөцлөлт, сэргээх боломжийг production орчны найдвартай ажиллагааны нэг хэсэг болгон төлөвлөсөн. Нөөцлөлтийн үндсэн зорилго нь серверийн эвдрэл, өгөгдөл санамсаргүй устгах, deployment-ийн алдаа гарах, эсвэл өгөгдлийн сангийн гэмтэл үүсэх үед сүүлийн хадгалсан төлөв рүү сэргээх боломж бүрдүүлэх явдал юм.

Нөөцлөлтийн шийдлийг боловсруулахдаа эмнэлгийн өгөгдлийн нууцлалын эрсдэлийг харгалзан үзсэн. Иймээс backup файлыг AWS S3 рүү илгээхээс өмнө сервер талд шифрлэж, S3 дээр зөвхөн шифрлэгдсэн файл хадгалагдах байдлаар зохион байгуулсан. Өөрөөр хэлбэл S3 нь зөвхөн backup object хадгалах үүрэгтэй бөгөөд хэрэглэгчийн өдөр тутмын файл хадгалалт түүн дээр байрлахгүй. Мөн өгөгдлийг тайлах private key-г AWS орчинд хадгалахгүй. Backup үүсгэдэг сервер дээр зөвхөн encrypt хийх public key байрлах бөгөөд restore хийхэд шаардлагатай private key-г байгууллагын дотоод, тусгаарлагдсан аюулгүй орчинд хадгална.

4.6.1 Backup workflow-ийн бүтэц

Нөөцлөлтийн процессийг серверийн хэрэглээ харьцангуй бага байх үе буюу шөнийн 03:00 цагт автоматаар ажиллуулахаар хэрэгжүүлсэн. Энэ нь `backend/src/cron/scheduler.ts`-д байгаа өдөр тутмын тайлан, цаг сануулах job-уудаас тусдаа, production deployment дээр OS-level cron эсвэл тусгай scheduler-ээр ажиллах зохион байгуулалт юм. Процесс эхлэхэд MongoDB өгөгдлийн сангаас `mongodump` ашиглан `archive + gzip` хэлбэрийн dump файл үүсгэнэ. Үүний дараа dump файлыг `age` public key encryption ашиглан шифрлэж, зөвхөн `.age` өргөтгөлтэй шифрлэгдсэн файлыг S3 bucket-ийн `backups/mongodb/ prefix` рүү өдөр бүр байршуулж байна. Upload амжилттай болсон тохиолдолд сервер дээр түр үүссэн plain dump болон encrypted temporary файлуудыг устгана.

Хүснэгт 4.5: MongoDB backup workflow-ийн алхамууд

№	Алхам	Тайлбар
1	Schedule trigger	Үйлдлийн системийн cron өдөр бүр 03:00 цагт backup script-ийг автоматаар ажиллуулж байна.
2	Database dump	<code>mongodump --archive --gzip</code> ашиглан MongoDB өгөгдлийн сангийн compressed archive үүсгэнэ.
3	Client-side encryption	Archive файлыг backup сервер дээр public key ашиглан шифрлэнэ. Private key backup сервер болон AWS дээр хадгалагдахгүй.
4	S3 upload	Зөвхөн шифрлэгдсэн <code>.archive.gz.age</code> файлыг <code>s3://bucket/backups/mongodb/</code> зам руу өдөр бүр upload хийнэ.
5	Local cleanup	Plain dump болон түр файлуудыг сервер дээрээс автоматаар устгана.
6	Retention policy	S3 lifecycle rule нь 7 хоногоос хуучин backup object-уудыг автоматаар устгана.

Backup файлын нэршилд өвчтөн, эмнэлэг, онош, албан тушаал зэрэг мэдрэмтгий утга агуулахгүй байхаар generic хэлбэр ашиглана. Жишээ нь `mongodb-2026-05-08-03-00-00.archive.gz.age` гэсэн нэршил нь зөвхөн backup-ийн төрөл болон үүссэн хугацааг илэрхийлнэ. Энэ нь object metadata алдагдсан тохиолдолд backup-ийн агуулгыг нэршлээс таамаглах эрсдэлийг бууруулна.

4.6.2 Түлхүүрийн хадгалалт ба нууцлал

Системийн backup архитектурт symmetric password-оос илүү asymmetric public/private key загварыг ашиглахаар сонгосон. Backup сервер дээр зөвхөн public key байрлах тул тухайн сервер backup файлыг encrypt хийх боломжтой боловч буцаан decrypt хийх боломжгүй. Харин decrypt хийх private key-г AWS, S3, repository, .env файл, эсвэл application server дээр хадгалахгүй. Энэ шийдэл нь cloud storage provider талын хандалт алдагдсан, S3 bucket буруу тохируулагдсан, эсвэл backup файл гаднын этгээдэд очсон үед өгөгдлийг шууд унших боломжийг хаана.

Хүснэгт 4.6: Backup түлхүүрийн хадгалалтын зарчим

Бүрэлдэхүүн	Хаана байрлах	Үүрэг ба хязгаарлалт
Public key	Backup серверийн хамгаалагдсан тохиргоо	Backup файлыг encrypt хийхэд ашиглана. Public key алдагдсан ч backup-ийг decrypt хийх боломжгүй.
Private key	Байгууллагын дотоод offline эсвэл тусгаарлагдсан secure орчин	Зөвхөн restore хийх үед ашиглана. AWS, S3, Git, production сервер дээр хадгалахгүй.
Encrypted backup file	AWS S3 backups/mongodb/prefix	Cloud дээр хадгалагдах object. Private key байхгүй үед агуулгыг унших боломжгүй.
S3 lifecycle policy	AWS S3 bucket тохиргоо	Backup object-уудыг 7 хоног хадгалаад автоматаар устгана.

Ингэснээр AWS S3 нь зөвхөн шифрлэгдсэн object хадгалах storage үүрэгтэй болж, өгөгдлийг тайлах эрх байгууллагын хяналтад үлдэнэ. Мөн S3 upload хийх үед --sse AES256

тохиргоог ашиглаж server-side encryption-ийг defense-in-depth хэлбэрээр нэмэлт хамгаалалт болгон хэрэглэж болно. Гэхдээ үндсэн хамгаалалт нь S3 рүү илгээхээс өмнө хийгдэх client-side public key encryption юм.

4.6.3 Автомат устгал ба сэргээх ажиллагаа

Backup хадгалалтын хугацааг 7 хоногоор тогтоосон. Энэ нь сүүлийн долоо хоногийн аль нэг backup төлөв рүү сэргээх боломжийг бүрдүүлэхийн зэрэгцээ cloud хадгалалтын зардал, илүүдэл өгөгдлийн хуримтлалыг хянах давуу талтай. S3 lifecycle rule нь backups/mongodb/ prefix-д хамаарах object-уудад үйлчилж, үүссэнээс хойш 7 хоног өнгөрсөн backup файлуудыг автоматаар устгана.

Restore ажиллагаа нь автомат backup-ээс ялгаатайгаар зөвхөн эрх бүхий ажилтан гар аргаар баталгаажуулсны дараа хийгдэнэ. Сэргээх үед тухайн өдөр, цагийн encrypted backup файлыг S3-ээс татаж, байгууллагын secure орчинд хадгалж буй private key ашиглан decrypt хийж, mongorestore --gzip --archive командаар MongoDB өгөгдлийн сан руу сэргээнэ. Сэргээх ажиллагааг production өгөгдөлд шууд нөлөөлүүлэхээс өмнө staging орчинд шалгах зарчмаар баталгаажуулсан.

```
1 0 3 * * * cd /path/to/NUM-Hospital-service-main/backend && \  
2 /bin/bash ./scripts/backup-mongodb-s3.sh >> \  
3 /var/log/num-hospital-backup.log 2>&1
```

Код 4.9: Өдөр бүр 03:00 цагт backup ажиллуулах cron тохиргооны жишээ

Энэхүү зохион байгуулалт хэрэгжсэнээр backup нь хэрэглэгчийн оролцоогүйгээр тогтмол үүсэх боловч restore хийх эрх болон private key-ийн хяналт байгууллагын дотоод аюулгүй орчинд үлдэхээр шийдэгдсэн. Энэ нь эмнэлгийн өгөгдлийн нууцлал, cloud storage-ийн ашиглалт, disaster recovery-ийн хэрэгцээг тэнцвэржүүлсэн шийдэл юм.

4.7 Хэрэглэгчийн сэтгэл ханамжийн судалгаа

Эмнэлгийн системд үйлчлүүлэгчийн сэтгэл ханамжийн судалгааг patient portal болон admin portal-той уялдуулан хэрэгжүүлсэн. Судалгааны зорилго нь эмнэлгийн үйлчилгээний чанар, ажилтны харилцаа, орчин нөхцөл, үйлчилгээний хүртээмжийн талаар хэрэглэгчээс үнэлгээ авч, цаашдын сайжруулалтад ашиглах явдал юм.

Судалгааны үндсэн загвар нь admin талаас удирдагдах динамик бүтэцтэй. Өөрөөр хэлбэл, асуултууд frontend код дотор тогтмол бичигдээгүй бөгөөд backend-ийн өгөгдлийн санд хадгалагдсан template-ээс patient талд татагдан харуулагдана. Ингэснээр асуулт нэмэх, засах, бүлгээр ангилах, идэвхгүй болгох зэрэг өөрчлөлтийг код өөрчлөхгүйгээр admin portal-оос хийх боломжтой болсон.

4.7.1 Admin талын удирдлага ба version history

Superadmin эрхтэй хэрэглэгч admin portal-ийн “Судалгаа” хэсэгт нэвтэрч survey template-ийг удирдана. Энэ хэсэгт судалгааны гарчиг, тайлбар засах, үнэлгээний асуулт нэмэх, асуултын текст болон category өөрчлөх, асуултыг идэвхтэй/идэвхгүй болгох, судалгааны хувилбарын түүх харах, template устгах эсвэл архивлах боломжийг хэрэгжүүлсэн.

Судалгааны асуултыг хадгалах бүрт хуучин хувилбар алдагдахгүй. Систем нь template-ийн шинэ version үүсгэж, өмнөх version-ийн ашиглагдсан хугацааг validFrom, validTo огноогоор хадгална. Ингэснээр тухайн асуулгын аль хувилбар ямар хугацаанд хэрэглэгдэж байсан нь бүртгэгдэнэ.

```
1 Survey Template
2   currentVersion: 2
3   active: true
4   versions:
5     - version: 1
6       validFrom: 2026-05-14
```

```

7     validTo: 2026-05-20
8     questions: [...]
9   - version: 2
10    validFrom: 2026-05-20
11    validTo: null
12    questions: [...]

```

Код 4.10: Survey template version history-ийн жишээ

4.7.2 Patient малын бөглөлт

Patient portal дээр хэрэглэгч “Судалгаа” цэсээр орж судалгааг сайн дурын үндсэн дээр бөглөх боломжтой. Судалгаанд ерөнхий мэдээлэл болон 1–5 онооны үнэлгээний асуултууд орно. Үнэлгээний онооны утга дараах байдлаар тайлбарлагдана.

Хүснэгт 4.7: Сэтгэл ханамжийн үнэлгээний онооны тайлбар

Оноо	Тайлбар
1	Огт хангалтгүй
2	Хангалтгүй
3	Дундаж
4	Хангалттай
5	Хангалттай сайн

Patient судалгааг бөглөх үед тухайн үед идэвхтэй байгаа survey template version ашиглагдана. Хариулт хадгалах үед зөвхөн оноо хадгалахаас гадна тухайн асуултын label, category, score мэдээллийг response дотор хамт хадгална. Энэ snapshot зарчим нь дараа нь асуултын текст өөрчлөгдсөн ч өмнө бөглөсөн судалгааны агуулга алдагдахгүй байх давуу талтай.

4.7.3 Заавал бөглүүлэх бизнес логик

Системд судалгааг заавал бөглүүлэх нөхцөл хэрэгжүүлсэн. Patient 3 удаагийн completed үйлчилгээ авсны дараа тухайн үеийн идэвхтэй survey version-ийг бөглөөгүй бол систем тухайн хэрэглэгчийг survey бөглөх шаардлагатай төлөвт оруулна.

```
1 if (completedServiceCount >= 3 &&
2     currentSurveyVersion not submitted) {
3     surveyRequired = true;
4 }
```

Код 4.11: Survey required шалгах бизнес дүрэм

Энэ үед patient portal-ийн protected page рүү ороход хэрэглэгч автоматаар /survey?next=... хуудас руу шилжинэ. Судалгаа бөглөсний дараа систем хэрэглэгчийг өмнө орох гэж байсан хуудас руу буцаана. Харин /survey хуудас өөрөө үргэлж нээлттэй тул хэрэглэгч сайн дурын үндсэн дээр ч бөглөх боломжтой.

4.7.4 Устгах ба архивлах логик

Survey template-ийг устгах үед өгөгдлийн бүрэн бүтэн байдлыг хамгаалах логик хэрэгжүүлсэн. Хэрэв тухайн template дээр нэг ч patient хариулт бөглөөгүй бол template-ийг шууд устгаж болно. Харин тухайн template дээр дор хаяж нэг хариулт бүртгэгдсэн бол систем template-ийг устгахгүй, харин архивын төлөвт шилжүүлнэ.

```
1 if (responseCount == 0) {
2     deleteTemplate();
3 } else {
4     archiveTemplate();
5 }
```

Код 4.12: Template delete/archive дүрэм

Архивлах үед `active = false` болж, `archivedAt`, `archivedBy` мэдээлэл хадгалагдана. Мөн тухайн үеийн `active version`-ийн `validTo` бөглөгдөж, өмнөх `patient response`-ууд хадгалагдсан хэвээр үлдэнэ. Ингэснээр бөглөгдсөн судалгааны дата устахгүй бөгөөд судалгааны түүх, тайлангийн үнэн зөв байдал хадгалагдана.

4.7.5 GraphQL API ба хадгалалт

Backend талд `getActiveSatisfactionSurveyTemplate`, `getMySatisfactionSurvey`, `getMySatisfactionSurveyRequirement`, `listSatisfactionSurveys`, `submitSatisfactionSurvey`, `updateSatisfactionSurveyTemplate`, `removeSatisfactionSurveyTemplate` query болон mutation-уудыг хэрэгжүүлсэн. Ялангуяа `getMySatisfactionSurveyRequirement` query нь тухайн `patient 3 completed` үйлчилгээ авсан эсэх, одоогийн `survey version`-ийг бөглөсөн эсэхийг шалгаж, `survey` заавал бөглөх шаардлагатай эсэхийг frontend-д буцаана.

Хүснэгт 4.8: Сэтгэл ханамжийн судалгааны хадгалах өгөгдөл

Collection	Гол талбарууд
satisfaction_survey_template	description, active, currentVersion, questions, versions, updatedBy, archivedAt, archivedBy
satisfaction_surveys	templateId, templateVersion, userId, patientId, occupation, studentHousing, hasVisited, visitFrequency, servicesReceived, wouldReturn, wouldReturnReason, improvementSuggestion, ratings, overallRating, createdAt

Энэхүү шийдлээр судалгааны асуултуудыг код өөрчлөхгүйгээр admin талаас удирдах, patient сайн дурын болон шаардлагатай нөхцөлд бөглөх, 3 completed үйлчилгээний дараа судалгааг заавал бөглүүлэх, version history болон response snapshot хадгалах, бөглөгдсөн судалгаатай template-ийг archive төлөвт шилжүүлэх боломж бүрдсэн.

4.8 Хэрэгжүүлэлтийн хамрах хүрээ

Эх код дээр тулгуурлан хэрэгжүүлэлтийн хамрах хүрээг хүснэгт 4.9 байдлаар нэгтгэв. Энэ хүснэгт нь хамгаалалтын үед аль хэсэг бүрэн хэрэгжсэн, аль хэсэг дараагийн шатанд өргөтгөх шаардлагатайг тодорхой ялгах зорилготой.

Хүснэгт 4.9: Модулиудын хэрэгжүүлэлтийн хамрах хүрээ

Модуль	Төлөв	Тайлбар
Authentication	Хэрэгжсэн	JWT, bcrypt, email OTP, OAuth callback route, RBAC helper
Patient profile	Хэрэгжсэн	Profile бөглөх, update хийх, өөрийн profile авах, profile completion check
Patient booking	Хэрэгжсэн	Service/resource/device сонголт, available slot, create/cancel appointment
Service/resource scheduling	Хэрэгжсэн	Capacity room, device buffer, duration, slot interval, default scheduling seed
Unavailable block	Хэрэгжсэн	Staff/resource-ийн боломжгүй хугацаа, owner/admin эрхийн шалгалт
Admin patient management	Хэрэгжсэн	Өвчтөн хайх, бүртгэх, дэлгэрэнгүй, visit/prescription history
Doctor queue	Хэрэгжсэн	Тухайн өдрийн дараалал, appointment эхлүүлэх, visit үүсгэх
Doctor examination	Хэрэгжсэн	Visit update, vital signs, ICD diagnosis, prescription, complete visit
ICD-11 integration	Хэрэгжсэн	Backend proxy, root/children/search query, frontend ICD picker
Monthly report	Хэрэгжсэн	Completed appointment/visit дээр үндэслэн нас, хүйс, service, resource, doctor-оор нэгтгэх

Модуль	Төлөв	Тайлбар
Backup/restore	Хэрэгжсэн	MongoDB dump, public key encryption, өдөр тутмын S3 upload, 7 хоногийн lifecycle retention, restore ажиллагааны баталгаажуулалт
Notification	Суурь хэрэгжсэн	Model, read mutation, GraphQL subscription суурь; reminder logic дараагийн шат
Audit log	Суурь хэрэгжсэн	Login, create/update төрлийн чухал үйлдэл бүртгэх; audit UI placeholder
Satisfaction survey	Хэрэгжсэн	Admin-аас удирдах dynamic template, version history, patient response snapshot, 3 completed үйлчилгээний дараах required redirect, archive/delete logic
Inventory, nurse queue	Дараагийн шат	Admin frontend дээр placeholder route байдлаар үлдсэн
Cron jobs	Хэрэгжсэн	<code>scheduler.ts</code> -д daily report, appointment reminder job бүртгэгдсэн; backup нь OS-level cron-оор тусдаа ажиллана

4.9 Системийг ажиллуулах ба баталгаажуулах командууд

Backend, admin frontend, patient frontend тус бүрийн build болон ажиллуулах script-үүдийг `package.json` файлд тусгасан. Local development үед дараах командаар ажиллуулна.

```
1 cd backend
```

```
2 corepack yarn install
3 corepack yarn dev
```

Код 4.13: Backend ажиллуулах команд

```
1 cd patient-frontend
2 npm install
3 npm run dev
4
5 cd admin-frontend
6 npm install
7 npm run dev
```

Код 4.14: Frontend ажиллуулах командууд

Build шалгалтыг дараах байдлаар гүйцэтгэнэ.

```
1 cd backend
2 corepack yarn build
3
4 cd ../patient-frontend
5 npm run build
6
7 cd ../admin-frontend
8 npm run build
```

Код 4.15: Build шалгах командууд

Backend health route нь сервер ажиллаж байгаа эсэхийг шалгахад ашиглагдана.

```
1 GET http://localhost:4000/health
2 GET http://localhost:4000/
```

Код 4.16: Health endpoint шалгах жишээ

4.10 Функциональ тестийн хувилбарууд

Эх кодын хэрэгжүүлэлтийн дагуу дараах functional test case-үүдийг гараар шалгаж, гол workflow-ууд хүлээгдсэн үр дүнтэй ажиллаж байгааг баталгаажуулсан.

Хүснэгт 4.10: Функциональ тестийн баталгаажуулсан хувилбарууд

ID	Тестийн хувилбар	Хүлээгдэж буй үр дүн
TC-01	Staff утас/нууц үгээр нэвтрэх	HttpOnly session cookie үүсэж admin dashboard руу орно
TC-02	Patient email OTP авах	МУИС-ийн зөвшөөрөгдсөн домэйнд OTP илгээх хүсэлт амжилттай болно
TC-03	Patient OTP-оор нэвтрэх	HttpOnly session cookie үүсэж patient portal нээгдэнэ
TC-04	Patient profile бөглөх	Profile хадгалагдаж profileCompletedAt үүснэ
TC-05	Service сонгож slot харах	available, booked, blocked, past төлөвтэй slot-ууд ирнэ
TC-06	Дуслын өрөөнд capacity шалгах	Нэг slot дээр capacity хүрэх хүртэл remaining буурч, capacity дүүрэхэд booked болно
TC-07	Төхөөрөмж дээр buffer шалгах	Шарлага/массажны дараах buffer хугацаанд дараагийн цаг давхцахгүй
TC-08	Appointment үүсгэх	scheduled төлөвтэй appointment үүсэж queueNumber оноогдоно

ID	Тестийн хувилбар	Хүлээгдэж буй үр дүн
TC-09	Давхардсан цаг үүсгэх	Capacity хүрсэн үед алдааны мессеж буцна
TC-10	Unavailable block үүсгэх	Тухайн хугацааны slot blocked төлөвтэй болно
TC-11	Эмч appointment эхлүүлэх	Appointment in_progress болж, visit draft үүснэ
TC-12	Амин үзүүлэлт бүртгэх	VitalSign record тухайн visit-тэй холбогдоно
TC-13	ICD онош нэмэх	Diagnosis record ICD code/title/URI metadata-тай хадгалагдана
TC-14	Жор бичих	Prescription number үүсэж patient history-д харагдана
TC-15	Үзлэг дуусгах	Visit completed төлөвтэй болж completedAt үүснэ
TC-16	Сарын тайлан харах	Нас, хүйс, үйлчилгээ, resource, эмчээр нэгтгэсэн тайлан гарна
TC-17	Backup script-ийн ажиллагаа шалгах	OS-level cron, public key encryption, S3 upload, 7 хоногийн lifecycle тохиргоо баталгаажсан
TC-18	Идэвхтэй survey template татах	Patient portal дээр тухайн үеийн active template, currentVersion, асуултууд харагдана
TC-19	Сэтгэл ханамжийн судалгаа илгээх	Response нь templateId, templateVersion, ratings snapshot, overallRating, createdAt мэдээлэлтэй хадгалагдана

ID	Тестийн хувилбар	Хүлээгдэж буй үр дүн
TC-20	3 completed үйлчилгээний дараах шаардлага шалгах	Одоогийн survey version бөглөөгүй patient protected page рүү ороход /survey?next=... рүү шилжинэ
TC-21	Admin template update хийх	Асуулт өөрчлөхөд currentVersion нэмэгдэж, өмнөх version validFrom/validTo хугацаатай хадгалагдана
TC-22	Хариулттай template устгах	Response бүртгэгдсэн template шууд устахгүй, active=false болж archivedAt/archivedBy хадгалагдана

4.10.1 Автоматжуулсан unit, RBAC ба performance тест

Гараар шалгасан функциональ тестүүдээс гадна backend-ийн хамгаалалтын логик болон бодит deployment дээрх GraphQL endpoint-д автомат тест ажиллуулсан. Unit test нь source code түвшинд requireAuth, requireRole, requireOwnerOrRole, adminAccess болон JWT helper функцүүдийг шалгасан. RBAC abuse test нь anonymous болон patient эрхээр admin/doctor түвшний query, mutation ажиллуулах оролдлогыг хориглож байгаа эсэхийг баталгаажуулсан. Performance smoke test нь production-like live endpoint дээр бага ачааллаар response time хэмжсэн.

```

1 cd backend
2 yarn test:unit
3

```



```
4 # Live RBAC negative test
5 GRAPHQL_TEST_URL=http://202.131.1.77:3101/graphql \
6 ADMIN_PHONE=<admin_phone> ADMIN_PASSWORD=<admin_password> \
7 CREATE_TEST_PATIENT=true yarn test:rbac
8
9 # Live performance smoke test
10 GRAPHQL_TEST_URL=http://202.131.1.77:3101/graphql \
11 ADMIN_PHONE=<admin_phone> ADMIN_PASSWORD=<admin_password> \
12 PERF_REQUESTS=20 PERF_CONCURRENCY=4 yarn test:perf
```

Код 4.17: Автомат тест ажиллуулсан командууд

Хүснэгт 4.11: Автоматжуулсан тестийн нэгтгэл

Тестийн төрөл	Шалгасан зүйл	Үр дүн
Unit test	Authentication guard, role guard, owner-or-role guard, superadmin guard, JWT encode/decode helper	6/6 passed
RBAC abuse test	Anonymous болон patient role-oop searchPatients, monthlyReport, seedDefaultScheduling, createService, icd11Search зэрэг protected operation ажиллуулах оролдлого	8/8 passed
Performance smoke test	listDepartments, me, searchPatients, monthlyReport query-г тус бүр 20 request, concurrency 4 тохиргоогоор хэмжсэн	80/80 request амжилттай

Хүснэгт 4.12: GraphQL performance smoke test-ийн хэмжилт

Scenario	Амжилт	Avg ms	P95 ms	Max ms
listDepartments	20/20	156.6	177.3	181.2
me	20/20	158.7	179.4	182.6
searchPatients	20/20	281.2	307.5	325.9
monthlyReport	20/20	722.9	716.9	1622.5

Энэхүү хэмжилтээс харахад энгийн public болон authenticated identity query-үүд 180

ms орчим P95 response time-тэй, patient хайлт 307.5 ms P95 response time-тэй ажилласан. monthlyReport нь aggregation logic ашигладаг тул бусдаасаа удаан боловч 20 request-ийн smoke test-д алдаагүй ажилласан.

4.11 Хэрэгжүүлэлтийн хязгаарлалт ба цаашдын ажил

Энэ хувилбарт үндсэн patient booking, admin management, doctor examination workflow хэрэгжсэн боловч production түвшинд ашиглахын өмнө дараах зүйлсийг сайжруулах шаардлагатай.

- Email OTP store-г memory-оос Redis зэрэг төвлөрсөн cache рүү шилжүүлэх;
- OAuth provider-ийн бодит NUM/SISI credential болон callback URL-ийг production орчинд бүрэн баталгаажуулах;
- Автоматжуулсан unit болон RBAC negative test-ийн coverage-ийг resolver/service түвшинд өргөтгөж CI pipeline-д холбох;
- Audit log харах UI, nurse queue, inventory module-ийг дараагийн шатанд хэрэгжүүлэх;
- Cron scheduler дээр appointment reminder болон daily report job-ийн бодит service-г холбох;
- Performance smoke test-ээс цааш MongoDB explain plan, stress test, monitoring ашиглан bottleneck-ийг нарийвчлан тодорхойлох;
- Сэтгэл ханамжийн судалгааны үр дүнд тулгуурлан workflow-ийн нэршил, талбар, тайлангийн форматыг үргэлжлүүлэн сайжруулах.

4.12 Бүлгийн дүгнэлт

Энэ бүлэгт МУИС-ийн эмнэлгийн мэдээллийн системийн source code дээр тулгуурлан хэрэгжүүлэлтийн бүтцийг тайлбарлав. Backend талд Express, Apollo GraphQL,

MongoDB/Mongoose, service layer, JWT/RBAC, email OTP, OAuth callback, ICD-11 proxy, service/resource scheduling, monthly report, audit log, encrypted backup/restore болон сэтгэл ханамжийн судалгааны урсгал зэрэг гол хэсгүүд хэрэгжсэн. Frontend талд patient portal болон admin portal-ийг React, Vite, TypeScript, Apollo Client, Tailwind CSS ашиглан тусдаа application хэлбэрээр хөгжүүлсэн.

Системийн хамгийн чухал хэрэгжүүлэлтийн шийдэл нь цаг товлолтыг **үйлчилгээ + нөөц + хугацаа** гэсэн загвараар зохион байгуулсан явдал юм. Энэ шийдэл нь зөвхөн эмчийн үзлэгээс гадна дуслын өрөөний олон суудал, шарлагын төхөөрөмж, массажны сандал зэрэг эмнэлгийн бодит нөөцүүдийг нэг scheduling logic-оор удирдах боломжийг бүрдүүлсэн. Ингэснээр өмнөх бүлгүүдэд тодорхойлсон шаардлага, зохиомжийг бодит веб системийн MVP түвшинд хэрэгжүүлсэн гэж дүгнэж болно.

Дүгнэлт

Энэхүү бакалаврын судалгааны ажлын хүрээнд МУИС-ийн эмнэлгийн өдөр тутмын үндсэн урсгалд нийцсэн мэдээллийн системийн шинжилгээ, зохиомж, хэрэгжүүлэлтийн ажлыг гүйцэтгэв. Ажлын гол үр дүнгүүдийг нэгтгэн дурдвал:

1. Судалгааны орон зайг тодорхойлсон. 2019–2025 оны хооронд хийгдсэн 4 бакалаврын судалгааны ажлыг хамрах хүрээ, арга зүй, хэрэгжүүлэлтийн түвшин, модуль хоорондын уялдаагаар харьцуулж, өмнөх ажлууд тус тусдаа модулийн түвшинд төвлөрсөн боловч өвчтөний бүртгэлээс үзлэг, оношлогоо, жор хүртэлх нэгдсэн өгөгдлийн урсгал бүрэн шийдэгдээгүй байсныг судалгааны орон зай болгон тодорхойлов.

2. Нэгдсэн системийн шинжилгээ ба зохиомж боловсруулсан. Өвчтөн, цаг товлолт, үзлэг, оношлогоо, жор, хэрэглэгчийн эрх, мэдэгдэл, аудит, үйлчилгээ ба нөөцийн бүртгэл болон сэтгэл ханамжийн судалгааг хамарсан нийт 17 collection-ийн өгөгдлийн зохиомжийг гаргав. Хэрэглэгчийн урсгалд шууд оролцох гол collection-уудыг ER диаграммаар, үйлчилгээ ба нөөцийн хуваарилалт болон survey template/response collection-уудыг туслах зохиомжийн хэсэгт тодорхойлов.

3. Гол функциональ модулиудыг хэрэгжүүлсэн. Нэвтрэлт, өвчтөний бүртгэл, цаг товлолт, үзлэг, оношлогоо, жорын үндсэн урсгалыг хэрэгжүүлж, GraphQL query/mutation-уудаар дамжуулан frontend болон backend-ийн өгөгдлийн гэрээг тодорхойлов. Тайлан, аудит, мэдэгдэл, үйлчилгээ ба нөөцийн хуваарилалт, encrypted backup/restore болон template/version бүхий хэрэглэгчийн сэтгэл ханамжийн судалгааны урсгалыг тусгасан.

4. Аюулгүй байдал ба эрхийн хяналтын суурь шийдэл боловсруулсан. 4 үндсэн дүрийн RBAC (patient, doctor, nurse, superadmin), HttpOnly cookie-д суурилсан JWT authentication, bcrypt password hashing, СИСИ OAuth 2.0 нэгтгэлийн callback урсгалын бэлт-

гэл, audit log-ийн өгөгдлийн загвар болон encrypted backup/restore шийдлийг тодорхойлсон. Эмнэлгийн өгөгдөл нь эмзэг мэдээлэл тул production орчинд CSRF хамгаалалт, refresh token rotation, PKCE, session invalidation, data retention зэрэг нэмэлт хамгаалалтыг цаашид нарийвчлах шаардлагатай.

5. Хэрэгжүүлэлтийн баталгаажуулалт хийсэн. Гол хэрэглэгчийн урсгалуудыг API болон функциональ тестийн хувилбараар гараар шалгав. Backup ажиллагаа болон сэтгэл ханамжийн судалгааны active template татах, response хадгалах, 3 completed үйлчилгээний дараах required redirect, version history, archive/delete logic-ийг хэрэгжүүлэлтийн баталгаажуулалтад нэмж тусгасан. Мөн шаардлага, use case, хэрэгжүүлэлт, тестийн хоорондын уялдааг traceability хүснэгтээр нэгтгэснээр шаардлагын хамрах хүрээ болон хэрэгжүүлэлтийн төлөв илүү тодорхой болов.

Практик ач холбогдол. Системийн зохиомж нь МУИС-ийн эмнэлгийн өвчтөний бүртгэл, цаг товлолт, эмчийн үзлэг, оношлогоо, жорын мэдээллийг нэг өгөгдлийн урсгалд холбож, цаасан бүртгэлээс үүдэх хайлт, давхардал, тайлан гаргалтын хүндрэл, хандалтын хяналтын сул талыг бууруулах суурь шийдэл болж байна.

Ирээдүйн чиглэл. Системийг цаашид дараах чиглэлээр хөгжүүлж болно:

- Нэгж тест, интеграцийн тест, RBAC negative test, performance test-ийг автоматжуулах
- Тайлан, статистик, аудит, мэдэгдлийн модулийг production түвшинд бүрэн хэрэгжүүлэх
- Лабораторийн шинжилгээ болон эмийн сангийн нөөцийн модулийг нэмэх
- OAuth 2.0 болон cookie session урсгалд PKCE, CSRF хамгаалалт, refresh token rotation, session invalidation зэрэг хамгаалалтыг нэмж нарийвчлах
- HL7 FHIR зэрэг эрүүл мэндийн өгөгдөл солилцооны стандарттай нийцүүлэх боломжийг судлах
- Docker/Kubernetes deployment, monitoring болон log/alerting орчныг бүрдүүлэх

- Сэтгэл ханамжийн судалгааны үр дүнд тулгуурлан workflow болон интерфейсийн сайжруулалтыг давтан хийх

Номзүй

- [1] С.Оюунбилэг, “МУИС-ийн эмнэлгийн мэдээллийн системийн шинжилгээ”, МУИС ХШУИС МКУТ, Бакалаврын судалгааны ажил, 2019.
- [2] Н.Наранчимэг, “МУИС-ийн эмнэлэг – Үзлэгийн программ”, МУИС ХШУИС МКУТ, Бакалаврын судалгааны ажил, 2023.
- [3] Э.Мягмарсүрэн, “МУИС-ийн эмнэлэг – Эмчилгээний програм”, МУИС ХШУИС МКУТ, Бакалаврын судалгааны ажил, 2023.
- [4] Ө.Агиймаа, “Цэвэр архитектур ба түүний хэрэгжүүлэлт”, МУИС МТЭС МКУТ, Бакалаврын судалгааны ажил, 2025.
- [5] GraphQL Foundation, “GraphQL Specification”, <https://spec.graphql.org/>, 2024.
- [6] Apollo GraphQL, “Apollo Server 4 Documentation”, <https://www.apollographql.com/docs/apollo-server/>, 2024.
- [7] Meta, “React Documentation”, <https://react.dev/>, 2024.
- [8] MongoDB Inc., “MongoDB Manual”, <https://www.mongodb.com/docs/manual/>, 2024.
- [9] Microsoft, “TypeScript Documentation”, <https://www.typescriptlang.org/docs/>, 2024.
- [10] Evan You, “Vite Documentation”, <https://vitejs.dev/>, 2024.
- [11] Tailwind Labs, “Tailwind CSS Documentation”, <https://tailwindcss.com/docs>, 2024.
- [12] Auth0, “JSON Web Tokens”, <https://jwt.io/introduction>, 2024.
- [13] D. Hardt, “The OAuth 2.0 Authorization Framework”, RFC 6749, IETF, 2012.
- [14] N. Sakimura, J. Bradley, N. Agarwal, “Proof Key for Code Exchange by OAuth Public Clients”, RFC 7636, IETF, 2015.
- [15] OWASP Foundation, “OWASP Application Security Verification Standard”, OWASP, 2024.
- [16] Automattic, “Mongoose ODM Documentation”, <https://mongoosejs.com/docs/>, 2024.
- [17] OpenMRS, “OpenMRS Documentation”, <https://openmrs.org/>, 2024.

- [18] GNU Solidario, “GNU Health Documentation”, <https://www.gnuhealth.org/>, 2024.
- [19] HospitalRun, “HospitalRun Documentation”, <https://hospitalrun.io/>, 2024.
- [20] R.C. Martin, “Clean Architecture: A Craftsman’s Guide to Software Structure and Design”, Prentice Hall, 2017.
- [21] I. Fette, A. Melnikov, “The WebSocket Protocol”, RFC 6455, IETF, 2011.
- [22] OpenJS Foundation, “Express.js Documentation”, <https://expressjs.com/>, 2024.
- [23] World Health Organization, “International Classification of Diseases 10th Revision”, <https://icd.who.int/browse10/2019/en>, 2019.
- [24] World Health Organization, “Health Management Information Systems (HMIS)”, Technical Report, 2020.
- [25] D. Ferraiolo, R. Kuhn, “Role-Based Access Control”, 15th NIST National Computer Security Conference, 1992.
- [26] МУИС, “СИСИ Мэдээллийн Систем”, <https://sisi.num.edu.mn/>, 2024.

A. GRAPHQL SCHEMA

Системийн GraphQL schema-ийн гол хэсгүүдийг энд оруулав.

A.1 Root Query ба Mutation

```
1 type Query {
2   # Auth
3   me: User
4
5   # Patients
6   searchPatients(query: String!, page: Int,
7     limit: Int): PatientList!
8   getPatient(_id: ID!): Patient
9   getMyPatientProfile: Patient
10
11  # Appointments
12  getAppointment(_id: ID!): Appointment
13  listAppointments(
14    filter: AppointmentFilterInput!
15  ): AppointmentList!
16  getMyAppointments(status: String,
17    page: Int, limit: Int): AppointmentList!
18  getDoctorQueue(doctorId: ID!,
19    date: Date!): [Appointment!]!
20  getAvailableSlots(doctorId: ID!,
21    date: Date!): [String!]!
22
```

```
23 # Visits
24 getVisit(_id: ID!): Visit
25 getMyVisitHistory(page: Int,
26   limit: Int): [Visit!]!
27
28 # Diagnoses & Prescriptions
29 getDiagnosesByVisit(
30   visitId: ID!): [Diagnosis!]!
31 getMyPrescriptions(page: Int,
32   limit: Int): [Prescription!]!
33 }
```

Код A.1: Root Query type (хэсэгчлэн)

```
1 type Mutation {
2   # Auth
3   loginUser(phone: String!,
4     password: String!): AuthPayload!
5   registerUser(
6     input: RegisterUserInput!): User!
7   initiateOAuth(provider: String!,
8     origin: String): OAuthInitPayload!
9
10  # Appointments
11  createAppointment(
12    input: CreateAppointmentInput!
13  ): Appointment!
14  checkInAppointment(_id: ID!): Appointment!
15  cancelAppointment(_id: ID!,
```

```
16     reason: String!): Appointment!
17 startAppointment(_id: ID!): Appointment!
18 completeAppointment(_id: ID!): Appointment!
19
20 # Visits
21 createVisit(
22     input: CreateVisitInput!): Visit!
23 completeVisit(_id: ID!): Visit!
24 recordVitalSigns(visitId: ID!,
25     input: VitalSignInput!): VitalSign!
26
27 # Diagnoses
28 createDiagnosis(
29     input: CreateDiagnosisInput!
30 ): Diagnosis!
31
32 # Prescriptions
33 createPrescription(
34     input: CreatePrescriptionInput!
35 ): Prescription!
36 }
```

Код A.2: Root Mutation type (хэсэгчлэн)

В. КОДЫН ХЭРЭГЖҮҮЛЭЛТ

В.1 Backend — Appointment Service

```
1 export const getAvailableSlots = async (  
2   _: any,  
3   { doctorId, date }:  
4     { doctorId: string; date: Date },  
5   ctx: ContextType  
6 ) => {  
7   if (!ctx.authenticated)  
8     throw new Error("Authentication required");  
9  
10  const staff = await Staff  
11    .findById(doctorId);  
12  if (!staff)  
13    throw new Error("Doctor not found");  
14  
15  const dayOfWeek = new Date(date).getDay();  
16  const schedule = staff.workSchedule?.find(  
17    (s: any) => s.dayOfWeek === dayOfWeek  
18  );  
19  if (!schedule) return [];  
20  
21  // Generate all possible slots  
22  const allSlots: string[] = [];  
23  let current = parseTime(schedule.startTime);
```

```
24  const end = parseTime(schedule.endTime);
25
26  while (current < end) {
27    allSlots.push(formatTime(current));
28    current += 30; // 30 min intervals
29  }
30
31  // Find booked slots
32  const booked = await Appointment.find({
33    doctorId,
34    scheduledDate: date,
35    status: {
36      $nin: ["cancelled", "no_show"]
37    },
38  }).select("scheduledTime");
39
40  const bookedTimes = booked
41    .map((a: any) => a.scheduledTime);
42
43  return allSlots
44    .filter(s => !bookedTimes.includes(s));
45  };
```

Код B.1: getAvailableSlots service

B.2 Frontend — useAuth Hook

```
1  export const useAuth = () => {
2    const [user, setUser] =
```

```
3     useState<User | null>(null);
4   const [loading, setLoading] =
5     useState(true);
6
7   const { data, loading: queryLoading } =
8     useQuery(ME_QUERY, {
9       fetchPolicy: "network-only",
10      onError: () => setUser(null),
11    });
12
13   useEffect(() => {
14     if (data?.me) {
15       setUser(data.me);
16     }
17     setLoading(queryLoading);
18   }, [data, queryLoading]);
19
20   const login = async (
21     phone: string, password: string
22   ) => {
23     const { data } = await loginMutation({
24       variables: { phone, password },
25     });
26     setUser(data.loginUser.user);
27   };
28
29   const logout = async () => {
30     await logoutMutation();
```

```
31     setUser(null);  
32     client.clearStore();  
33 };  
34  
35 return { user, loading, login, logout };  
36 };
```

Код В.2: useAuth custom hook

С. ӨГӨГДЛИЙН ЗАГВАР — НЭМЭЛТ ЗАГВАРУУД

Энэ хавсралтад үндсэн бүлэгт дэлгэрэнгүй ороогүй нэмэлт Mongoose загваруудын жишээг нэгтгэн харуулав. Гол collection-уудын ангиллыг 3-р бүлгийн өгөгдлийн сангийн зохиомжийн хэсэгт тусгасан.

C.1 Vital Signs загвар

```
1 const VitalSignSchema = new mongoose.Schema({
2   _id: { type: mongoose.Schema.Types.ObjectId,
3     auto: true },
4   visitId: {
5     type: mongoose.Schema.Types.ObjectId,
6     ref: "Visit", required: true
7   },
8   patientId: {
9     type: mongoose.Schema.Types.ObjectId,
10    ref: "Patient", required: true
11  },
12  temperature: { type: Number },
13  bloodPressureSystolic: { type: Number },
14  bloodPressureDiastolic: { type: Number },
15  heartRate: { type: Number },
16  respiratoryRate: { type: Number },
17  oxygenSaturation: { type: Number },
18  weight: { type: Number },
```

```
19 height: { type: Number },
20 recordedBy: {
21   type: mongoose.Schema.Types.ObjectId,
22   ref: "User"
23 },
24 }, { collection: "vital_signs",
25   timestamps: true });
```

Код C.1: VitalSign model

C.2 Diagnosis загвар

```
1 const DiagnosisSchema = new mongoose.Schema({
2   _id: { type: mongoose.Schema.Types.ObjectId,
3     auto: true },
4   visitId: {
5     type: mongoose.Schema.Types.ObjectId,
6     ref: "Visit", required: true
7   },
8   patientId: {
9     type: mongoose.Schema.Types.ObjectId,
10    ref: "Patient", required: true
11  },
12  doctorId: {
13    type: mongoose.Schema.Types.ObjectId,
14    ref: "Staff"
15  },
16  icdCode: { type: String },
17  name: { type: String, required: true },
```

```
18 description: { type: String },
19 type: { type: String,
20   enum: ["primary", "secondary",
21     "differential"],
22   default: "primary"
23 },
24 severity: { type: String,
25   enum: ["mild", "moderate",
26     "severe", "critical"]
27 },
28 notes: { type: String },
29 }, { collection: "diagnoses",
30   timestamps: true });
```

Код C.2: Diagnosis model

C.3 Prescription загвар

```
1 const PrescriptionItemSchema =
2   new mongoose.Schema({
3     medicationName: { type: String,
4       required: true },
5     dosage: { type: String, required: true },
6     frequency: { type: String, required: true },
7     duration: { type: String },
8     quantity: { type: Number },
9     unit: { type: String },
10    instructions: { type: String },
11  }, { _id: false });
```

```
12
13 const PrescriptionSchema =
14   new mongoose.Schema({
15     _id: { type: mongoose.Schema.Types.ObjectId,
16       auto: true },
17     visitId: {
18       type: mongoose.Schema.Types.ObjectId,
19       ref: "Visit", required: true
20     },
21     patientId: {
22       type: mongoose.Schema.Types.ObjectId,
23       ref: "Patient", required: true
24     },
25     doctorId: {
26       type: mongoose.Schema.Types.ObjectId,
27       ref: "Staff", required: true
28     },
29     prescriptionNumber: { type: String,
30       required: true, unique: true },
31     items: [PrescriptionItemSchema],
32     notes: { type: String },
33     status: { type: String,
34       enum: ["active", "dispensed",
35         "cancelled"],
36       default: "active"
37     },
38   }, { collection: "prescriptions",
39     timestamps: true });
```

Код C.3: Prescription model

C.4 Audit Log загвар

```
1 const AuditLogSchema = new mongoose.Schema({
2   _id: { type: mongoose.Schema.Types.ObjectId,
3     auto: true },
4   userId: {
5     type: mongoose.Schema.Types.ObjectId,
6     ref: "User"
7   },
8   action: { type: String,
9     enum: ["create", "update", "delete",
10      "login", "logout", "access", "export"],
11     required: true
12   },
13   resource: { type: String, required: true },
14   resourceId: { type: String },
15   details: { type: Object },
16   ipAddress: { type: String },
17   userAgent: { type: String },
18 }, { collection: "audit_logs",
19   timestamps: true });
20
21 // TTL: auto-delete after 1 year
22 AuditLogSchema.index(
23   { createdAt: 1 },
```

```
24 { expireAfterSeconds: 365 * 24 * 60 * 60 }  
25 );
```

Код C.4: AuditLog model